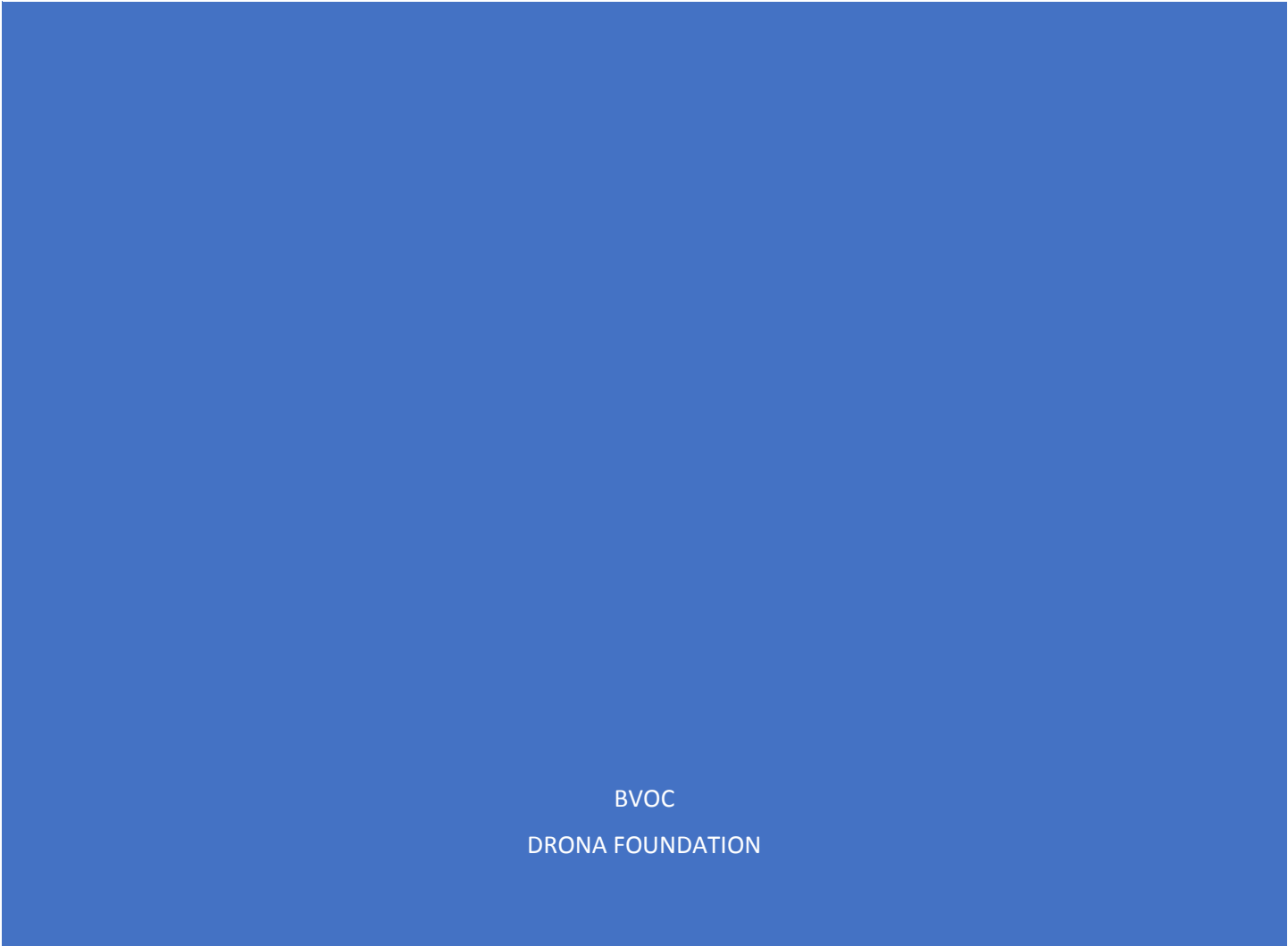




LEARN KOTLIN



BVOC
DRONA FOUNDATION

Sr No	INDEX
1	INTRODUCTION TO KOTLIN
1.1	What is Kotlin?
1.2	First Kotlin Program
1.3	The main() Function
1.4	Print Statements
1.5	Order of Execution
1.6	Comments
1.7	Errors and Bugs
1.8	Formatting Conventions
1.9	Review and Resources
	Article : From Editor to Output
	Concept Review
1	QUIZ
1	PROJECT : ASCII ART
2	DATA TYPES & VARIABLES
2.1	Introduction
2.2	Declaring a Mutable Variable
2.3	Declaring an Immutable Variable
2.4	Type Inference
2.5	Review
	Concept Review
2	QUIZ

2	PROJECT : MAD LIBS
3	PROJECT : MATH MAGIC
3	USING TEXT VARIABLES
3.1	Strings & Characters
3.2	String Concatenation
3.3	String Templates
3.4	User Input
3.5	Built-in Properties and Functions
3.6	Character Escape Sequences
3.7	Review
4	USING NUMBER VARIABLES
4.1	Number Types
4.2	Arithmetic Operators
4.3	Order of Operations
4.4	Augmented Assignment Operators
4.5	Increment and Decrement Operators
4.6	The Math Library
4.7	Review
5	CONDITIONAL EXPRESSIONS
5.1	Introduction to Conditionals
5.2	if Expression

5.3	if/else Expression
5.4	Comparison Operators
5.5	Equality and Inequality Operators
5.6	else-if Expressions
5.7	Logical Operators
5.8	Order of Evaluation
5.9	Nested Conditionals
5.10	when Expressions
5.11	Range
5.12	Review
	Concept Review
5	QUIZ
4	PROJECT : SELF GRADING QUIZ
6	LISTS
6.1	Introduction to Lists
6.2	Creating an Immutable List
6.3	Accessing Elements in a List
6.4	Creating a Mutable List
6.5	The size Property
6.6	List Functions
6.7	Review
	Concept Review

7	SETS
7.1	Introduction to Sets
7.2	Creating an Immutable Set
7.3	Accessing Elements in a Set
7.4	Creating a Mutable Set
7.5	Adding and Removing Elements
7.6	Additional Set Functions
7.7	Review
8	MAPS
8.1	Introduction to Maps
8.2	Creating an Immutable Map
8.3	Retrieving Keys and Values
8.4	Creating a Mutable Map
8.5	Adding and Removing Entries
8.6	Review
8	QUIZ
5	PROJECT : ANIMAL SANCTUARY
9	LOOPS
9.1	Introduction to Loops
9.2	for Loop
9.3	Controlling the Iteration
9.4	Iterating Through Collections

9.5	Iterating Through Maps
9.6	while Loop
9.7	do..while Loop
9.8	Nested Loops
9.9	Jump Expressions
9.10	Labeled Jump Expressions
9	QUIZ
6	PROJECT : SHAPE MAKER
10	FUNCTIONS
10.1	Introduction to Functions
10.2	Creating and Calling a Function
10.3	Arguments
10.4	Named and Default Arguments
10.5	Return Statements
10.6	Single Expression Functions
10.7	Function Literals
10.8	Review
	Article : Recursion
	Concept Review
11	CLASSES
11.1	Introduction to Classes
11.2	Creating a Class

11.3	Creating an Instance
11.4	Primary Constructor
11.5	The Init Block
11.6	Member Functions
11.7	Review
	Concept Review
11	QUIZ
8	PROJECT : CODEY'S CALCULATOR

1. INTRODUCTION TO KOTLIN

1.1 What is Kotlin?

Welcome to the world of coding with Kotlin!

Kotlin is a programming language that, after its release in 2016 by [JetBrains](#), quickly gained popularity amongst developers due to its concise syntax, additional safety features, and compatibility with the Java programming language.

Being a general-purpose programming language, Kotlin can be used for Android development, web development, data science, and more. In 2019, [Google announced Kotlin as their preferred language](#) for Android development.

The following applications use Kotlin in their tech stack:

- Netflix
- Duolingo
- Pinterest
- And more!

Throughout this lesson, we'll learn about Kotlin syntax and how to run a basic program.

1.1 What is Kotlin? Instructions

Take a look at the image to the right. Codey is starting on the path to Dev City which is filled with mobile apps, websites, and other applications.

In order to reach Dev City, one must journey through various landscapes like Variable Valley, Conditional Creek, Land of Loops, and Function Forest.

In fact, this journey to Dev City is quite similar to the sequence of topics you'll encounter throughout this course. By completing each section of this course, you'll have gained a strong understanding of programming fundamentals in Kotlin.

When you're ready to continue, hit **Next**.



1.2 First Kotlin Program

What better way to start learning about Kotlin than to run a Kotlin program? Before we run our first program, let's take a moment to understand our current environment.

Code is written and edited in something called a code editor. The code editor for a Codecademy exercise lives in the middle of the screen. If we look at the top of the editor, we will see the file name **Hello.kt**. The **.kt** extension signifies that the file contains Kotlin code.

In our code editor, there is a short Kotlin program:

```
fun main() {  
    println("Hello, World!")  
}
```

Don't worry about understanding what this code means yet. All this program does is print the text "Hello, World!" to the output terminal, which exists on the right side of our screen. When we click **Run**, the output terminal displays our code's final output.

We'll be checking the output terminal a lot to ensure our code works as expected.

1.2 First Kotlin Program Instructions

1.

Hit **Run** to execute our first Kotlin program.

Hello.kt

```
fun main() {  
    println("Hello, World!")  
}
```

OUTPUT

Hello, World!

1.3 The main() Function

Think of writing code like going on an adventure: we always need to start somewhere.

In Kotlin, the journey begins in the `main()` function.

The `main()` function is the “jumping off” point for our code and is the first part of our program that will execute when we run our program. It will exist in every single Kotlin project we create; in fact, without a `main()` function present, the program will return an error.

The syntax for the `main()` function looks like this:

```
fun main() {  
    // Code will go here  
}
```

- `fun` is a Kotlin keyword that is used to declare a function.
- `main` is the required name of the starting function.
- Following the function name is a pair of empty parentheses `()`.

Code that is written inside the curly brackets `{ }` is considered the “body” of a function. When we hit Enter to create a new line of code inside the body, the code will automatically indent itself:

```
fun main() {  
    println("Hello, World!")  
}
```

1.3 The `main()` Function Instructions

1.

In **Main.kt**, create the `main()` function.

Leave the body empty for now.

2.

Add the following code to the body of the `main()` function:

```
println("Coding is fun!")
```

OUTPUT

Coding is fun!

1.4 Print Statements

In the last exercise, we were able to display the text Coding is fun! in the output terminal with the following code:

```
fun main() {  
    println("Coding is fun!")  
}
```

We accomplished this feat using a print statement. A *print statement* outputs values to the terminal.

When we want to use a print statement in Kotlin, we use the `println()` function, which prints a value before creating a new line in the output terminal:

```
println("To infinity and beyond!")
```

Any value we want to print needs to be placed within the parentheses of the `println()` statement. If we want to output text, we need to contain the text in double quotes (`""`). The code in the snippet above will output:

To infinity and beyond!

We can even use print statements to output number values and the result of expressions. For example:

```
println(5 + 5)
```

Will have an output of:

We'll mainly use `println()` throughout this course; however, there is another option for printing values to the output terminal: `print()`.

The difference between `println()` and `print()` is that `println()` creates a new line after outputting a value, while `print()` doesn't. For example:

```
println("One")  
print("Two")  
print("Three")
```

Will output:

```
One  
TwoThree
```

1.4 Print Statements Instructions

1.

In **Print.kt**, use `println()` to output "Just keep swimming!"

2.

In **Print.kt**, use `print()` to output the expression `15 * 4`.

Print.kt

```
fun main() {  
    // Write your code below!
```

```
}
```

OUTPUT

Just keep swimming!

60

1.5 Order of Execution

Similar to instructions in a recipe, code is read, compiled, and executed in a top-down order.

When we run a program, a [compiler](#) translates our Kotlin code into something that can be read by the Java Virtual Machine (JVM). The JVM is responsible for executing code as instructions.

All of this happens behind the scenes between running our code and seeing its output in the terminal.

Inside the `main()` function, the top line of the code is the first instruction that gets read by the compiler. For example, let's create a program that contains three print statements:

```
fun main() {  
    println("Huzzah!")  
    println("Woohoo!")  
    println("Yehaw!")  
}
```

In the snippet above, the print statement containing "Huzzah!" is the first line of code in the `main()` function; therefore, it will be printed to the output terminal first followed by "Woohoo!" and then "Yehaw!":

Huzzah!
Woohoo!
Yehaw!

1.5 Order of Execution Instructions

1.

Use `println()` to create a timeline for [The Great Emu War of 1932](#).

Use the following text to create 3 print statements:

- "11/2 - AU military goes to war against local emus."
- "11/4 - Ambush planned by military fails."
- "11/8 - AU military withdraws."

The content of the print statements must appear in the correct time order.

Order.kt

```
fun main() {  
    // Add your code below  
  
}
```

OUTPUT

11/2 - AU military goes to war against local emus.
11/4 - Ambush planned by military fails.
11/8 - AU military withdraws.

1.6 Comments

Documenting what our code is accomplishing is a good programming practice because it makes our code understandable for both the programmer and anyone else who reads it.

Comments allow us to write documentation inside of our programs. [Comments](#) can be written anywhere throughout a program and can contain text, symbols, or even code - all of which will be ignored by the compiler.

To write a single-line comment, type `//` into the program followed by the comment we want to add:

```
// To be or not to be: that is the question.
```

`//` symbolizes the start of a comment. Note how text that appears in a comment has a gray font-color.

We also have the option of using a *multiline comment* which allows us to write comments throughout multiple lines. Multiline comments start with `/*` and end with `*/`:

```
/*  
If you prick us, do we not bleed?  
If you tickle us, do we not laugh?  
If you poison us, do we not die?  
And if you wrong us, shall we not revenge?  
  
- William Shakespeare, The Merchant of Venice  
*/
```

1.6 Comments Instructions

1.

Add the following text as a single-line comment in **Comments.kt**:

Code prints "Howdy, partner!" to the terminal.

2.

Create a multiline comment that describes what you have learned so far about programming with Kotlin.

Comments.kt

```
fun main() {  
    println("Howdy, partner!")  
}
```

OUTPUT

Howdy, partner!

1.7 Errors and Bugs

While every programmer is different, there is one experience we all share: getting an error message. An error, or bug, causes our programs to behave in unexpected ways.

When we receive an error message, it's almost always because of something we wrote incorrectly in our code. As a result, the compiler cannot translate our program and returns an error message.

There are many different kinds of errors we can encounter. This may sound daunting, but luckily the Kotlin compiler provides accurate and descriptive error messages, allowing us to quickly and efficiently locate the issue.

For example, take a very close look at the following code:

```
fun main() {  
    printlnn("To err is human.")
```

```
}
```

When we run this, we receive the following error:

```
Error.kt:2:3: error: unresolved reference: printlnn  
printlnn("To err is human.")  
  ^
```

This error message provides a lot of useful information:

- Error.kt is the name of the file that contains the bug.
- The first number (2) tells us the line in which the error exists.
- The second number (3) tells us the character number in which the error lives on that line.
- The text following error: describes what error we are encountering.

We received this error because we wrote `printlnn` instead of `println` and the compiler couldn't recognize the instruction.

Once we fix and rerun our code, we get the following output:

To err is human.

1.7 Errors and Bugs Instructions

1.

Run the code in **Error.kt**.

We will receive an error message.

2.

Read the error message and fix the code in the editor so that it can execute properly.

PS: Check the hint for additional help!

Error.kt.

```
fun main() {  
    println("An error a day keeps the programmer away.")  
}
```

OUTPUT

An error a day keeps the programmer away.

1.8 Formatting Conventions

In programming, formatting conventions are the suggested rules for how code should be structured in a specific programming language. By following Kotlin's formatting standards, or coding conventions, we can drastically improve the readability of our programs.

Some of Kotlin's formatting conventions include:

Indentation:

Code within curly braces should be indented 4 spaces. While the Codecademy code editor generally uses 2 spaces, or 1 tab, for indentation, be sure to use 4 spaces when writing Kotlin code off-platform.

```
fun main() {  
    // Code starts here  
}
```

If we didn't include indentation in our program, our code would still run without error; however, our code would look like a giant block of text making it hard to read. As programs get larger and more complicated, indentation for the purpose of organization becomes more essential.

Comments:

When writing a comment, add a space (" ") between `//` and the start of the comment:

```
// Write your comment here
```

Curly Brackets:

When creating the `main()` function, place the opening curly bracket `{` on the same line as the `main()` function construct (in this case: `fun main()`). The closing bracket `}` should be placed on its own separate line and should be vertically aligned with the `fun` keyword:

```
fun main() {  
    // Code goes here  
}
```

Keep the above conventions in mind when writing Kotlin code throughout this course and off-platform. We'll encounter more formatting conventions as we progress through this course. To learn more, check out the coding conventions in [Kotlin's Language Guide](#).

1.8 Formatting Conventions Instructions

Take a look at the code in **Syntax.kt**.

It's unformatted but produces no error. Take some time to format the code using different coding conventions.

Click **Next** when you're ready to move on.

Syntax.kt

```
fun main()
{
    //    Organize the code below
    println("Down by the bay")
    println("Where the watermelons grow,")
        println("Back to my home")
    println("I dare not go.");
        println("For if I do");
    println("My mother will say")
        println("Did you ever use an app that taught you how to snap?")
    println("Down by the bay!")
}
```

OUTPUT

Down by the bay

Where the watermelons grow,

Back to my home

I dare not go.

For if I do

My mother will say

Did you ever use an app that taught you how to snap?

Down by the bay!

1.9 Review and Resources

Great job completing the first lesson on learning Kotlin! Let's go over all the content we have discussed:

- Kotlin is a general-purpose programming language.
- The `main()` function is the starting point of all Kotlin programs.
- When we want to output something to the terminal, we can use print statements like `println()` or `print()`.
- Code is read from top to bottom.
- [Comments](#) are used to explain what is happening in our code.

Language and syntax and rules - oh my! Learning a new language can feel daunting; luckily, there are many online resources available to us for the Kotlin language:

- The [Kotlin Language Guide](#) contains the official documentation for Kotlin.
- The [Kotlin's Language Guide on Formatting](#) discusses Kotlin's coding conventions and formatting standards.
- The [Kotlin Style Guide](#) contains Google's Android coding standards for Kotlin.
- The [Codecademy Forum](#) is a great resource and friendly place where learners can view and ask coding questions.

These language guides and resources are invaluable to programmers, and we highly encourage keeping them at hand and referencing frequently when developing in Kotlin.

1.9 Review and Resources Instructions

Take some time to practice using Kotlin in **Review.kt**:

Here are some ideas on what you can do:

- Create the `main()` function.
- Print your age by subtracting the current year and your birth year.
- Print your favorite quote.
- Add comments about what your code is accomplishing.

When you're ready to move on, click **Up Next**.

From Editor to Output

Learn what happens behind the scenes when you Run a Kotlin program.

How does a program work? From our perspective, we write some code, run it, and then something (usually) appears in the output terminal; however, that isn't the full story. In this article, we're going to break down all of the work and processes that happen between running our Kotlin code and seeing its output.

Behind the Scenes

When we run a program, a series of processes work together to evaluate and format the code as needed. This process varies by language, but in Kotlin it looks like this:

First, the compiler evaluates our entire program and ensures that it does not contain any errors. It then translates our source code to Java bytecode in the form of **.class** files. These files then get passed on to the JVM.

JVM stands for Java Virtual Machine and is the software responsible for executing Java bytecode given to it from a compiler. The JVM doesn't understand programming languages like Kotlin; instead, it speaks to and analyzes bytecode. The JVM then executes these instructions in the order they were given. Once the

JVM completes the final instruction, it terminates - meaning our program has been fully executed.

What happens when there's an error?

Since Kotlin is a statically compiled language, its code gets translated and its types are checked before run time, specifically during compilation. Any errors in our code get caught during this phase. If the compiler recognizes an error, it displays an error message in the output terminal and terminates the rest of the processes as shown below:

The bytecode is never generated, the JVM is never reached, and finally, our code's output is not displayed. Assuming we have an error such as this one:

Error.kt:2:25: error: expecting ')'

```
println("Hello, World"
```

^

Until we resolve this error by adding a `)` and rerunning our code, we will not be able to see any output.

No Kotlin supported IDE? No problem!

In the case that we're off the Codecademy platform and writing our Kotlin code locally in an [IDE](#) without a built-in converter between Kotlin and Java code, we can rely on the bash terminal.

The bash terminal is a command language interpreter that has the ability to execute a series of commands contained in a script or program. With Kotlin installed, we can run a series of commands which will manually compile our code, execute it, and display its output. Take a look at how it's done manually.

Assume we have the following program in a **HelloWorld.kt** file:

```
fun main() {  
    println("Hello, World!")  
}
```


In the bash terminal, we can enter the first command to compile our code and save it in a **.jar** file:

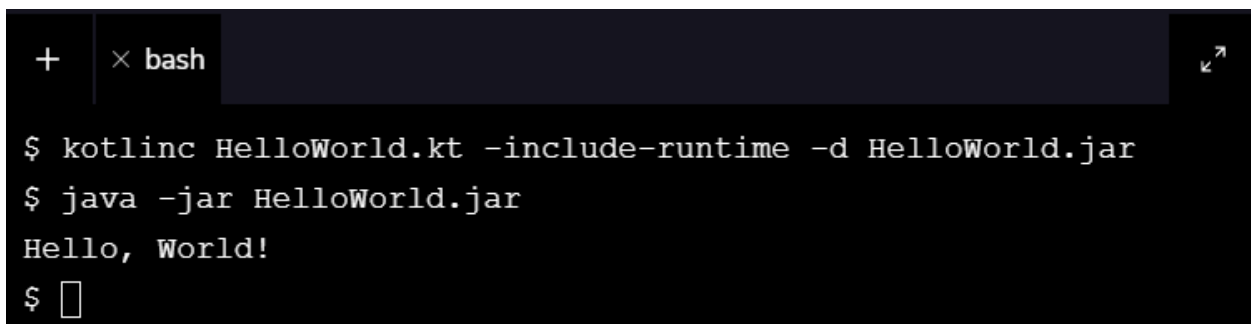
```
$ kotlinc HelloWorld.kt -include-runtime -d HelloWorld.jar
```

- **kotlinc** represents the compiler.
- **HelloWorld.kt** represents the source code file that we need to compile into Java bytecode.
- The **-include-runtime** option results in the **.jar** file being runnable by including the Kotlin runtime library within it.
- The **.jar** extension represents a file format that indicates many Java files zipped into one. When executed, Java can extract and use its information.
- The **-d** flag specifies the output path for the newly generated class files.

After we've hit enter, we can then run our program with the following command:

```
java -jar HelloWorld.jar
```

- The **java** command launches an application written in Java. It does this by starting the JVM, loading the **.class** file, and executing the code within the **main()** function.
- By specifying the **-jar** flag, we are instructing Java to execute a program within a **.jar** file.

A screenshot of a terminal window with a dark background. The terminal title bar shows a '+' icon, a 'x' icon, and the text 'bash'. The terminal content shows the following sequence of commands and output:

```
$ kotlinc HelloWorld.kt -include-runtime -d HelloWorld.jar
$ java -jar HelloWorld.jar
Hello, World!
$
```

Note: If we make changes to our code, we'd need to recompile our program and run the above two commands again.

You've reached the end of the article - great job! We covered a vast majority of information that gives us insight into what happens after we run our Kotlin code.

We now understand that our Kotlin code needs to be compiled into bytecode before it can be passed on to the Java Virtual Machine which executes the code as instructions. IDEs often automatically compile our code, but if we don't have that luxury, we can use a bash terminal to manually compile and execute our program. Next time you're running your code, you'll know what's happening from editor to output. And if you want to learn more about how to use your bash terminal, check out our [Bash courses](#).

CONCEPT REVIEW

The main() Function

The main() function is the starting off point of every Kotlin program and must be included in the code before execution.

```
fun main() {  
    // Code goes here  
}
```

Print Statements

A print statement outputs values to the output terminal.

Using a println() statement will create a new line in the output terminal after printing values, while a print() statement will not.

```
println("Greetings, earthling!")
```

```
print("Take me to ")
```

```
print("your leader.")
```

```
/*
```

Prints:

Greetings, earthling!

Take me to your leader.

```
*/
```

Comments

Comments are used to document what is happening inside of a program and are ignored by the compiler.

A single-line comment is used to write a comment on one line, while a multiline comment is used to write longer comments that span over multiple lines.

```
// This is a single-line comment
```

```
/*
```

This

comment

uses

multiple

lines

```
*/
```

Order of Execution

Code is read, compiled, and executed in top-down order.

```
fun main() {  
    println("I will be printed first.")  
    println("I will be printed second.")  
    println("I will be printed third.")  
}
```

1. Quiz

1. Fill in the code below to create a program that prints, "**Salutations, good fellow!**".

```
Fun _____ () {  
    _____ ("_____")  
}
```

- outputLine
- Salutations, good fellow!
- println
- start
- main
- Goodbye, friend

Click or drag and drop to fill in the blank

2. True or False: The program below will run without an error.

```
fun main() {  
    println(Hello, World)  
}
```

- a) False
- b) True

3. Where is the starting point of a Kotlin program?

- a) The start() function.
- b) The first() function.
- c) The Kotlin() function.
- d) The main() function.

4. What will the following program output?

```
fun main() {  
    println(1)  
    println(2 * 2)  
    println(10 - 1)  
}
```

- a) 9
4
1
- b) Nine
Four
One
- c) One
Four

Nine

- d) 1
- 4
- 9

5. Which of the following options contain a single-line Kotlin comment?

- a) / I am a comment /
- b) // I am a comment.
- c) /*
I
am
a
comment
*/
- d) # I am a comment

6. After running this program, which of the following choices will appear first in the output terminal?

```
fun main() {  
    // println("First")  
    println("Second")  
    println("Third")  
    println("Fourth")  
}
```

- a) Third
- b) First
- c) Second
- d) Fourth

1. Project

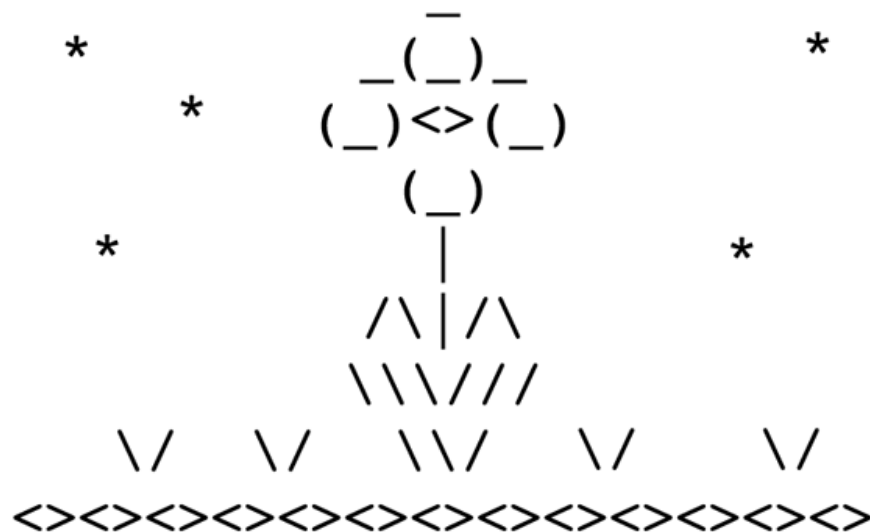
ASCII Art

Programming and art are often viewed as opposites, but that couldn't be farther from the truth. Coding can be a great medium for artistic creativity! One example of this is ASCII art.

ASCII stands for American Standard Code for Information Interchange. Its general purpose is to give every character, from letters to numbers to symbols, its own unique character code so that the character can be recognized in a digital space.

It can also be used to create art!

ASCII art is a drawing technique that uses a combination of different keyboard characters to create both simple and complex images:



The ASCII image above is made up of several characters; for example, the petals on the flower were created using (, , and).

In this project, we will be using print statements to create our own ASCII art image in Kotlin!

TASKS

Planning the Art

1.

Decide on an image you would like to recreate via ASCII art.

For example:

- Robot
- Plant
- Cat

2.

Before we try to use print statements, let's draft our ASCII drawing in the editor.

Create a multiline comment. Inside the comment, use different letters, numbers, and symbols to draft out your image.

Creating the Program

3.

Now that we have some ASCII art, let's create a program that outputs our image to the terminal!

First, create the `main()` function. We'll add code to the `main()` function body in the next step.

4.

Now, we'll create our print statements.

For every line in the ASCII art image, use `println()` to create a print statement that contains all the values that exist in that line of art.

5.

Once all the print statements are created, run the program and see how the drawing looks!

If something doesn't look right, now is the time to go back into the code to make the proper edits.

Keep editing the program until you feel satisfied with the final result.

Optional

6.

Great job completing the project!

If you want to keep challenging yourself, check out these additional tasks:

- Add a background to your ASCII image.
- Recreate the ASCII art using only one print statement. To accomplish this, you'll need to use `\n`. Check out the hint below to learn more!

2. DATA TYPES & VARIABLES

2.1 Introduction

In this lesson, we'll explore the building blocks of a programming language - [data types](#) and [variables](#). A *data type* represents the type of value an object possesses. Here are a few of the types of data supported in Kotlin:

- **String**: a sequence of characters enclosed in double quotes
- **Integer** (Int): whole numbers and their opposites
- **Boolean**: true or false values

Each variable in a Kotlin program will possess one of these types of data. Think of a variable as a unique, virtual container holding a certain piece of information that can then be used and referenced throughout a program.

Take a look at the animation on the right. Each item represents a variable holding a specific type of data dependent on its value. Codey packs their backpack with a lunchBox holding the value "sandwich" and a waterBottle with 32 ounces of water. tentPacked holds a Boolean value which represents if the tent is in Codey's backpack. With these values, we can then write a program that checks if Codey has everything they need for their journey.

This is just one of many programs we can build with these variables. Each variable would have its own place and purpose in a program, and their respective data types determine how we can interact with them.

2.1 Introduction Instructions

Click **Up Next** to learn more about variables and how they're used in Kotlin.



2.2 Declaring a Mutable Variable

Now that we've familiarized ourselves with the concept of [variables](#), let's learn how to create a variable in Kotlin. A complete variable declaration and initialization resembles the following form:

```
var variableName: Type = value
```

Declaration:

- The `var` keyword specifies the beginning of a variable declaration. It signifies a mutable variable which means its value can change throughout the program.
- Following the keyword is the name of the variable in camelCase format. camelCase refers to a naming convention where the first word is lowercased, the first letter of the second is capitalized, and so on. It's important to create concise, yet descriptive variable names for readability and maintainability. A famous software engineer once said:

“..I will cheerfully spend literally hours on identifier names: variable names, method names, and so forth, to make my code readable...” - Joshua Bloch

- The variable name is then followed by the type of data that the variable is intended to store. The type must always be capital.

2.2 Declaring a Mutable Variable Initialization:

- Once the variable has been declared, we can initialize it with a value. The = sign, known as an assignment operator in programming, assigns a value to the variable.

Let's apply this syntax to our first Kotlin variable:

```
var guitarName: String = "Fender Stratocaster"
```

The guitarName variable has been allocated a position in memory by the compiler and can now be referenced and used throughout our program.

The compiler also recognizes that this variable is mutable due to the var keyword ahead of the variable name; let's explore what that means.

We may sometimes need to declare a variable in our program, knowing it will be of use later on, but don't have a value to initialize it with just yet. We can do so in this way:

```
var variableName: Type
```

Assume we're building a music application that will listen for the notes played on a guitar. We can declare a notePlayed variable and initialize it with a value when the user plays a note:

```
var notePlayed: String // declaration
```

```
notePlayed = "B" // initialization
```

```
println(notePlayed) // Prints: B
```

Notice how we did not need to use the var keyword on the second line as it is only used upon initial declaration. We've also omitted the type of data since this information needs to only be specified once.

2.2 Declaring a Mutable Variable Initialization Instructions

1.

Within the main() function of **Date.kt**, declare a String variable named todaysDate and initialize it with today's date using the following form:

mm/dd/yyyy

On the following line, add a println() statement and output the value of todaysDate.

2.

On the following line, declare another String variable, currentWeather. This time we will initialize the value separately, on the next line.

- Take a peek outside, and then assign a weather description to currentWeather.
- Add a println() below the initialized value and output currentWeather.

Date.kt

```
fun main() {  
  
    // Write your code below ☀️  
  
}
```

OUTPUT

11/05/2023

Sunny with a few clouds

2.3 Declaring an Immutable Variable

Another type of variable that exists in Kotlin is the immutable variable. The term, immutable, signifies a variable whose value cannot mutate or change after its initialization.

Immutable variables are denoted by the `val` keyword and are often referred to as constants in some languages. Here's how they're declared in Kotlin:

```
val variableName: Type = value
```

Let's see an example. The following variable contains the value of the [Golden Ratio](#):

```
val goldenRatio: Double = 1.618
```

Since we know this value stays constant, it's best to declare it with the `val` keyword as opposed to `var` as we don't intend on changing it. Although a declaration with `var` would have also been accepted by the compiler, the preferred strategy is to declare [variables](#) with `var` *only* if we're certain the value will change somewhere throughout the program, otherwise always use `val`.

Let's see what happens when we try to reassign `goldenRatio` to another value:

```
goldenRatio = 3.2
```

The above line of code would result in the following error in the terminal stating that reassignment is not allowed:

```
Pi.kt:5:3: error: val cannot be reassigned
```

```
goldenRatio = 3.2
```

```
^
```

2.3 Declaring an Immutable Variable Instructions

- 1.

In **Pi.kt**, declare an immutable variable, `pi`.

- It must have a type of `Double`
- It must hold the value, `3.14`

Note: Since we haven't utilized this variable and only declared it, you'll see a Warning message thrown by the compiler stating that the variable hasn't been used. A Warning is different from an Error and doesn't prevent execution of your program. Feel free to disregard this for now; it is just Kotlin's way of ensuring that we don't have any unused code in our programs.

2.

On the following line, type the variable name and reassign it a different `Double` value. Observe the error in the terminal.

```
fun main() {  
    // Write your code below  
  
}
```

OUTPUT

Pi.kt:5:3: error: val cannot be reassigned

```
pi = 5.5
```

2.4 Type Inference

In Kotlin, we can save keystrokes and optimize our variable declarations with a feature known as type inference.

Type inference indicates that the compiler can infer the type of a declared variable, and its data type can be omitted in the declaration. Take a look at the following variable declaration:

```
var lightsOn: Boolean = true
```

We're explicitly stating the type of variable, but the Kotlin compiler is also intelligent enough to make this assumption even with the absence of a type:

```
var lightsOn = true // valid declaration
```

The compiler recognizes that a true or false value stored in a variable falls under the Boolean data type, thus the lightsOn variable is a Boolean.

It's important to know that regardless of how a variable is declared, its type cannot change throughout a program. A Boolean variable, whether explicitly stated or inferred, can *only* hold a true or false value.

If we try to assign a value that's not a Boolean:

```
lightsOn = "no" // error
```

The compiler will throw the following error:

```
Inference.kt:4:14: error: type mismatch: inferred type is String but Boolean was expected
```

```
lightsOn = "no"
```

Note: Variable declarations with explicitly specified [data types](#) and those with inferred data types are both valid ways in creating [variables](#). Each syntax will be accepted by the compiler, thus choosing one or the other is simply up to the developer's preference.

2.4 Type Inference Instructions

1.

In **Inference.kt**, we've added a print() statement with some Kotlin code within the main() function that outputs the inferred data type for a variable, called typeTest. Don't worry about understanding exactly how this code works

now, we'll cover what these symbols and names mean later in the course. For now, we'll just use it to show the type of data that was inferred by the compiler.

Above the `print()` statement, declare `typeTest` and initialize it with any value from this list:

- "6"
- 6
- true
- "false"
- 2.6

Observe the inferred type in the terminal. Was it what you expected?

Assign and test a few other values for `typeTest`.

Inference.kt

```
fun main() {  
  
    // Declare your variable above ↑  
    print("${typeTest::class.simpleName}")  
}
```

OUTPUT

String

2.5 Review

Excellent job! You've completed the first lesson of the [Data Types](#) & [Variables](#) section. Let's review what was covered.

- A variable declaration consists of a `val` or `var` keyword followed by a name, colon and data type.
- A variable initialization occurs when a value is assigned to a variable using the assignment operator (`=`).
- A variable declaration and initialization can happen simultaneously on one line or separately.
- A mutable variable is denoted by the `var` keyword and represents a value that is expected to change throughout a program.
- An immutable variable is denoted by the `val` keyword and represents a constant value.
- Kotlin can infer a variable's type through type inference.

Feel free to utilize the empty **Review.kt** file and output terminal on the right to hone your understanding of variables and data types and practice writing Kotlin code.

2.5 Review Instructions

Here are some ways in which you can practice your new knowledge:

- Declare a `String` variable with the name of your favorite song.
- Declare an `Int` variable with the minute duration of the song.
- Declare a `Boolean` variable and assign it `true` or `false` depending on if you're currently listening to your favorite song!
- Output these values to the terminal.

Think about which of these variables should be declared as mutable or immutable. If you've declared them with specified types, try omitting the types and shortening your code.

CONCEPT REVIEW

Mutable Variables

A mutable variable is declared with the `var` keyword and represents a value that is expected to change throughout a program.

```
var age = 25
```

```
age = 26
```

Immutable Variables

An immutable variable is declared with the `val` keyword and represents a value that must remain constant throughout a program.

```
val goldenRatio = 1.618
```

Type Inference

When a data type is not specified in a variable declaration, the variable's data type can be inferred through type inference.

// The following variable is assigned a text value within double quotes, thus the inferred type is `String`

```
var color = "Purple"
```

String Concatenation

String concatenation is the process of combining `Strings` using the `+` operator.

```
var streetAddress = "123 Main St."
```

```
var cityState = "Brooklyn, NY"

println(streetAddress + " " + cityState)

// Prints: 123 Main St. Brooklyn, NY
```

String Templates

String templates contain String values along with variables or expressions preceded by a \$ symbol.

```
var address = "123 Main St. Brooklyn, NY"

println("The address is $address")

// Prints: The address is 123 Main St. Brooklyn, NY
```

Built-in Properties and Functions

The Kotlin String and Character data types contain various built-in properties and functions. The length property returns the number of characters in a String, and the capitalize() function capitalizes the first letter of a String.

```
var monument = "the Statue of Liberty"

println(monument.capitalize()) // Prints: The Statue of Liberty
println(monument.length) // Prints: 21
```

Character Escape Sequences

Character escape sequences consist of a backslash and character and are used to format text.

- `\n` Inserts a new line

- `\t` Inserts a tab
- `\r` Inserts a carriage return
- `\'` Inserts a single quote
- `\"` Inserts a double quote
- `\\` Inserts a backslash
- `\$` Inserts the dollar symbol

```
print("\tExcellent!\t" I cried. \Elementary,\" said he.)
```

```
// Prints: "Excellent!" I cried. "Elementary," said he.
```

Arithmetic Operators

The arithmetic operators supported in Kotlin include + addition, - subtraction, * multiplication, / division, and % modulus.

```
5 + 7 // 12
```

```
9 - 2 // 7
```

```
8 * 4 // 32
```

```
25 / 5 // 5
```

```
31 % 2 // 1
```

Order of Operations

The order of operations for compound arithmetic expressions is as follows:

1. Parentheses
2. Multiplication
3. Division

4. Modulus
5. Addition
6. Subtraction

When an expression contains operations such as multiplication and division or addition and subtraction side by side, the compiler will evaluate the expression in a left to right order.

```
5 + 8 * 2 / 4 - 3 // 6
```

```
3 + (4 + 4) / 2 // 7
```

```
4 * 2 + 1 * 7 // 15
```

```
3 + 18 / 2 * 1 // 12
```

```
6 - 3 % 2 + 2 // 7
```

Augmented Assignment Operators

An augmented assignment operator includes a single arithmetic and assignment operator used to calculate and reassign a value in one step.

```
var batteryPercentage = 80
```

```
// Long Syntax
```

```
batteryPercentage = batteryPercentage + 10
```

```
// Short Syntax with an Augmented Assignment Operator
```

```
batteryPercentage += 10
```

Increment and Decrement Operators

Increment and decrement operators provide a shorthand syntax for adding or subtracting 1 from a value. An increment operator consists of two consecutive plus symbols, ++, meanwhile a decrement operator consists of two consecutive minus symbols, --.

```
var year = 2019
```

```
year++ // 2020
```

```
year-- // 2019
```

The Math Library

The Math library, inherited from Java, contains various mathematical functions that can be used within a Kotlin program.

```
Math.pow(2.0, 3.0) // 8.0
```

```
Math.min(6, 9) // 6
```

```
Math.max(10, 12) // 12
```

```
Math.round(13.7) // 14
```

2.Quiz

1. Which choice correctly uses an augmented assignment operator that matches the value of the arithmetic expression below?

```
var volume = 2
```

```
// Turn up the volume 🔊
```

```
volume = volume + 5
```

- a) `year++`
- b) `volume += 5`
- c) `year++ 5`
- d) `year =+ 5`

2. Which data type will the compiler infer from the following variable declaration?

`var learningToCode = true`

- a) Char
- b) String
- c) Int
- d) Boolean

3. What is the order of operations in Kotlin?

- a) Addition, Subtraction, Parentheses, Modulus, Multiplication, Division
- b) Multiplication, Division, Parentheses, Addition, Modulus, Subtraction
- c) Parentheses, Multiplication, Division, Modulus, Addition, Subtraction
- d) Parentheses, Multiplication, Division, Addition, Modulus, Subtraction

4. Which of the following is false about the Math library?

- a) The Math library contains mathematical functions that can be implemented in a Kotlin program.
- b) The Math library is inherited from Java.
- c) Each Math library function begins with the keyword, Math.
- d) The functions in the Math library should be used instead of arithmetic operators.

5. Fill in the code below to display the result of using a String property and a String function on the variable, **funFact**.

```
fun main() {  
    var funFact = "kotlin is named after an island." ?  
  
    println(funFact.length) // Prints: _____  
    println(funFact.capitalize()) // Prints: _____  
  
}
```

- 32
- 6
- 15
- long
- Kotlin is named after an island.
- kotlin is named after an Island.
- Kotlin

Click or drag and drop to fill in the blank

6. Which of the following two data types get assigned by default to numerical variables?

- a) Long, Float
- b) Byte, Float
- c) Int, Double
- d) Short, Double

7. Fill in the correct output for each arithmetic expression.

```
fun main() {  
    println(5 + 6) // Prints: _11_____
```

```
println(1 - 2) // Prints: ____-1____  
println(4 * 5) // Prints: ____20____  
println(9 / 3) // Prints: ____3____  
println(10 % 8) // Prints: ____2____  
}
```

8. Which option is true for mutable variables?

- a) A mutable variable stores a value that cannot be reassigned throughout a program.
- b) A mutable variable is declared with the **val** keyword.
- c) A mutable variable stores a value that can be reassigned throughout a program.
- d) A mutable variable cannot exist in a **main()** function.

9. Which of the following values should NOT be stored in an immutable variable?

- a) The price of gas
- b) The year the first computer was invented
- c) The name of the first human in space
- d) The value of Pi: **3.14**

10. Which of the following choices display the correct use of String concatenation with the given variables?

```
var name = "Grace Hopper"  
var year = 1952  
var invention = "compiler"
```

- a) `println('In ' + year + ', ' + name + ' completed her first ' + invention + '.')`
- b) `println("In $year, $name completed her first $invention.")`

c) `println("In " + year + ", " + name + " completed her first " + invention + ".")`

d) `println("In + year, + name + completed her first + invention")`

11. Complete the String quote with the appropriate character escape sequences to match this output:

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

– Martin Fowler

```
fun main() {  
    var quote = "_____ Any fool can write code that a computer can understand.  
    Good programmers write code that humans can understand._____ –  
    Martin Fowler"  
    print(quote)  
}
```

- `\"`
- `\n`
- `\`
- `\newline`
- `\'`

Click or drag and drop to fill in the blank

12. What are increment and decrement operators in Kotlin?

- a) Increment and decrement operators are used to perform operations only on immutable variables.
- b) Increment and decrement operators are used to control the order of operations.

- c) Increment and decrement operators provide a shorthand syntax for adding or subtracting **10** from a variable.
- d) Increment and decrement operators provide a shorthand syntax for adding or subtracting **1** from a variable.

13. What is the benefit of using String templates over String concatenation?

- a) String templates resemble a more concise syntax.
- b) String concatenation does not exist in Java.
- c) Developers only use String templates in their code.
- d) String concatenation is a deprecated feature.

2.Project

Mad Libs

For this project, we'll use Kotlin to write a Mad Libs word game! Mad Libs have short stories with blank spaces that a player can fill in, giving a funny or unique result!

In order to create a Mad Libs game, you will need a story to be completed and a specific type of words to fill in the blanks.

For this story, you and your friend will be working for the summer to save money for a road trip! One of your favorite bands is playing at a venue far away and you'll be traveling with a friend to go see them.

In order to complete the program you will have to:

- Prompt the user for specific words.
- Use your knowledge of built-in methods to work with different data types and conversions.

- Print the complete story with the variables in the right places.
Note: We'll be using the [bash terminal](#) to collect [user input](#). For instructions on running the program in the bash terminal, see [this](#) exercise or instruction 10 of this project.

User Prompts

1.

In order to complete the story, you will have to prompt the user for certain words. We'll be gathering 7 values from the user in order to complete our story.

Take a look at lines 6 and 7 in **Madlibs.kt**. Line 6 prompts the user to enter their name and line 7 stores their answer in a variable called name.

2.

Use a print statement to prompt the user to enter a friend's name. Below the prompt, store their answer in a variable called friendsName.

3.

An adjective will be used to describe the nature of your trip!

Use a print statement to prompt the user to enter an adjective. Store the answer in a variable called adjective.

4.

What city will you and your friend be driving to?!

Use a print statement to prompt the user to enter a city. Store the answer in a variable called city.

5.

We need to know what band you and your friend will be driving to see!

Use a print statement to prompt the user to enter a band name. Store the answer in a variable called bandName.

6.

For this story, you and your friend will be working for the summer to save money for your trip! You have already stored your friend's name in a variable called `friendsName`.

Using a string template, write print statement to prompt the user to enter what kind of job they and `friendsName` will do in order to save money. Store their answer in a variable called `work`.

Working with Int/Number-based data types

7.

Now, we need to figure out how many miles you and your friend will be traveling! Below the commented line declare a constant variable called `distance` of data type `Int`.

8.

We will now prompt the user for an integer between 400 and 600 to represent the distance to be traveled, and store the value in `distance`. However, since `distance` is expecting a value of data type `Int` we must convert the value from `readLine()`, which is a `String`, into an `Int`.

There are a number of ways to do this, look into the [Kotlin Documentation](#) to find some built-in methods from `Integer` to convert a `String` into an `Int`.

Note: See the **Hint** to see an example of storing user input as an `Int`.

9.

Now that we know the cost of gas and the distance to be traveled, we need to calculate how much money you and your friend need for gas!

The math to calculate the cost for gas is: $(\text{cost of gas} / \text{mileage}) * \text{distance}$. Use the formula above and the variables created to find the total cost of gas and store it in a variable called `totalForGas`.

10.

If we were to print the total cost for gas, it's likely to have an unnecessary amount of decimal places.

In order to round down the value to 2 decimal points, we can use the following equation:

`Math.round(totalForGas * 100) / 100.00`

Reassign the value of `totalForGas` to the result of that equation.

Note: See the **Hint** to see the complete syntax.

11.

You and your friend managed to save up more money than expected to cover all costs for your trip!

Create a new variable called `totalMoneySaved` and set it equal to `totalForGas` plus 200.

Printing Your Story

12.

We're almost done with our program! Now let's work on our story.

At the bottom of this program is a long `println()` statement containing our story. Note the words surrounded by parentheses `()`. The word placed in each pair of parentheses represents one of the variables we declared earlier in this program.

Go through this text and replace each pair of parentheses with the correct variable using a String template.

13.

In order to run our program in this exercise we will follow these 3 steps:

1. Click the Run button when the code is complete
2. Use the following command in the terminal:

```
kotlinc Madlibs.kt -include-runtime -d Madlibs.jar
```

3. Follow up with the command:

```
java -jar Madlibs.jar
```

Optional

14.

Great job completing this project. If you feel like challenging yourself, create your own "Mad Libs" story with new variables and prompts!

```

fun main () {

    val costOfGas = 2.28
    val mileage = 25
    // Declare the variable distance below

    println("What's your name?")
    var name = readLine()
    // Write your code below

    print("""
        A long time ago, (name) and their friend (friendName) planned a(n) (adjective)
        road trip to see their favorite band: (bandName).

        The venue was (distance) miles away which meant they needed to save
        $(totalForGas) dollars to get there.\n

        Both of them worked as (work) for the whole summer and managed to save up
        a total of $(totalMoneySaved) dollars!

        To celebrate, they ended up driving to (city) for the whole weekend; seeing
        their band and buying (bandName) t-shirts to remember their trip.

        """).trimIndent()
}

```


3.Project

Math Magic

Up until this point, you were a Kotlin developer in training. For this project, you'll take on another role and become a mathemagician 🧙♂️

We'll build a program that determines a user's age using our knowledge of fundamental Kotlin concepts and arithmetic operations. Take a peek at the solution code at the end of the project if you'd like to check your answers.

Now, grab your top hat and card deck, and let's program a magic trick!

Variable Declarations

1.

Within the `main()` function, declare an immutable variable, `userName`, and assign it a `String` value of your name.

2.

Next, declare a mutable variable, `magicNum`, to hold a value of type, `Int`. Don't initialize the variable with a value just yet; we will do that in the next step.

Abracadabra 🧙♂️

3.

Now that we've declared all the variables we'll need in this program, we're ready to start coding our magic trick.

The first step is to multiply the first digit of your age by 5 and assign this operation to `magicNum`.

4.

Next, add 3 to the value of magicNum.

5.

On the following line, multiply the value of magicNum by 2 and reassign the result back to magicNum.

6.

Add the second digit of your age to magicNum and reassign this new value to magicNum.

7.

Finally, subtract 6 from magicNum and reassign this new value back to magicNum.

On the following line, wrap magicNum in a println() statement that outputs: [name]'s age is [magicNum].

8.

Awesome work, Mathemagician 🧙 You've utilized your knowledge of arithmetic operators and previous fundamental Kotlin concepts to program a magic trick.

- Run your program several times with different age digits and observe the different values on the right.
- Add a comment to the top of your code to explain to other developers what your program is doing.

```
fun main() {  
  
}
```

OUTPUT

Galina's age is 25

3. USING TEXT VARIABLES

3.1 Strings & Characters

In this lesson, we'll continue working with [variables](#) and take a deeper dive into the [data types](#) Strings and Characters.

Recall that a String in Kotlin is a sequence of characters wrapped in double-quotes. Its characters can include various combinations of digits, letters, symbols, and whitespace.

```
var variableName: String = "some text"
```

Strings are made up of characters and in Kotlin, a *character* is a data type representing a single letter, digit, or symbol wrapped in single quotes. It is denoted by the Char type.

```
var variableName: Char = 'A'
```

Some things to keep in mind about characters:

- The character, 'A', is not the same as 'a' as these letters represent different [Unicode](#)s and won't be recognized as equivalent by the compiler.
- The character, '1', is not the same as the integer, 1, as these values signify two different types of data: Char and Int.

Reference Kotlin's documentation to learn more about [Strings](#) and [Characters](#).

3.1 Strings & Characters Instructions

Take a look at the code editor on the right and observe some of the shorthand variable declarations for String and Char values.

Click **Next** when you're ready to move on.

```
// Strings
```

```
val landmark = "Statue of Liberty 🗽"
```

```
val city = "New York"
```

```
val country = "USA"
```

```
// Characters
```

```
var letter = 'A'
```

```
var digit = '1'
```

```
var symbol = '*'
```

```
// Fun fact
```

```
/* A single emoji is made up of Unicode characters.
```

```
Thus, it will be interpreted as a String
```

```
and must always be wrapped in double quotes: */
```

```
var emoji = "🇺🇸"
```

3.2 String Concatenation

[Strings](#) in Kotlin can stand alone or be combined with other Strings in a process known as *String concatenation*. With a plus operator, +, we are able to concatenate or append one String to another resulting in a new word, phrase, or sentence.

Take a look at the following example:

```
var firstName = "Mike"
var lastName = "Wazowski"

println(firstName + " " + lastName) // Prints: Mike Wazowski
```

In the code above, we've combined the [variables](#), firstName and lastName to produce a full name with the + operator. Notice how we've added an empty String, " ", between the two variables to create natural whitespace between the two words.

In addition to outputting the result of a concatenated String, we can also store this process in a variable for later use. For example:

```
val userGreeting = firstName + " " + lastName + ", how are you doing today?"

println(userGreeting)
```

The compiler replaces each variable with its value and outputs the final String:

Mike Wazowski, how are you doing today?

3.2 String Concatenation Instructions

1.

In **WizardOfOz.kt**, use the given variables, dog, state, and movie to output the following quote using String concatenation:

Toto, I've a feeling we're not in Kansas anymore. - The Wizard of Oz, 1939

```
fun main() {  
    val dog = "Toto"  
    val state = "Kansas"  
    val movie = "The Wizard of Oz"
```

```
// Write your code below ?
```

```
}
```

OUTPUT

Toto, I've a feeling we're not in Kansas anymore. - The Wizard of Oz, 1939

3.3 String Templates

An alternative to String concatenation is a syntax known as String templates. Many developers prefer String templates over String concatenation for its conciseness and efficiency. Take a look at the difference:

Concatenation: "Hello, " + variableName + "!"

Template: "Hello, \$variableName!"

String templates contain a String with any variable names preceded by a \$ symbol. Notice how in contrast to String concatenation, we don't need to specify whitespace, thus saving keystrokes and limiting the amount of code we write.

Assume we're building the following program for a travel company that provides a user with their itinerary:

```
val passengerName = "Alex"  
val trainNumber = 2039  
val timeOfDeparture = "16:00"  
val timeOfArrival = "20:00"
```

```
println("Mrs. $passengerName will be on the $trainNumber train departing at  
$timeOfDeparture to DevCity and arriving at $timeOfArrival.")
```

Each variable gets evaluated within the text producing the final itinerary information:

Mrs. Alex will be on the 2039 train departing at 16:00 to DevCity and arriving at 20:00.

String templates are widely used and are often the preferred syntax by Kotlin developers. Keep in mind that String concatenation is also a valid strategy, so it's important to be familiar with both when collaborating with others.

3.3 String Templates Instructions

1.

In **Plant.kt**, set up a String template using the given variables to output the following text:

An [plant] in a [potSize] inch pot must be watered every [dayNum] days.

Make sure to replace the brackets around each variable with the correct String template notation.

```
fun main() {
```



```
val plant = "orchid"  
val potSize = 6 // in inches  
val dayNum = 7
```

```
// Write your code below 🌱
```

```
}
```

OUTPUT

An orchid in a 6 inch pot must be watered every 7 days.

3.4 User Input

User input is an important part of computer applications. To unlock our phones we enter a code. When shopping online we give our payment information. Our input helps the operation of many applications we use every day.

The `readLine()` function in Kotlin is used to tell the program to pause until some type of input is given by the user. Once the Enter key is pressed, anything typed into the console is returned by the `readLine()` function as a `String`. We can then save the input to a variable. Let's take a look at `readLine()` in action:

```
println("Please enter your favorite color.")  
val myColor = readLine()
```

The two statements above can be broken into 4 parts:

1. A `println()` statement to prompt the user with a question to answer
2. The name of our immutable variable, `myColor`

3. The assignment operator, =
4. The `readLine()` function

The program will prompt the user with a question, accept their input with the `readLine()` function and return the `String` input once the Enter key is pressed. The returned `String` will initialize the declared variable `myColor`.

We can now use a `String` template to output the user input contained in the variable:

```
println("Your favorite color is $myColor")
```

Before we continue, let's note a change to our current environment. In order to work with user input, the output terminal on the right side of our screen has been changed to a terminal running the [Bash shell](#). We use this new terminal to type input into our program as well as use it to compile and run our program.

In order to run our program in this exercise we will follow these 3 steps:

1. Click the Run button when the code is complete
2. Use the following command in the terminal:

```
kotlinc myCode.kt -include-runtime -d myCode.jar
```

1. Followed by this final command:

```
java -jar myCode.jar
```

The first step ensures the Kotlin file is saved. The second step compiles the Kotlin file, **myCode.kt**, into bytecode and saves it in a **.jar** file. The third step runs the compiled application. This process is described in more detail in this [Kotlin Tutorial](#).

3.4 User Input Instructions

- 1.

The first step in gathering input is to create a variable that will store the input from the user.

After the `println()` statement in the editor, declare the variable `myName` and assign it the value `readLine()`.

Click Run to save your work. Remember there will be no output yet.

2.

Now it's time to compile the code.

In the terminal, use `kotlinc` to compile **UserInput.kt** to **UserInput.jar**.

Be sure to use the complete command with the options `-include-runtime` and `-d` as discussed above.

3.

Great work! One more step. Using the `java` command with the `-jar` flag, run the compiled code using the **UserInput.jar** file.

Once it runs, you will be prompted to type your name. Type your response and hit Enter. You should then see the output from line 6.

```
fun main() {  
    println("Please type your name and hit Enter.")  
    // Write your code below  
  
    println("Your name is $myName!")  
}
```

3.5 Built-in Properties and Functions

Kotlin contains built-in properties and [functions](#) that can be used with its various [data types](#). We'll be looking at the most common built-in property and function for [variables](#) of the String type. You can access the full list in [Kotlin's documentation](#).

In short, a *property* provides information on a particular value. Kotlin supports a single String property, `length` which returns the number of characters in a String. Let's see an example:

```
val tallestMountain = "Mount Everest"
print(tallestMountain.length) // Prints: 13
```

The number of characters in "Mount Everest" equates to 13. Notice how the whitespace is also included in the count.

In addition, a String data type also contains built-in functions that can be used to perform certain actions on the String it's appended on. For example, the `capitalize()` function returns a capitalized String:

```
var name = "codecademy"
println(name.capitalize()) // Prints: Codecademy
```

Note that although the returned value is Codecademy, the original lowercase String, "codecademy", goes unchanged:

```
println(name) // Prints: codecademy
```

To persist any changes made by the function, we can reassign the result of `name.capitalize()` to `name` and update the variable:

```
name = name.capitalize()
println(name) // Prints: Codecademy
```

Or we can store the result in an entirely new variable:

```
var capitalizedName = name.capitalize()
println(capitalizedName) // Prints: Codecademy
```

Note: Characters have their own built-in properties and functions in Kotlin. Reference [this page in Kotlin's documentation](#) for a list of all of them.

3.5 Built-in Properties and Functions Instructions

1.

In **MaryPoppins.kt**, below the initialization of the given variable, reassign the value of `word` to the result of `word.capitalize()`. Make sure to do this on one line.

2.

On the following line, declare a variable, `wordSize` and initialize it with the result of appending the `length` property on `word`.

Next, use a String template or String concatenation within a print statement to output the following text:

[word] has [wordSize] letters.

Replace the brackets with the correct notation.

```
fun main() {
    var word = "supercalifragilisticexpialidocious"
```

```
// Write your code below ↕
```

```
}
```

OUTPUT

Supercalifragilisticexpialidocious has 34 letters.

3.6 Character Escape Sequences

In Kotlin, we have the ability to format our text with the help of character escape [sequences](#). When escaped, specific characters such as t and n represent an action or command to be performed by the compiler.

Take a look at some of the following escape sequences and their operations:

- \n Inserts a new line
- \t Inserts a tab
- \r Inserts a carriage return
- \' Inserts a single quote
- \" Inserts a double quote
- \\ Inserts a backslash
- \\$ Inserts the dollar symbol

When used in text, each backslash, \, and character will be recognized and evaluated by the compiler. More than one escape sequence can be used in a single String.

One of the most commonly used escape sequences is the \" as it allows us to cite quotations in a String. Take a look:

```
print("\"May the Force be with you.\" - Star Wars, 1977")
```

In the output terminal, we'd see:

```
"May the Force be with you." - Star Wars, 1977
```

Try the character escape sequence challenge below to hone your understanding of some of these behaviors.

3.6 Character Escape Sequences Instructions

Help Codey format the code in **Challenge.kt** using the appropriate character escape sequences on the given String. The final text should look like this:

Dear Learner 🙌,

I hope you're enjoying the Learn Kotlin course.
You've made tremendous progress. Keep going!
And remember, "Knowledge is power." (Frances Beacon).

With love,
Codey

One Rule: You can only use a single print statement. Good luck!

P.S. When you're ready, move on to the next exercise to review what you've learned and see the answer to the above prompt.

```
fun main() {  
  
    // Add the correct escape sequences to the text below to achieve the output  
    shown in the instructions  
  
    print("Dear Learner 🙌, I hope you're enjoying the Learn Kotlin course. You've  
made tremendous progress. Keep going! And remember, Knowledge is power.  
(Frances Beacon). With love, Codey")  
}
```

OUTPUT

Dear Learner 🙌, I hope you're enjoying the Learn Kotlin course. You've made tremendous progress. Keep going! And remember, Knowledge is power. (Frances Beacon). With love, Codey

3.7 Review

Great job! You've completed the second lesson on [variables](#) and learned the fundamentals of text in Kotlin. Let's review what was covered.

- [Strings](#) are [sequences](#) of characters wrapped in double-quotes.
- A Character represents a single digit, symbol, or letter wrapped in single quotes.
- String concatenation is the process of joining Strings using a + operator.
- String templates are used to substitute variable values within a String using a \$ symbol.
- The `readLine()` function is used to accept a user's input via the bash shell.
- The built-in `length` property determines the number of characters in a String.
- The built-in `capitalize()` function is used to capitalize the String onto which it's appended.
- Character escape sequences allow for the compiler to perform certain formatting operations on a String.

Run the code in **Review.kt** and view Codey's message. Feel free to also use this space to hone your understanding of text variables and practice writing Kotlin code.

3.7 Review Instructions

Click **Up Next** when you've ready to move on.

```
fun main() {  
    println("Dear Learner 🙌, \n\nI hope you're enjoying the Learn Kotlin  
course.\nYou've made tremendous progress. Keep going! And remember,  
\"Knowledge is power.\" (Frances Beacon). \n\nWith love, \nCodey")  
}
```

4. USING NUMBER VARIABLES

4.1 Number Types

We've briefly worked with integers in the previous lesson, and in this section, we'll take a deeper dive into the various types of numbers that exist in Kotlin.

Kotlin numbers fall into two categories: integers and decimals as part of the [Number](#) superclass. Take a look at the image on the right. Each book occupies a certain amount of space on the shelf depending on its size. The same goes for the various numerical data types in Kotlin. The larger a variable is in size (Bits), the more space it occupies in our computer's memory.

The books on the left side represent 4 different types of data for an integer value:

Name	Bit (Size)	Min Value	Max Value
Long	64	-9,223,372,036,854...	9,223,372,036,854...
Int	32	-2,147,483,648	2,147,483,647
Short	16	-32,768	32,767
Byte	8	-128	127

A Long variable is meant to store significantly large values, whereas a Byte variable cannot store a value larger than 127. Hence, their size and occupation in memory vary.

The books on the right represent 2 various types of data for decimal or floating-point values:

Name	Bit (Size)	Decimal Digit Precision
------	------------	-------------------------

Double	64	15-16
--------	----	-------

Float	32	6-7
-------	----	-----

A Double variable contains a longer, more precise decimal precision than a Float.
For example:

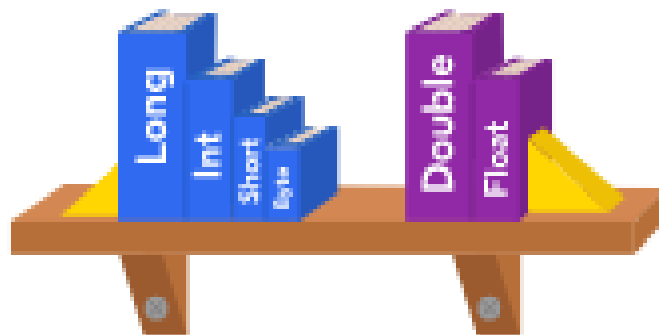
```
var doubleNum: Double = 6.0 / 13.0
```

```
var floatNum: Float = 6.0f / 13.0f
```

```
println(doubleNum) // Prints: 0.46153846153846156
```

```
println(floatNum) // Prints: 0.46153846
```

When declaring a numerical variable in Kotlin, the compiler will determine which of the above data types to assign to it. The default type for whole numbers is Int and the default type for decimals is Double, thus we'll be working with these two types of data more closely throughout this lesson.



4.2 Arithmetic Operators

The arithmetic [operators](#) that exist in Kotlin resemble the operators we know from basic math. They can be used in various applications from calculating the average test grade for a student class to determining the total restaurant bill.

Kotlin supports the following arithmetic operators:

- + addition
- - subtraction
- * multiplication
- / division
- % modulus (divides and gives the remainder)

The above arithmetic operators aside from modulus, %, should all be familiar. Modulus is a programming symbol that is used to calculate the division and return the remainder. For example:

```
println(7 % 2)
```

7 gets divided by 2 and results in 3 remainder 1. The remainder then gets returned and printed to the terminal:

```
1
```

Within the `println()` statement above, we built an arithmetic expression.

An *arithmetic expression* is the grouping of arithmetic operators and numeric values. Take a look at some of the ways in which we can set up and use arithmetic expressions throughout a Kotlin program.

With variables:

```
var operand1 = 5.5  
var operand2 = 4.3  
println(operand1 + operand2)
```

```
// Prints: 9.8
```

In a print statement:

```
println(8 - 4)
// Prints: 4
```

Concatenated within a String:

```
println("The quotient of -9 and 3 is " + -9 / 3)
// Prints: The quotient of -9 and 3 is -3
```

In the { } of a String template:

```
println("The product of 6 and 2, is ${6 * 2}")
// Prints: The product of 6 and 2 is 12
```

4.2 Arithmetic Operators Instructions

1.

In **Arithmetic.kt**, we're going to use our knowledge of arithmetic operators to calculate the total restaurant bill. Let's get started!

Observe the variables containing the price of each ordered meal. Declare two more variables:

- total containing an arithmetic expression that adds each meal price:

```
meal1 + meal2 + meal3
```

- tip holding the value, 0.15.

2.

Declare the final variable, `finalAmount`, which we'll use to calculate the total amount including the tip. Initialize `finalAmount` with the following arithmetic expression:

```
total * tip + total
```

Finally, use a `println()` statement to output `finalAmount` and see the total bill.

```
fun main() {  
  
    // Appetizer  
    val caesarSalad = 8.50  
  
    // Entree  
    val beefStroganoff = 18  
  
    // Dessert  
    val cheesecake = 6  
  
    // Add your code below ↓  
  
}
```

4.3 Order of Operations

As seen in the previous exercise, Kotlin allows us to combine or chain arithmetic expressions in order to achieve more complex calculations. When doing so, we must keep in mind the order in which each expression gets evaluated.

Most programming languages follow a certain order of operations (similar to [PEMDAS](#)) when evaluating such expressions. In Kotlin, this order looks like:

1. Parentheses
2. Multiplication
3. Division
4. Modulus
5. Addition
6. Subtraction

You may have noticed that we skipped over the E (Exponents) in PEMDAS. This is because Kotlin does not have an exponent operator and instead relies on the `.pow()` function to calculate exponentiation. We'll cover this more in-depth in the following exercise.

Let's see a few examples of compound arithmetic expressions:

```
println(5 + 8 * 2 / 4 - 3) // Prints: 6
println(3 + (5 + 5) / 2)  // Prints: 8
println(3 * 2 + 1 * 7)    // Prints: 13
```

Another rule to keep in mind is when an expression contains sister operations such as multiplication and division or addition and subtraction, side by side. The compiler will evaluate this expression in a left to right order. For example:

```
println(3 + 16 / 2 * 4) // Prints: 35
```

Since we have division, `/`, and multiplication, `*`, side by side, the left to right order goes into effect and the expression evaluates as such:

Keep in mind both sets of rules when creating complex arithmetic expressions.

4.3 Order of Operations Instructions

1.

In **Operations.kt**, we've set up multiple arithmetic expressions that are missing arithmetic operators.

Add an arithmetic operator in place of ____ to complete each expression.

- answer1 should be 14
- answer2 should be 3
- answer3 should be 11

Note: You will see an error in the terminal on the right if there are still any ____ present in your code. The error should go away after you've replaced each ____ with the correct operator.

```
fun main() {  
    var answer1 = 7 - 5 + 4 ____ 3  
    var answer2 = (9 - 3) ____ 2  
    var answer3 = (6 ____ 8) + (4 - 7)  
  
    println(answer1)  
    println(answer2)  
    println(answer3)  
}
```

4.4 Augmented Assignment Operators

As we build upon our programs, we'll often need to update the values of variables following certain calculations. Assume a program that increases the current speed of a vehicle to match a speed limit of 55:

```
var currentSpeed = 45
currentSpeed = currentSpeed + 10
print(currentSpeed) // Prints: 55
```

In order to increase and update the value of `currentSpeed`, we've added 10 to `currentSpeed` and reassigned it back to `currentSpeed`. Kotlin provides an even shorter syntax for operations like this with augmented assignment operators.

Augmented assignment operators execute a calculation and reassign its result to a variable all in one step. Each consists of an arithmetic operator immediately followed by the `=` operator. Take a look at the difference in syntaxes:

Long Syntax:

```
a = a + b
```

Short Syntax with an Augmented Assignment Operator:

```
a += b
```

Here's a breakdown of each:

Operation	Long Syntax	Short Syntax
Add	<code>a = a + b</code>	<code>a += b</code>
Subtract	<code>a = a - b</code>	<code>a -= b</code>
Multiply	<code>a = a * b</code>	<code>a *= b</code>
Divide	<code>a = a / b</code>	<code>a /= b</code>
Mod	<code>a = a % b</code>	<code>a %= b</code>

Keeping the short syntax in mind, we can now refactor and optimize our previous program to its final, concise state:

```
currentSpeed += 10  
println(currentSpeed) // Prints: 55
```



4.4 Augmented Assignment Operators Instructions

1.

In **NumberFacts.kt**, we've declared several variables and initialized them with numerical values that are erroneous. Use the following instructions to update each variable with its correct value using Kotlin's augmented assignment operators:

- speedOfLight: add 282 and reassign the new value
- perfectSquare: multiply by 2 and reassign the new value
- sheldonsFavoriteNum: divide by 3 and reassign the new value
- emergency: subtract 9 and reassign the new value
- firstCountingNum: use modulo to divide by 5 and reassign the remainder

Run your program to see the correct numerical values and facts.

```
fun main() {  
    var speedOfLight = 186_000  
    // Write your code here  
  
    var perfectSquare = 32  
    // Write your code here
```

```
var sheldonsFavoriteNum = 219
```

```
// Write your code here
```

```
var emergency = 920
```

```
// Write your code here
```

```
var firstCountingNum = 10
```

```
// Write your code here
```

```
println("The speed of light is ${speedOfLight}.")
```

```
println("A perfect square is ${perfectSquare}.")
```

```
println("Sheldon's favorite number is ${sheldonsFavoriteNum}.")
```

```
println("The emergency number in the United States is ${emergency}.")
```

```
println("The first counting number in programming is ${firstCountingNum}.")
```

```
}
```

4.5 Increment and Decrement Operators

In addition to augmented assignment [operators](#), Kotlin provides us with two arithmetic shortcuts that can be used to perform quick addition and subtraction calculations. These are known as increment and decrement operators.

An *increment operator* represents the addition of 1 to a variable using two consecutive plus operators, ++:

```
variableName++
```

The *decrement operator* represents the subtraction of 1 from a variable using two consecutive minus operators, --:

```
variableName--
```

This shorthand syntax is most useful with [loops](#) and [conditionals](#) - concepts we'll explore later in this course. For now, let's just get familiar with the syntax.

Assume we're writing a program that keeps track of how many apples we have in our basket whilst apple picking. 🍏

```
var apples = 5
```

We picked up a Gala apple, so let's add 1 to our basket using an increment operator:

```
apples++  
println(apples) // Prints: 6
```

We ate a Honey Crisp apple from our basket, so let's subtract 1 from the total:

```
apples--  
println(apples) // Prints: 5
```

Note: ++ and += 1 are the same operations and can be used interchangeably throughout a program.

4.5 Increment and Decrement Operators Instructions

1.

In **Challenge.kt**, we've set up variables, a, b, c, and d to hold numerical values or expressions. Use your knowledge of shorthand arithmetic operators to determine the final value of d.

Store your answer in a single line comment when you're ready as so:

```
// d: some value
```

Refrain from using a print statement to get the answer. Instead, check the hint for a breakdown of the solution when you're ready.

```
fun main() {
```

```
    var a = 5
```

```
    a += 6
```

```
    var b = a * 9
```

```
    var c = b / 3
```

```
    c++
```

```
    var d = c + 8
```

```
    d %= 4
```

```
    d--
```

```
}
```

4.6 The Math Library

For more complex mathematical calculations, we can reference the Math library which is inherited from Java. It consists of various mathematical functionalities that cover a range of topics from logarithms, trigonometry, statistics, and more. Let's take a look at some of the most commonly used [functions](#) it provides.

Math.pow() returns the first value raised to the power of the second value. It accepts and returns Double values.

```
Math.pow(5.0, 3.0) // 125.0
```

Math.min() returns the minimum value from two given numbers.

```
Math.min(5, 3) // 3
```

Alternatively, **Math.max()** determines the maximum value.

Math.random() returns a randomly generated number between 0 and 1.

```
Math.random() // 0.3335735290527727
```

Math.round() accepts a single Double value and returns the result of rounding that value to the nearest Integer.

```
Math.round(15.7) // 16
```

4.6 The Math Library Instructions

1.

Explore the various other Math functions in the code editor on the right. As you run the code and witness the output, write a description in comment format below each function describing what you think it does.

Check your answers against the ones in the Hint when you're ready.

```
fun main() {
```

```
    var ceil = Math.ceil(3.5)
```

```
    println(ceil)
```

```
    // Add your notes below
```

```
var sqrt = Math.sqrt(25.0)
println(sqrt)
// Add your notes below
```

```
var abs = Math.abs(-20)
println(abs)
// Add your notes below
```

```
var floor = Math.floor(6.9)
println(floor)
// Add your notes below
```

```
}
```

OUTPUT

4.0

5.0

20

6.0

4.7 Review

Excellent work! You've completed the final lesson on [variables](#) and learned the fundamentals of numbers in Kotlin. Let's review what was covered.

- The Number superclass contains Int, Long, Short, Byte, Float and Double [data types](#).

- The arithmetic [operators](#) supported by Kotlin include +, -, *, / and %.
- Augmented assignment operators include an arithmetic and assignment operator used to calculate and reassign a value in one step.
- The increment operator includes two consecutive plus operators, ++, used to add 1 to a value and reassign the result.
- The decrement operator includes two consecutive minus operators, --, used to subtract one from a value and reassign the result.
- The order of operations includes parentheses, multiplication, division, modulus, addition and subtraction.
- The Math library is inherited from Java and contains various mathematical [functions](#) for use in a Kotlin program.

Feel free to utilize the empty **Review.kt** file and output terminal on the right to hone your understanding of numbers and practice writing Kotlin code.

4.7 Review Instructions

Click **Up Next** when you've completed the review.

5. CONDITIONAL EXPRESSIONS

5.1 Introduction to Conditionals

When we go on adventures, sometimes the road is straightforward and other times, the road splits and we must choose a path to take. [Conditionals](#) are like forks in the road: they determine the direction our program takes.

A *conditional* is an expression that evaluates true or false values in order to control the flow of code in a program.

When we discussed Kotlin in terms of the order of execution, we stated that Kotlin code is compiled and executed in top-down order. This remains true; however, a conditional statement can be used to determine what instructions are executed and what instructions are ignored.

In this lesson, we will learn how to control the flow of code execution in our Kotlin programs.

5.1 Introduction to Conditionals Instructions

Take a look at the image to the right.

In order to reach Dev City, Codey must pass through Conditional Creek, where there are many river paths that each lead to a different location.

In order to prevent travelers from getting lost, they are given colored, patterned sails that will help them navigate to their destination. At each split in the river, there are signs that say which path a traveler should take depending on the color and pattern of their boat sail.

To get to Dev City, Codey will follow the directions for red striped sails.



5.2 if Expression

An if expression is a conditional that executes a body of code only when a specified Boolean value returns true.

Assume we had a Boolean variable called raining:

```
var raining = true
```

We can use an if expression to evaluate the value of raining to determine whether or not we execute some code:

```
if (raining) {  
    println("Bring an umbrella.")  
}
```

```
}
```

- The expression starts with the keyword, if.
- A pair of parentheses contains a condition (raining) that evaluates to either true or false.
- Following the condition is a pair of brackets { and } that holds a block of code.

The code contained within the code block of the if expression will only execute if the condition is true; otherwise, that code block will be ignored.

When we run the program above, we get the following output:

Bring an umbrella.

The value of raining inside of the if expression is true; therefore, the instruction `println("Bring an umbrella.")` gets executed.

5.2 if Expression Instructions

1.

Your neighbor keeps forgetting what day the garbage gets picked up. You decide to create a small program to remind them when to take their garbage out.

Create an if expression that checks if the value of `isTuesday` is true.

Inside the if expression, create a `println()` statement that outputs "Take out the trash."

```
fun main(){  
    var isTuesday = true  
    // Write your code below
```

```
}
```

OUTPUT

Take out the trash.

5.3 if/else Expression

Often accompanying the if expression is another conditional called the else expression. The else expression executes a block of instructions when the condition in the if expression has a false value.

For example:

```
var isHerbivore = false  
var diet: String
```

```
if (isHerbivore) {  
    diet = "vegetables"  
} else {  
    diet = "meat"  
}  
println("This animal can eat $diet.")
```

```
// Prints: This animal can eat meat.
```

The else expression is not followed by a condition since it will only run code if the if expression's condition is false.

This expression is always placed as the last conditional and cannot exist without an accompanying if expression.

5.3 if/else Expression Instructions

1.

Create an if expression that checks if the value of saleHappening is true.

Inside the if expression, set the value of price to 12.

2.

Underneath the if expression, create an else expression that sets the value of price to 15.

3.

Outside of the conditional expression, use println() and a string template to output the following statement:

The total is [price] dollars.

```
fun main() {  
    var saleHappening = false  
    var price: Int  
    // Write your code below
```

```
}
```

OUTPUT

The total is 15 dollars.

5.4 Comparison Operators

To create more complex conditions for our conditionals to evaluate, we can use comparison operators. *Comparison operators* evaluate the relationship between two values in order to return an expression as true or false.

There are several comparison operators we can use in Kotlin: `<`, `>`, `>=`, and `<=`.

The *less than* operator (`<`) evaluates to true if the value on the left side of the `<` is smaller than the value on the right:

```
var username = "beetle"

if (username.length < 10) {
    println("Please choose a longer username.")
}
// Prints: Please choose a longer username.
```

For the *greater than* operator (`>`), if the value on the left side of the `>` symbol is larger than the value on the right, the expression returns true:

```
var username = "beetlejuice_beetlejuice_beetlejuice"

if (username.length > 25) {
    println("Please choose a shorter username.")
}
// Prints: Please choose a shorter username.
```

The *less than or equal to* operator (`<=`) returns true only if the value on the left is less than or equal to the value on the right:

```
var liquidTemp = 27 // Fahrenheit

if (liquidTemp <= 32) {
    println("The liquid will freeze.")
}
```

```
} else {  
    println("The liquid will remain liquid.")  
}  
// Prints: The liquid will freeze.
```

The *greater than or equal to* operator `>=` will return true if the value on the left is greater than or equal to the value on the right:

```
var liquidTemp = 50 // Fahrenheit
```

```
if (liquidTemp >= 212) {  
    println("The liquid will vaporize.")  
} else {  
    println("The liquid will remain liquid.")  
}  
// Prints: The liquid will remain liquid.
```

5.4 Comparison Operators Instructions

1.

[Dungeons & Dragons](#) is a roleplaying game that uses dice to determine the success of a player's action.

Create an if expression that checks if the value of d20, which represents a 20-sided pair of dice, is greater than or equal to 17.

Inside the if expression, create a `println()` statement that outputs, "The incantation works and the dragon is defeated."

2.

Create an else expression that contains a `println()` statement that outputs "The incantation fails and the dragon wins."

```
fun main() {  
    var d20 = 13  
    // Write your code below  
  
}
```

OUTPUT

The incantation fails and the dragon wins.

5.5 Equality and Inequality Operators

Another way to compare two values is with the equality [operators](#).

We can use the *equality* operator (`==`) to check if the value on the right side of the operator is equal to the value on the left. If the values are equivalent the expression will return true; otherwise, it will return false:

```
var userPassword = "secret884"  
var attemptedPassword = "secret884"  
  
if (userPassword == attemptedPassword) {  
    println("The password you entered is correct.")  
}
```

This code snippet will output "The password you entered is correct." because `userPassword` and `attemptedPassword` have the same value.

On the other hand, the *inequality* operator (`!=`) is used to determine if the values on each side of the `!=` operator are different. This expression returns true when the two values being compared are not equal to each other.

We can use `!=` to check if a gathering of 9 people can evenly split 16 slices of pizza amongst themselves:

```
var slicesOfPizza = 16
var numPeople = 9

if (slicesOfPizza % numPeople != 0) {
    println("The pizza cannot be split evenly.")
}
// Prints: The pizza cannot be split evenly.
```

Using modulo `%`, we can find if dividing 16 by 9 has a remainder of 0. Since the remainder value does not equal 0, "The pizza cannot be split evenly." is outputted to the terminal.

5.5 Equality and Inequality Operators Instructions

1.

Create an if expression that checks if the value of `num1` is even.

In the body of the if expression, use `println()` and a String template to output the following:

[`num1`] is even.

2.

Create a second if expression that checks if the value of num2 is odd using !=.

In the body of the if expression, use println() and a String template to output the following:

[num2] is odd.

```
fun main() {  
    var num1 = 8  
    var num2 = 9  
    // Write your code below  
  
}
```

OUTPUT

8 is even.

9 is odd.

5.6 else-if Expressions

An else-if expression can be used to add a plethora of more [conditionals](#) for our program to evaluate.

Let's say we created a Kotlin program that determines what we will wear depending on the temperature outside. If we just use an if/else expression, our options are limited:

```
var temp = 60
```

```
if (temp > 65) {  
    println("Wear a t-shirt")  
} else {  
    println("Wear a winter coat")  
}  
// Prints: Wear a winter coat
```

In real life, wearing a winter coat in 60-degree weather can cause a person to overheat. We need to wear something in between a heavy coat and a t-shirt, like a light jacket. We can place an else-if expression in our code to represent this “in-between” situation:

```
var temp = 60
```

```
if (temp > 65) {  
    println("Wear a t-shirt")  
} else if (temp > 45) {  
    println("Wear a light coat")  
} else {  
    println("Wear a winter coat")  
}  
// Prints: Wear a light coat
```

Let's examine this else-if statement:

- The else-if expression must come after an if expression but before the else expression.
- The else-if expression contains its own condition placed within parentheses: temp > 45.

The instruction println("Wear a light coat") is executed only if the condition in the if expression is false **and** the condition within the else-if expression is true. If

both conditions are false, the instructions in the next conditional, the else expression, will execute.

An if-else expression can contain as many else-if expressions as needed. When using multiple else-if statements, the order in which they appear matters. The moment the program evaluates one of the conditionals as true, the program will execute that body of code and will not evaluate any of the conditionals that appear after.

5.6 else-if Expressions Instructions

1.

The local coffee shop started a rewards program based on the amount of coffee a customer buys. Use your knowledge of conditionals to create a program that determines what kind of member a customer is based on the amount of rewards points they have.

If rewardsPoints is greater than or equal to 50:

- Set the value of memberType to "Platinum".

Otherwise, if rewardsPoints is greater than or equal to 25:

- Set the value of memberType to "Gold".

Otherwise, if rewardsPoints is greater than or equal to 10:

- Set the value of memberType to "Silver".

Else:

- Set the value of memberType to "Bronze".

2.

Outside of the conditional expressions, use `println()` and a String template to output the following:

```
fun main() {  
    var rewardsPoints = 17  
    var memberType: String  
    // Write your code below  
  
}
```

OUTPUT

You are a Silver type member.

5.7 Logical Operators

Logical [operators](#) evaluate the relationship between two or more Boolean expressions and return a true or false value. There are several operators that we can use: `&&`, `||`, and `!`.

The logical AND operator (`&&`) returns true if and only if the expressions on both sides of the operator return true.

```
var isSunny = true  
var temp = 85  
  
if (isSunny && temp > 80) {  
    println("Wear sunglasses.")  
}  
// Prints: Wear sunglasses.
```

Since isSunny is true and temp has a value above 80, the expression inside the condition becomes true so Wear sunglasses. is outputted to the terminal.

The logical OR operator (||) will only return true if at least one of the Boolean expressions being compared has a true value.

```
var isSnowing = false
```

```
var temp = 38
```

```
if (isSnowing || temp < 40 ) {  
    println("Wear a scarf.")  
}  
// Prints: Wear a scarf.
```

Even though the first value isSnowing is false, the full expression will return true because the other Boolean expression, temp < 40, is true.

The logical NOT operator, denoted by !, will evaluate the truth value of a Boolean expression and then return its inverse. For example:

```
var isSnowing = true  
println(!isSnowing) // Prints: false
```

We can even add ! before a multi-part expression. By adding the ! operator to our previous code example, the Boolean expression now returns a false value:

```
var isSnowing = false  
var temp = 38  
  
if (!(isSnowing || temp < 40)) {  
    println("Wear a scarf.")  
}  
// This program will not have any output.
```

Note: The truth tables displayed on the right side of the screen display the various outcomes of Boolean expressions being evaluated using logical operators.

5.7 Logical Operators Instructions

1.

Take a look at **Logic.kt**.

Replace each `__` with a logical operator so that each expression evaluates to true and 4 statements are printed to the terminal.

Use the truth tables on the right side of the screen to help make a decision about which logical operator to use.

Note: Feel free to adjust the size of the code editor to get a better view of the code.

```
fun main() {  
    var a = true  
    var b = false  
    var c = true  
    var d = false  
  
    if (a __ b) {  
        println("Hoorah!")  
    }  
  
    if (!b __ c) {  
        println("Woohoo!")  
    }  
  
    if (a __ d) {
```

```
println("Yay!")  
}  
  
if (!(!d __ b)) {  
    println("Wow!")  
}  
  
}
```

OUTPUT

Hoorah!
Woohoo!
Yay!
Wow!

AND		
A	B	A && B
True	True	True
True	False	False
False	True	False
False	False	False

OR		
A	B	A B
True	True	True
True	False	True
False	True	True
False	False	False

NOT	
A	!A
True	False
False	True

5.8 Order of Evaluation

We can combine expressions containing logical [operators](#) to create even more complex Boolean expressions. When combining logical operators, the order of evaluation is as follows:

1. Boolean expressions contained within parentheses
2. NOT - !
3. AND - &&
4. OR - ||

Take a look at the following snippet of code:

```
var isSnowing = true
var isSunny = false
```

```
var temp = 38
```

```
if (!isSunny && (temp > 40 || isSnowing)) {  
    println("Wear a coat.")  
} else {  
    println("Don't wear a coat.")  
}
```

Let's discuss the order in which this expression is evaluated. Here is the Boolean expression we are evaluating:

```
!isSunny && (temp > 40 || isSnowing)
```

(temp > 40 || isSnowing) is evaluated first because the expression is contained within parentheses. This expression results in true, making our expression look like this:

```
!isSunny && true
```

!isSunny is evaluated next because of the NOT operator. This expression is true because it negates the value of isSunny:

```
true && true
```

Finally, we evaluate the expression using the logical AND operator (&&). Since both sides of the operator have a true value, the expression inside our if statement is true and we get the final output:

```
Wear a coat.
```

5.8 Order of Evaluation Instructions

- 1.

Use your understanding of logical operators and their order of evaluation to determine the value of the expression inside of the if expression.

If you believe the program will output "Hello", set the value of expressionValue to true.

If you believe the program will output "Goodbye", set the value of expressionValue to false.

Finally, use println() to output the value of expressionValue.

Note: Running the code will output the answer. Make sure to declare your variable and finalize your answer before you hit **Run**.

```
fun main() {  
    var expressionValue: Boolean  
  
    if (!true && (false && true || !false)) {  
        println("Hello")  
    } else {  
        println("Goodbye")  
    }  
}
```

// Write your code below

```
}
```

OUTPUT

Goodbye

false

5.9 Nested Conditionals

Similar to the concept of a “dream inside of a dream” from the movie [Inception](#), a conditional can exist inside of another conditional. This occurrence is called a *nested conditional*.

By nesting a conditional inside of another conditional, we can achieve more complex and precise logical solutions.

For example, remember Conditional Creek? We can use a nested conditional to replicate this situation:

```
var sailColor = "red"
var sailPattern = "striped"

if (sailColor == "red") {
  println("Follow the lower river.")
  if (sailPattern == "striped") {
    println("Then take the right path.")
  } else {
    println("Then take the left path.")
  }
} else {
  println("Take the higher river.")
}
```

The first conditional checks the truth value of `sailColor` while the nested conditional checks the truth value of `sailPattern`. This program will output the following:

```
Follow the lower river.
Then take the right path.
```

5.9 Nested Conditionals Instructions

1.

In order to be considered a planet, a celestial body must accomplish [three things](#) according to the International Astronomical Union:

1. Must orbit a star (like the sun).
2. Must have enough mass in order to achieve hydrostatic equilibrium (a nearly round shape).
3. Must have enough mass to clear debris/small objects from its orbit.

The difference between a planet and a dwarf planet comes from rule 3.

In **Planet.kt** create an if expression that checks if `orbitsStar` AND `hydrostaticEquilibrium` are true.

Leave the body of the if expression blank for now.

2.

Inside the if expression, create a nested conditional that checks if the value of `clearedOrbit` is true.

Inside the body of this nested if expression, create a `println()` statement that outputs "Celestial body is a planet."

3.

Underneath the nested if expression, create an else expression that outputs "Celestial body is a dwarf planet." using a `println()` statement.

```
fun main() {  
    var orbitsStar = true // Rule 1
```

```
var hydrostaticEquilibrium = true // Rule 2
```

```
var clearedOrbit = false // Rule 3
```

```
// Write your code below
```

```
}
```

OUTPUT

Celestial body is a dwarf planet.

5.10 when Expressions

Another expression that controls the flow of execution in a program is the when expression.

A when expression is made up of multiple branches that contain values, or arguments, to check for. If the value of the variable being evaluated matches one of the branches' arguments the instructions contained in that branch will be executed.

For example:

```
var lightColor = "red"
```

```
when (lightColor) {  
  "green" -> println("Go.")  
  "yellow" -> println("Slow down.")  
  "red" -> println("Stop.")  
  else -> println("Not a valid traffic light color.")  
}
```

```
}  
// Prints: Stop.
```

- The keyword `when` is followed by a variable placed within parentheses.
- The set up for each branch starts with a value to check for, an arrow (`->`), and then an instruction to execute.
- The `else` at the end of a `when` expression is an optional addition that can be used to catch any unexpected or additional values the variable being evaluated could contain.

In the example above, the value of `lightColor` was `"red"`, so the instruction `println("Stop.")` was executed.

Programmers often use `when` expressions in lieu of `if/else`, `else-if` [conditionals](#) because of their significantly shorter syntax. For example, we can create a `when` expression that evaluates multiple expressions:

```
var num = 19  
when {  
    num < 0 -> println("$num is negative.")  
    num == 0 -> println("$num is zero.")  
    num > 0 -> println("$num is positive.")  
    else -> println("Not a valid number.")  
}  
// Prints: 19 is positive.
```

Note how in the example above, we did not place a value inside parentheses after the `when` keyword and instead placed full expressions inside the branches' arguments.

To explore other ways we can implement a `when` expression in our programs, check out the [Kotlin Language Guide](#).

5.10 `when` Expressions Instructions

1.

We're going to create a program that determines what kind of crop to grow based on the season.

Create a when expression that evaluates the value of the variable season.

When the value of season is:

- "Winter", print "Grow kale."
- "Spring", print "Grow lettuce."
- "Summer", print "Grow corn."
- "Fall", print "Grow pumpkins."

Add an else expression that prints "Not a valid season."

2.

Optional

Take some time to experiment with your code by changing the value of season and running your program.

```
fun main() {  
    var season = "Fall"  
    // Write your code below  
  
}
```


OUTPUT

Grow pumpkins.

5.11 Range

In Kotlin, range provides a powerful tool that represents a consecutive succession of values that can often be used in conjunction with [conditionals](#).

In Kotlin, ranges are created using the `..` operator:

`StartingValue..EndingValue`

The range will start at the value that appears before `..` and will continue until it reaches the value that appears after `...`

We can check if values are contained within a specified range with the help of the `in` keyword:

```
var num = 5
```

```
if (num in 1..10) {  
    println("This value is between 1 and 10.")  
}
```

The `in` keyword is used to check if a value exists inside of a collection or range. The program above checks if `num` exists within the range of 1 through 10.

We can also check if a character, or `Char`, value appears inside of a range:

```
var letter = 'c'
```

```
when (letter) {  
    in 'a'..'m' -> println("Letter is in 1st half of alphabet.")  
    in 'n'..'z' -> println("Letter is in 2nd half of alphabet.")  
    else -> println("Not a valid value")  
}
```

```
}
```

Note: Letter ranges are case-sensitive.

5.11 Range Instructions

1.

The [Scoville Scale](#) is used to measure the heat, or pungency, of a pepper.

Use if, else-if, and else expression to determine the pungency of a chili pepper based on the pepper's Scoville Heat Units(SHU).

If the value of sHU is between...

0-699:

- Set pungency to "very mild".

700-2999:

- Set pungency to "mild".

3000-24999:

- Set pungency to "moderate".

25000-69999:

- Set pungency to "high".

Else:

- Set pungency to "very high"

2.

Use a String template and println() to output the following statement:

```
fun main() {  
    var sHU = 17000  
    var pungency: String  
  
    // Write your code below  
  
}
```

OUTPUT

A pepper with 17000 Scoville Heat Units has a moderate pungency.

5.12 Review

Great job getting to the end of this lesson. Let's review the content we learned:

- Conditional expressions control the flow of execution in a program.
- An if expression executes a block of code when the value of its condition is true.
- An else expression executes code when none of the previous expressions have a true value.
- Comparison operators like `>`, `<`, `>=`, and `<=` are used to compare two values to determine a true or false value.
- The `==` operator checks if two values are equivalent while the `!=` operator checks if two values are not equivalent.

- The logical AND operator (&&) returns true when two Boolean expressions being compared are both true.
- The logical OR operator (||) returns true when at least one Boolean expression being compared is true.
- The logical NOT operator (!) negates the value of a Boolean expression.
- else-if expressions are used to add more conditions to a traditional if/else statement.
- A nested conditional is a conditional that appears inside of another conditional.
- A when expression evaluates the value of a variable or expression in order to dictate the flow of a program.
- The range operator .. creates a consecutive succession of values.

5.12 Review Instructions

Use the space in **Review.kt** to practice the content learned in this lesson.

When you're ready to move on, click **Up Next**.

```
fun main() {
```

```
// Write your code below
```

```
}
```

CONCEPT REVIEW

If Expressions

An if expression is a conditional that runs a block of code when its condition has a true value.

```
var morning = true
```

```
if (morning) {  
    println("Rise and shine!")  
}  
// Prints: Rise and shine!
```

Else Expressions

An else expression is a conditional that runs a block of code only when the conditions contained in the previous expressions have false values.

```
var rained = false
```

```
if (rained) {  
    println("No need to water the plants today.")  
} else {  
    println("Plants need to be watered!")  
}
```

```
}  
// Prints: Plants need to be watered!
```

Else-If Expressions

An else-if expression allows for more conditions to be evaluated within an if/else expression.

You can use multiple else-if expressions as long as they appear after the if expression and before the else expression.

```
var age = 65
```

```
if (age < 18 ) {  
    println("You are considered a minor.")  
} else if (age < 60) {  
    println("You are considered an adult.")  
} else {  
    println("You are considered a senior.")  
}
```

```
// Prints: You are considered a senior.
```

Comparison Operators

Comparison operators are symbols that are used to compare two values in order to return a result of true or false. Comparison operators include >, <, >=, <=.

```
var myAge = 19
```

```
var sisterAge = 11
```

```
var cousinAge = 11
```

```
myAge > sisterAge // true
```

```
myAge < cousinAge // false
```

```
myAge >= cousinAge // true
```

```
myAge <= sisterAge // false
```

Logical Operators

Logical operators are symbols used to evaluate the relationship between two or more Boolean expressions in order to return a true or false value.

Logical operators include !, &&, and ||.

```
var humid = true
```

```
var raining = true
```

```
var jacket = false
```

```
println(!humid)
```

```
// Prints: false
```

```
println(jacket && raining)
```

```
// Prints: true
```

```
println(humid || raining)
```

```
// Prints: true
```

The AND Operator: &&

The logical AND operator (&&) is used to compare the relationship between two Boolean expressions and will only return a true value if both expressions are true.

```
var humid = true
```

```
var raining = true
```

```
var shorts = false
```

```
var sunny = false
```

```
// true AND true
```

```
println(humid && raining) // true
```

```
// true AND false
```

```
println(humid && shorts) // false
```

```
// false AND true
```

```
println(sunny && raining) // false
```

```
// false AND false
```

```
println(shorts && sunny) // false
```


The OR Operator : ||

The logical OR operator (||) is used to compare the relationship between two Boolean expressions and will return true when at least one of the expressions are true.

```
var late = true
```

```
var skipBreakfast = true
```

```
var underslept = false
```

```
var checkEmails = false
```

```
// true OR true
```

```
println(skipBreakfast || late) // true
```

```
// true OR false
```

```
println(late || checkEmails) // true
```

```
// false OR true
```

```
println(underslept || late) // true
```

```
// false OR false
```

```
println(checkEmails || underslept) // false
```

The NOT Operator: !

The logical NOT operator (!) evaluates the value of a Boolean expression and then returns its negated value.

```
var hungry = true
```

```
var full = false
```

```
println(!hungry) // false
```

```
println(!full) // true
```

Order of Evaluation

The order of evaluation when using multiple logical operators in a single Boolean expression is:

1. Expressions placed in parentheses.
2. NOT(!) operator.
3. AND(&&) operator.
4. OR(||) operator.

```
!true && (false || true) // false
```

```
/*
```

(false || true) is evaluated first returning true. Then,

!true && true is evaluated returning the final result, false.

```
*/
```

```
!false && true || false // true
```

```
/*
```

!false is evaluated first returning true. Then true && true are evaluated, returning true. Then, true || false is evaluated which ends up returning true.

*/

Nested Conditionals

A nested conditional is a conditional that exists within another conditional.

```
var studied = true
```

```
var wellRested = true
```

```
if (wellRested) {  
    println("Best of luck today!")  
    if (studied) {  
        println("You should be prepared for your exam!")  
    } else {  
        println("Take a few hours to study before your exam!")  
    }  
}
```

```
// Prints: Best of luck today!
```

```
// Prints: You should be prepared for your exam!
```

When Expressions

A when expression controls the flow of code by evaluating the value of a variable in order to determine what code gets executed.

```
var grade = "A"
```

```
when(grade) {  
    "A" -> println("Excellent job!")  
    "B" -> println("Very well done!")  
    "C" -> println("You passed!")  
    else -> println("Close! Make sure to perpare more next time!")  
}  
// Prints: Excellent job!
```

The Range Operator

The range operator (..) is used to create a succession of number or character values.

```
var height = 46 // inches
```

```
if (height in 1..53) {  
    println("Sorry, you must be at least 54 inches to ride the rollercoaster.")  
}  
// Prints: Sorry, you must be at least 54 inches to ride the rollercoaster.
```

Equality Operators

Equality operators are symbols that are used to compare the equivalence of two values in order to return true or false. Equality operators include == and !=.

```
var myAge = 22
```

```
var sisterAge = 21
```

```
myAge == sisterAge // false
```

```
myAge != sisterAge // true
```

5. QUIZ

1. Fill in the blanks to create a program that outputs **"Winter is here."** when the value of **isWinter** is **true** or **"Winter is coming."** when **isWinter** is **false**.

```
var isWinter = false
```

```
if (____) {  
  println("Winter is here.")  
} ____ {  
  println(____)  
}
```

- "Winter is coming."
- else

- Otherwise
- isWinter
- isSummer

Click or drag and drop to fill in the blank

2. Which of the following Boolean expressions will result in **true**?

- a) true && false || false
- b) !true && false
- c) true && (!false || true)**
- d) !(false || true && true)

3. Which part of the following expression will be evaluated first?

```
var isWindy = true
var isRaining = false
var temp = 45
```

```
if (!isRaining || isWindy && temp <= 45) {
    println("Wear a sweater.")
}
```

- a) **!isRaining** because it uses the NOT operator.
- b) **!isRaining || isWindy** because it uses the OR operator.
- c) **!isRaining && temp <= 45** because it uses the AND operator.
- d) **isWindy && temp <= 45** because it uses the AND operator.

4. How would the following program be written as a **when** expression?

```
var liquidTemp = 50 // Fahrenheit
```

```
if (liquidTemp <= 32) {  
    println("The liquid will freeze.")  
} else if (liquidTemp >= 212) {  
    println("The liquid will vaporize.")  
} else {  
    println("The liquid will remain liquid.")  
}
```

a) var liquidTemp = 50 // Fahrenheit

```
when liquidTemp <= 32 -> println("The liquid will freeze.")  
when liquidTemp >= 212 -> println("The liquid will vaporize.")  
when else -> println("The liquid will remain liquid.")
```

b) var liquidTemp = 50 // Fahrenheit

```
when {  
    liquidTemp <= 32 -> println("The liquid will freeze.")  
    liquidTemp >= 212 -> println("The liquid will vaporize.")  
    else -> println("The liquid will remain liquid.")  
}
```

c) var liquidTemp = 50 // Fahrenheit

```
when (liquidTemp) {  
    32 -> println("The liquid will freeze.")  
    212 -> println("The liquid will vaporize.")  
    else -> println("The liquid will remain liquid.")  
}
```

d) This code cannot be rewritten using a **when** expression.

5. When does the logical AND (**&&**) operator return **true**?

- a) The expression will always return **true** regardless of the values of each expression.
- b) When the two expressions being compared both have a true value.
- c) When at least one of the expressions being compared has a **true** value.
- d) When the two expressions being compared both have a **false** value.

6. When does the code inside of an **else** expression get executed?

- a) When the condition inside of an **else-if** expression is **true**.
- b) When the conditions inside of the **if** and **else-if** expressions are all **false**.
- c) Code inside of an **else** expression is automatically executed whenever a program is run.
- d) When the condition inside of an **if** expression is **true**.

7. What is an **if** expression?

- a) A variable that has a **true** or **false** value.
- b) An expression that executes a block of code when a specified condition is **true**.
- c) An expression that executes a block of code when a specified expression is **false**.
- d) A function that prints values to the terminal.

8. Fill in the blanks to create a nested conditional expression.

The outer **if** expression should check if **num** is divisible by 2 while the nested **if** expression should check if **num** is divisible by 3.

var num = 18

```
if ( _____ == 0) {  
  if (num % 3 _____ 0) {  
    println(_____)
```



```
}  
}
```

// Prints: 18 is both even and divisible by 3.

- "\$num is not even or divisible by 3."
- "\$num is both even and divisible by 3."
- num * 2
- ==
- !=
- num % 2

Click or drag and drop to fill in the blank

9. True or false: An **else-if** expression is placed after an **else** expression.

- a) True.
- b) False.

10. Fill in the blanks below to make the program output **"You will pass the test."** if the value of **studiedNotes** is **true**.

```
var studiedNotes = true
```

```
____ ( ____ ) {  
  println( ____ )  
}
```

- else
- studiedNotes
- "You will pass the test."
- If

- "true"

Click or drag and drop to fill in the blank

11. What will be the output of the following program?

```
var dogWeight = 49 // lbs
```

```
when (dogWeight) {  
  in 1..22 -> println("Small-sized dog.")  
  in 22..54 -> println("Medium-sized dog.")  
  in 55..150 -> println("Large-sized dog.")  
  else -> println("Not a valid weight input.")  
}
```

- a) **Medium-sized dog.**
- b) Not a valid weight input.
- c) Large-sized dog.
- d) Small-sized dog.

12. What will the following program output?

```
var a = false  
var b = false
```

```
if (!a || b) {  
  println("Good morning.")  
} else {  
  println("Good night.")  
}
```

- a) Good night.

b) true

c) **Good morning.**

d) (!a || b)

13. Which operator represents the logical AND operator?

a) !

b) ||

c) and

d) &&

14. What will the following program output?

```
var timeOfDay = "afternoon"
```

```
if (timeOfDay == "morning") {  
    println("Eat breakfast.")  
} else if (timeOfDay == "afternoon") {  
    println("Eat lunch.")  
} else {  
    println("Eat dinner.")  
}
```

a) Eat lunch.

b) Eat breakfast.

c) Eat snacks.

d) Eat dinner.

15. What will the following code output?

```
var isSunny = false
isSunny = true

if (isSunny) {
    println("Wear sunscreen!")
} else {
    println("Leave the sunscreen home!")
}
```

- a) Wear sunscreen!
- b) Leave the sunscreen home!
- c) false
- d) true

4.Project

Self-Grading Quiz

Working at a school that has recently switched to e-learning tactics, you have been assigned the task of creating a self-grading quiz program for the math class.

Using your knowledge of Kotlin and conditionals, you will create a program that collects and checks a student's answers and then gives them a final letter grade.

As you go through the project, check off the boxes next to each instruction as you complete them.

Note: We'll be using the [bash terminal](#) to collect [user input](#). For instructions on running the program in the bash terminal, see [this](#) exercise or the hint in instruction 11.

Creating Quiz Questions

1.

Take a look at the variables in **Quiz.kt**; each variable represents a quiz answer.

We'll create 4 quiz questions using print statements and 4 variables that store a user's input.

For question 1, create a print statement that asks "What is 75 / 5?".

Then, create a variable called `studentAnswer1` that stores the user's input as a number-based value.

Note: See the **Hint** to learn how to store user input as an `Int`.

2.

For question 2 of the quiz, ask the user for the value of y in the equation: $6 * 2y = 36$

Create a variable called `studentAnswer2` that stores the user's answer as an `Int`.

3.

Create a third print statement that asks the user to name the type of triangle that has two equal sides.

Store the user's answer as `String` in a variable called `studentAnswer3`.

4.

Create a print statement that asks the user for the value of $8 * 6 - (3 - 2)$.

Store the user's input as an `Int` variable named `studentAnswer4`.

Grading the Quiz

5.

Create a variable called points and set its value to 0.

This variable will be used to keep track of the student's score.

6.

First, let's grade question 1.

If the value of studentAnswer1 is equivalent to quizAnswer1, increase the value of points by 25.

7.

Let's move on to question 2. This question gives a student multiple opportunities to earn points.

If the value of studentAnswer2 equals quizAnswer2, increase points by 25.

However, if the value of studentAnswer2 is the value of quizAnswer2 plus or minus 1, increase points by 20 and output the following statement:

The answer [studentAnswer2] is within 1 point of [quizAnswer2].

Otherwise, increase points by 1.

8.

Let's grade question 3.

If the value of studentAnswer3 equals quizAnswer3, increase points by 25.

However, increase points by 10 if the value of studentAnswer3 is equal to "equilateral".

Otherwise, increase points by 1.

9.

Let's grade question 4.

If studentAnswer4 equals quizAnswer4, then increase points by 25.

However, if studentAnswer4 has a value between 44 and 54,
increase points by 20.

Otherwise, increase points by 1.

Returning a Final Grade

10.

Use the value of points to determine the student's letter grade.

If points is between...

0 to 59:

- Print that the student received an F.

60 to 69:

- Print that the student received a D.

70 to 79:

- Print that the student received a C.

80 to 89:

- Print that the student received a B.

90 to 100:

- Print that the student received an A.

101 to 110:

- Print that the student received an A+.

11.

Run the code in the bash terminal to test your program.

See the **Hint** for instructions on running the code in the bash terminal.

Optional

12.

Great job completing the project!

If you want to keep challenging yourself, check out these additional tasks:

- Create your own set of quiz questions.
- If you used a when expression to find the student's final letter grade, modify your code to use an if/else-if/else statement, or vice versa!

```
fun main() {  
    val quizAnswer1 = 15  
    var quizAnswer2 = 3  
    val quizAnswer3 = "isosceles"  
    val quizAnswer4 = 47  
  
    // Write your code below  
  
}
```


6. LISTS

6.1 Introduction to Lists

In this section, we're going to explore the various types of collections that exist in Kotlin. A *collection* is a group of elements that exist in a single structure. The first collection type we'll learn about is one that's most commonly used in Kotlin and many other programming languages: a list.

A Kotlin *list* is very much similar to a real-world list: it represents an ordered collection of elements that can contain duplicates. These elements can be of any primitive data type and represent a myriad of real-world scenarios. These can include student grades, restaurant ratings, a to-do list of tasks, and more.

Take a look at the image on the right. After a long day of sailing through the creek, Codey is going to set up camp and rest for the night. Being the environmentally friendly Kotlin developer that they are, they've opted to make a virtual to-do list of things they need to get done before nightfall:

```
var toDo = listOf("gather wood", "start a fire", "set up tent", "eat dinner",  
"stargaze")
```

Printing the list would result in the following output:

```
[gather wood, start a fire, set up tent, eat dinner, stargaze]
```

Notice how the syntax of a list in its definition is different from its output. If Kotlin isn't your first programming language, you might recognize the output syntax with square brackets to be identical to that of an array.

As opposed to lists, [arrays](#) in Kotlin have a fixed size, thus not being the most convenient or easily applicable [data structure](#). For that reason, we'll be focusing

our time on exploring lists, but we encourage you to read up on arrays in Kotlin's [documentation](#).



6.2 Creating an Immutable List

In Kotlin, the terms “mutable” and “immutable” apply not only to variables but also to collections. Much like an immutable variable representing a read-only value, an *immutable list* contains items that must never be altered. Its syntax is as follows:

```
var/val listName = listOf(value1, value2, value3)
```

- For a list to be used and referenced throughout a program, it must be stored in a variable. Make sure to use keywords `val` or `var` in your declaration as well as a descriptive list name.

- The creation of the Kotlin list begins on the right side of the assignment operator with the term, `listOf`. This indicates an immutable list.
- The term is then followed by a pair of parentheses that holds a collection of elements separated by commas.
- Like variable declarations, the type of list can be inferred by the compiler or declared by the user. For example, a list of `String` values would be referenced as a `List<String>`.

Note: Once initialized as a `List<String>`, the list can *only* hold `String` values.

Keeping the above syntax in mind, let's create an immutable list of `String` values that represent the countries we've visited:

```
var countriesVisited = listOf("Japan", "Colombia", "Kenya", "Jordan")
```

Since `countriesVisited` was declared a mutable variable with the `var` keyword, we can reassign the value of `countriesVisited` to store a different list if we'd like. However, since the list was declared immutable with `listOf`, the contents of the list cannot change. They must always remain `"Japan"`, `"Colombia"`, `"Kenya"`, and `"Jordan"`.

```
println(countriesVisited)
// Prints: [Japan, Colombia, Kenya, Jordan]
```

Lastly, when printed to the screen, the list is contained within square brackets as opposed to parentheses.

We've learned a handful of rules and new concepts in this exercise. Don't get discouraged if it doesn't all stick right away; we'll be getting ample practice throughout this lesson. Keep going, and let's create your first Kotlin list! 🛠️

6.2 Creating an Immutable List Instructions

1.

In **Sports.kt**, create an immutable list containing the following water sports:

- "Wind Surfing"
- "Sailing"
- "Swimming"
- "Jet Skiing"
- "Water Skiing"

Store this list in a variable, waterSports.

2.

Output the value of waterSports on the following line using a print statement.

```
fun main() {  
    // Write your code below 🏊♀  
  
}
```

OUTPUT

[Wind Surfing, Sailing, Swimming, Jet Skiing, Water Skiing]

6.3 Accessing Elements in a List

A collection is of no use to us if we can't access or work with its contents. In order to access the elements within a list, we must reference the element's "index". An *index* represents a numerical position within a list.

```
val oakIslandArtifacts = listOf("Lead cross", "Rhodolite garnet brooch", "Gold brooch")
```

Here's a breakdown of the indices of the above list elements:

Element	Index
"Lead cross"	0
"Rhodolite garnet brooch"	1
"Gold brooch"	2

Notice how the count begins with 0 and not 1; this is because Kotlin is a zero-based language and the count in every Kotlin collection begins with 0.

Keeping these indexed-positions in mind, we can construct the following syntax to access an element:

```
listName[index]
```

The name of the list is followed by square brackets which contains the index of the element we need to access. Applying this pseudo code to our previous list, we can access each element as such:

```
oakIslandArtifacts[0] // Lead cross
oakIslandArtifacts[1] // Rhodolite garnet brooch
oakIslandArtifacts[2] // Gold brooch
```

Each statement can be wrapped in a `println()` statement to see its output or stored in a variable for later use.

Note: Attempting to access an element within an index that does not exist will result in an error. For instance, if there are 2 elements within a list, and the specified index is 5, the compiler will throw the following Out of Bound error:

java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 2

6.3 Accessing Elements in a List Instructions

1.

Every year, the United States hosts hundreds of marathons, some of which have made it into the list of top 10 marathons worldwide. In **Marathon.kt**, we'll create a list containing some of the most popular ones.

Declare a variable, `unitedStatesMarathons` and in it store an immutable list containing the following marathon names:

- "Boston Marathon"
- "Bank of America Chicago Marathon"
- "TCS NYC Marathon"
- "Marine Corps Marathon"

Fun Fact: About 0.5% percent of the U.S. Population has run a full marathon. 🏃♀️

2.

Access "Bank of America Chicago Marathon" using square brackets and its index in the list. Wrap this code in a print statement to see the output.

```
fun main() {  
    // Write your code below 🏃♀️
```

```
}
```

OUTPUT

Bank of America Chicago Marathon

6.4 Creating a Mutable List

Elements can be accessed from any list, but only with mutable lists can we change the contents of our list by adding, replacing, or removing elements. Previously we've only created immutable lists; in this exercise, we'll learn how to create a mutable list and alter its contents.

The syntax for declaring a mutable list is as follows:

```
var/val listName = mutableListOf(value1, value2, value3)
```

A mutable list is defined by the term, `mutableListOf` as opposed to `listOf` as we've seen previously. Although the syntax and structure of mutable and immutable lists are the same, the compiler knows which operations can or can't be performed on the list given the term it's defined with.

Assume we have a list that stores a student's daily activities:

```
var dailyActivities = mutableListOf("going to school", "attending tennis practice",  
"reading")
```

Now that it's summertime and there's no school, we would like to update the first element of our list to reflect the new season. We can take our previous knowledge of accessing elements a step further and assign a new value to an element's indexed position with the following syntax:

```
dailyActivities[0] = "swimming at the beach"
```

The value following the assignment operator will get replaced with the current value at the 0 index. Printing the value of `dailyActivities` will result in the following output and updated list:

[swimming at the beach, attending tennis practice, reading]

6.4 Creating a Mutable List Instructions

1.

In **Broadway.kt**, create a mutable list, `openBroadwayShows` which will store a list of shows that are available for viewing on Broadway. This list should include the following titles:

- "The Lion King"
- "Chicago"
- "The Cher Show"
- "Les Miserables"

2.

The Cher Show has been closed as of August 2019. Let's remove it from the list and replace it with a show that's still open: "Wicked".

Next, use a print statement to output the value of `openBroadwayShows` and see the updated list.

```
fun main() {  
    // Write your code below 🎭
```

```
}
```

OUTPUT

[The Lion King, Chicago, Wicked, Les Miserables]

6.5 The size Property

In Kotlin, the size property determines the number of elements within a collection. It is not only applicable to lists but also to other collections that we'll be working with in the coming lessons.

Similarly to the length property for [Strings](#), size counts the number of elements in a collection and returns an Integer value.

Take a look at the following list of major European rivers:

```
val majorRivers = listOf("Volga", "Danube", "Loire", "Rhine", "Elbe")
```

We can append size to majorRivers and retrieve the number of elements in the list:

```
println(majorRivers.size) // Prints: 5
```

Additionally, this code can be used to construct an informative String about our list:

```
println("There are ${majorRivers.size} major rivers in Europe.")  
// Prints: There are 5 major rivers in Europe.
```

Furthermore, size can be used along with an element's index to retrieve the last element of a collection:

```
println(majorRivers[majorRivers.size - 1]) // Prints: Elbe
```

- Within the brackets, majorRivers.size equates to 5. Subtracting 1 from this value results in 4 which becomes the final numerical index within the brackets.
- majorRivers[4] returns the 4th and last element in the list, "Elbe".

This strategy is most commonly used when iterating over a collection or simply grabbing the last element of a list when we don't know how many list elements there are in total. We'll learn more about how to iterate over a collection later in the course.

6.5 The size Property Instructions

1.

In **Fruit.kt**, create a variable, `fruitTrees`, which stores a mutable list of the following fruit trees:

- "Apple"
- "Plum"
- "Pear"
- "Cherry"

2.

On the next line, construct the following sentence using String template syntax:

I am growing [____] different types of fruit in my garden.

Replace the value within the brackets with `fruitTrees.size`. Use a print statement to output this sentence to the screen.

```
fun main() {
```

```
    // Write your code below 🍏
```

```
}
```

OUTPUT

I am growing 4 different types of fruit in my garden.

6.6 List Functions

Every collection in Kotlin contains a series of built-in [functions](#) that allow us to perform certain operations in order to retrieve information about the collection or manipulate its values.

A function that retrieves information about a list would only require read access whereas a function that changes the contents of a list would require both read and write access. As a result of this, a mutable list supports functions that possess read and write functionalities, whereas an immutable list supports functions that possess read-only operations. Let's take a closer look:

Mutable and immutable list functions:

```
var vowels = listOf('A', 'E', 'I', 'O', 'U')
```

The `contains()` function accepts a single value within its parentheses and returns a Boolean true or false depending on whether or not that value exists within a list. It does not need to change the contents of the list, thus it can be used on an immutable list.

```
println(vowels.contains('A')) // Prints: true  
println(vowels.contains('B')) // Prints: false
```

The `random()` function also only requires read-only access and returns a random element from the list:

```
println(vowels.random()) // Prints: I
```

Mutable ONLY list functions:

```
var primeNumbers = mutableListOf(4, 5, 7, 11, 13)
```

The `add()` function accepts a single value within its parentheses and appends that value to the end of a list:

```
primeNumbers.add(17)  
println(primeNumbers) // Prints: 4, 5, 7, 11, 13, 17
```

Since this function changes the contents of the list, it can only be used on mutable lists. Similarly, the `remove()` function, also only used on mutable lists, accepts a single value within its parentheses and removes it from the list:

```
primeNumbers.remove(4)  
println(primeNumbers) // Prints: 5, 7, 11, 13, 17
```

Note: If the value to be removed does not exist, the compiler will not throw an error and simply return `false`.

These functions are just the tip of the iceberg of what's supported for lists in Kotlin; we encourage you to look through the documentation for [immutable lists](#) and [mutable lists](#) to explore other available functionalities.

6.6 List Functions Instructions

1.

In **Planets.kt**, we've declared a mutable list containing the names of planets within our solar system. We'll use Kotlin's built-in list functions to fix some of its inaccuracies.

First, use the `remove()` function to delete the planet that does not fit in the list. Check the hint for help.

2.

There is one planet that's currently missing. On the following line, use the `add()` function to add it to the list.

Hint: It's the only planet in the entire solar system with a liquid surface.

3.

Implement the `random()` function on the following line to generate a random planet from the list. Make sure to wrap your code in a `println()` statement to see the output.

Run your program multiple times to see the randomly generated values.

```
fun main() {
```

```
    val planets = mutableListOf("Mercury", "Venus", "Mars", "Jupiter", "Saturn",  
    "Uranus", "Neptune", "Pluto")
```

```
    // Write your code below
```

```
}
```

OUTPUT

Neptune

6.7 Review

Excellent work! In this lesson, we've learned the following concepts:

- A list is a type of Kotlin Collection that contains ordered elements.
- An immutable list is declared with `listOf` and indicates a list whose values cannot change throughout a program.
- A mutable list is declared with `mutableListOf` and indicates a list whose values can be altered.
- Elements within a list can be accessed using their index.
- The `size` property returns the number of elements in a collection.
- Immutable lists possess read-only functionalities, meanwhile, mutable lists support both read and write functionalities.
- The `contains()` function checks if an element exists in a given list. It returns a Boolean `true` or `false`.
- The `add()` function appends an element to the end of a mutable list.
- The `remove()` function removes an element from a mutable list.
- The `random()` function generates a random element from a given list.

You've covered a major, fundamental programming concept used in most programs to store and manipulate data. Feel free to utilize the empty **Review.kt** file and output terminal on the right to hone your understanding of lists and practice writing Kotlin code.

CONCEPT REVIEW

Immutable Lists

An immutable list represents a group of elements with read-only operations.

It can be declared with the term `listOf`, followed by a pair of parentheses containing elements that are separated by commas.

```
var programmingLanguages = listOf("C#", "Java", "Kotlin", "Ruby")
```

Mutable Lists

A mutable list represents a collection of ordered elements that possess read and write functionalities.

It can be declared with the term, `mutableListOf` followed by a pair of parentheses containing elements that are separated by commas.

```
var fruits = mutableListOf("Orange", "Apple", "Banana", "Mango")
```

Accessing List Elements

In order to retrieve an element from a list, we can reference its numerical position or index using square bracket notation.

Note: Remember that the first element of a list starts at 0.

```
var cars = listOf("BMW", "Ferrari", "Volvo", "Tesla")
```

```
println(cars[2]) // Prints: Volvo
```

The Size Property

The size property is used to determine the number of elements that exist in a collection.

```
var worldContinents = listOf("Asia", "Africa", "North America", "South America",  
"Antarctica", "Europe", "Australia")
```

```
println(worldContinents.size) // Prints: 7
```

List Operations

The list collection supports various operations in the form of built-in functions that can be performed on its elements.

Some functions perform read and write operations, whereas others perform read-only operations.

The functions that perform read and write operations can only be used on mutable lists while read-only operations can be performed on both mutable and immutable lists.

```
var seas = listOf("Black Sea", "Caribbean Sea", "North Sea")
```

```
println(seas.contains("North Sea")) // Prints: true
```

// The contains() function performs a read operation on any list and determines if an element exists.


```
seas.add("Baltic Sea") // Error: Can't perform write operation on immutable list
```

// The add() function can only be called on a mutable list thus the code above throws an error.

Immutable Sets

An immutable set represents a collection of unique elements in an unordered fashion whose contents cannot be altered throughout a program.

It is declared with the term, setOf, followed by a pair of parentheses holding unique values.

```
var primaryColors = setOf("Red", "Blue", "Yellow")
```

Mutable Sets

A mutable set represents a collection of ordered elements that possess both read and write functionalities.

It is declared with the term, mutableSetOf, followed by a pair of parentheses holding unique values.

```
var womenInTech = mutableSetOf("Ada Lovelace", "Grace Hopper", "Radia Perlman", "Sister Mary Kenneth Keller")
```

Accessing Set Elements

Elements in a set can be accessed using the `elementAt()` or `elementAtOrNull()` functions.

The `elementAt()` function gets appended onto a set name and returns the element at the specified position within the parentheses.

The `elementAtOrNull()` function is a safer variation of the `elementAt()` function and returns null if the position is out of bounds as opposed to throwing an error.

```
var companies = setOf("Facebook", "Apple", "Netflix", "Google")
```

```
println(companies.elementAt(3)) // Prints: Google
```

```
println(companies.elementAt(4)) // Returns and Error
```

```
println(companies.elementAtOrNull(4)) // Prints: null
```

Immutable Maps

An immutable Map represents a collection of entries that cannot be altered throughout a program.

It is declared with the term, `mapOf`, followed by a pair of parentheses. Within the parentheses, each key should be linked to its corresponding value with the `to` keyword, and each entry should be separated by a comma.

```
var averageTemp = mapOf("winter" to 35, "spring" to 60, "summer" to 85, "fall" to 55)
```

Mutable Maps

A mutable map represents a collection of entries that possess read and write functionalities. Entries can be added, removed, or updated in a mutable map.

A mutable map can be declared with the term, `mutableMapOf`, followed by a pair of parentheses holding key-value pairs.

```
var europeanDomains = mutableMapOf("Germany" to "de", "Slovakia" to "sk",  
"Hungary" to "hu", "Norway" to "no")
```

Retrieving Map Keys and Values

Keys and values within a map can be retrieved using the `.keys` and `.values` properties.

The `.keys` property returns a list of key elements, whereas the `.values` property returns a list of value elements.

To retrieve a single value associated with a key, the shorthand, `[key]`, syntax can be used.

```
var oscarWinners = mutableMapOf("Parasite" to "Bong Joon-ho", "Green Book"  
to "Jim Burke", "The Shape Of Water" to "Guillermo del Toro")
```

```
println(oscarWinners.keys)
```

```
// Prints: [Parasite, Green Book, The Shape Of Water]
```

```
println(oscarWinners.values)
```

```
// Prints: [Bong Joon-ho, Jim Burke, Guillermo del Toro]
```

```
println(oscarWinners["Parasite"])
```

```
// Prints: Bong Joon-ho
```

Adding and Removing Map Entries

An entry can be added to a mutable map using the `put()` function. Oppositely, an entry can be removed from a mutable map using the `remove()` function.

The `put()` function accepts a key and a value separated by a comma.

The `remove()` function accepts a key and removes the entry associated with that key.

```
var worldCapitals = mutableMapOf("United States" to "Washington D.C.",  
"Germany" to "Berlin", "Mexico" to "Mexico City", "France" to "Paris")
```

```
worldCapitals.put("Brazil", "Brasilia")
```

```
println(worldCapitals)
```

```
// Prints: {United States=Washington D.C., Germany=Berlin, Mexico=Mexico City,  
France=Paris, Brazil=Brasilia}
```

```
worldCapitals.remove("Germany")
```

```
println(worldCapitals)
```

```
// Prints: {United States=Washington D.C., Mexico=Mexico City, France=Paris,  
Brazil=Brasilia}
```

7. SETS

7.1 Introduction to Sets

In this lesson, we'll learn another fundamental Kotlin collection known as a set. A set is an un-ordered collection of unique elements. It's most commonly applicable in programs that work with data [sets](#) containing non-duplicate values such as usernames or passwords. Upon creating or adding elements to a set, the set automatically filters out any duplicates and always returns a unique collection.

As you progress through this lesson, you'll notice that lists and sets are very common in syntax and share a multitude of built-in [functions](#). Their key difference lies in their purpose and behavior in a program. A developer building a program for a subway line's train stops may choose to work with a list since trains often cross paths, whereas a program that stores bank account information would most likely be utilizing a set.

As we continue exploring other types of collections in Kotlin, keep in mind the similarities and differences between each one you've learned. A good developer doesn't waste time memorizing every syntax in the book, but instead, focuses on understanding what tools are available and how they can be applied to a program.

7.1 Introduction to Sets Instructions

Take a look at the image on the right.

After setting up camp, Codey is exploring the constellations in the night sky. Since each constellation appears only once, a set is the ideal collection type needed to store this data.

Click **Next** when you're ready to start learning more about sets.



7.2 Creating an Immutable Set

As previously discussed, Kotlin collections fall under two categories: mutable and immutable, and sets are no exception. An immutable set is declared using the `setOf` keyword and indicates a set whose values cannot change throughout a program:

```
var/val setName = setOf(val1, val2, val3)
```

Note: Similarly to lists, the set type can be inferred by the compiler or declared in the code. For example, a set comprised of String values would be inferred as a `Set<String>`.

In the code below, we are creating a new set and storing it in the variable, `colorsOfTheRainbow`:

```
var colorsOfTheRainbow = setOf("red", "orange", "yellow", "green", "blue", "red")
```

Logging `colorsOfTheRainbow` to the console, notice that there is one less color in the set. The last element, "red", was a duplicate String that the set object recognized and removed:

```
println(colorsOfTheRainbow)
```

```
// Prints: [red, orange, yellow, green, blue]
```



7.2 Creating an Immutable Set Instructions

1.

In **Technologies.kt**, declare a variable, `obsoleteTech`, and assign it an immutable set with the following values:

- "Rolodex"
- "Phonograph"
- "Videocassette recorder"

- "Video projector"
- "Rolodex"

2.

Output the value of the set using a print statement. Notice how there's one less item in the final set.

```
fun main() {
```

```
// Write your code below 
```

```
}
```

OUTPUT

[Rolodex, Phonograph, Videocassette recorder, Video projector]

7.3 Accessing Elements in a Set

Unlike lists, a set is an un-ordered collection, thus we can't use square brackets and indices to retrieve its elements. However, a set is an intelligent tool that stores its elements in the order in which each element was inserted. We can utilize this order along with a function, `elementAt()`, to access and retrieve elements.

The `elementAt()` function accepts an Integer value and returns the element at that position. This function is useful for collections that do not possess index access.

```
listName.elementAt(integer)
```


Assume we're writing a program that keeps track of animals that are no longer on the endangered list:

```
var nonEndangered = setOf("Louisiana Black Bear", "Northern Brown Kiwi", "Gray Wolf", "Arabian Oryx")
```

To retrieve the animal, "Gray Wolf", we'd use the following syntax:

```
nonEndangered.elementAt(2) // Gray Wolf
```

Since Kotlin is a [null safe](#) language, it encourages developers to avoid `NullPointerException` errors wherever they can by using safer variations of function.

In the case that an element does not exist at a specified position, our code would throw an error:

```
nonEndangered.elementAt(5) // Exception error
```

Exception error:

```
Exception in thread "main" java.lang.IndexOutOfBoundsException: Collection doesn't contain element at index 5.
```

To avoid this, we can use the `elementAtOrNull()` function, a safer variation of `elementAt()` to return null as opposed to an error when an element does not exist in the set:

```
nonEndangered.elementAtOrNull(5) // null
```

7.3 Accessing Elements in a Set Instructions

1.

In **Islands.kt**, declare a variable, `islandsOfHawaii`, and assign it an immutable set containing the following elements:

- "Maui"
- "Lanai"
- "Oahu"
- "Kauai"

2.

On the following line, retrieve the island of Oahu from the set using the `elementAt()` function and its position.

Wrap this code in a print statement to see the output.

3.

Use the `elementAtOrNull()` function to retrieve an element from the 6th position.

Wrap this code in a print statement to see the output. You should not see an error in the terminal.

```
fun main() {  
    // Write your code below
```

```
}
```

OUTPUT

Oahu

null

7.4 Creating a Mutable Set

With immutable sets, we don't have much control over the contents they store, but with mutable sets, we are able to add, remove, or replace as many elements as we'd like. This allows us to work both with sets that are populated with values and empty sets, which are sets that are initialized without any values but can be populated at a later time.

To initialize an empty mutable set, we'd use the following syntax:

```
var/val setName = mutableSetOf<Type>()
```

- The term, `mutableSetOf` is used to declare a mutable set.
- The term is followed by the type of data that we expect the set to hold and a pair of empty parentheses.

Assume we're developing a program for a building manager who would like to keep track of the apartments they own. Unfortunately, we haven't received the apartment information from them just yet, so we create a mutable set and wait until we can populate it with values:

```
var totalApts = mutableSetOf<String>()
```

While we wait, let's practice declaring a set that comes prepopulated with values. The syntax is as follows:

```
var/val setName = mutableSetOf(value1, value2, value3)
```

No need to specify the set type here as it will automatically be inferred by the compiler. Let's see an example:

```
var nycRadioStations = mutableSetOf(106.7, 101.1, 97.1, 103.5)
```

The compiler will infer `mutableSetOf<Double>` from the code above and will only accept values of type `Double` within this set going forward.

7.4 Creating a Mutable Set Instructions

1.

In **Moon.kt**, create a mutable set, `apolloLandingSites`, populated with the following String values:

- "Mare Tranquillitatis"
- "Fra Mauro"
- "Hadley/Apennines"
- "Descartes"
- "Taurus-Littrow"
- "Oceanus Procellarum"

These values represent the 6 Apollo Moon landing sites. ●

2.

Output the values of the set using a print statement. Think about what type of Kotlin program can be written with this data.

```
fun main() {
```

```
    // Write your code below 🧑🚀
```

```
}
```

OUTPUT

[Mare Tranquillitatis, Fra Mauro, Hadley/Apennines, Descartes, Taurus-Littrow, Oceanus Procellarum]

7.5 Adding and Removing Elements

To add or remove any elements from a set, we must utilize Kotlin's built-in set [functions](#). Luckily, many of these functions can also be applied to other collection types, thus you might recognize some of these syntaxes from a previous lesson.

To add a single element to a set, we'd use the `add()` function as we did in the lists lesson; however, if we were interested in adding many elements to a set at once, the `addAll()` function would be more appropriate. Note that in contrast to lists, we cannot add an element that already exists in the set.

Let's take a look at how `addAll()` works. Recall the building example from the previous exercise. The manager got back to us with the following list of apartments:

```
var aptList = listOf("1A", "1B", "1C", "2A", "2B", "2C", "2C", "1A")
```

We're ready to populate our empty set, `totalApts`, with these values. With the help of the `addAll()` function, we're able to populate our set with multiple values at once:

```
totalApts.addAll(aptList)
```

```
println(totalApts) // Prints: [1A, 1B, 1C, 2A, 2B, 2C]
```

Looks like the manager may have accidentally added duplicate apartment numbers. Luckily, the set removed the duplicate elements, and we're left with unique data.

Oppositely, we can also remove all elements from a collection using the `clear()` function. Assume the building is selling their property and all of the apartments need to be removed from the data set:

```
totalApts.clear()
println(totalApts) // Prints: []
```

`addAll()` and `clear()` are highly useful, quick, and efficient ways of adding or removing large amounts of data from a collection at once.

7.5 Adding and Removing Elements Instructions

1.

Codey and his friends want to host a talent show, but before they proceed, they need to ensure that there are at least 5 participants. They've gathered the following list of names in **TalentShow.kt**.

It's our job to ensure that no name appears twice and that there are enough participants for the show to take place.

First, use the `addAll()` function to add all of the list names to the `uniqueParticipants` set at once.

2.

On the following line, use the `size` property to determine how many participants are in the set. Create the following String template and wrap it in a `println()` statement:

```
"The talent show has [setName.size] unique participants."
```

3.

Great Work! Since there were more than 5 unique participants, the talent show was able to take place, and Codey won for knowing all 88 constellations by heart.

☆

Now it's time to clear all of the participants from the set. On the following line, use the correct function to empty the set.

Use a `println()` statement to output `uniqueParticipants` and ensure all values have been cleared.

```
fun main() {
```

```
    var uniqueParticipants = mutableSetOf<String>()
```

```
    var participants = listOf("elePHPant", "Gopher", "Lenny", "Moby Dock", "Codey",  
    "Gopher")
```

```
    // Write your code below 
```

```
}
```

OUTPUT

The talent show has 5 unique participants.

```
[]
```

7.6 Additional Set Functions

In addition to adding and removing elements from a set, the set collection supports a myriad of other functionalities. You can find the entire list in Kotlin's [documentation](#).

In this exercise, we'll explore some of the most common set [functions](#) using the following example:

```
var numbers = setOf(10, 15, 20, 25)
```

To retrieve, the first element of any set, we can use the `first()` function:

```
numbers.first() // 10
```

To retrieve the last element of any set, we can use the `last()` function:

```
numbers.last() // 25
```

Finally, if we'd like to sum up all of the elements in a set, we can use the `sum()` function:

```
numbers.sum() // 70
```

Note: To see the output of each function call on a set, it must be wrapped in a print statement.

7.6 Additional Set Functions Instructions

1.

In **Midterm.kt**, we'll write a program that determines if a class has failed or passed a midterm.

First let's gather the grades, and store them in a mutable set, `testGrades`, with the following numerical values:

- 65
- 50
- 72
- 80
- 53
- 84

2.

Next, we'll utilize the following formula to declare 3 variables that we'll use to calculate the average:

average = sum / number of elements

- First, declare the variable, sum, and within it store the result of calling the sum() function on testGrades.
- Declare the variable, numStudents, and within it store the result of using the size property on testGrades.
- Finally, declare the variable, average and assign it the division expression of sum divided by numStudents.

3.

Lastly, set up a conditional that follows this logic:

- If the average grade is less than 65, then output the text, "Failed".
- Otherwise, output the text, "Passed".

```
fun main() {
```

```
// Write your code below 🏰
```

```
}
```

OUTPUT

Passed

7.7 Review

Great job! In this lesson, we've learned the following concepts:

- A set is an unordered Kotlin collection that stores unique elements.
- An immutable set is declared with the `setOf` keyword and indicates a set whose values cannot change throughout a program.
- A mutable set is declared with the `mutableSetOf` keyword and indicates a set whose values can be altered.
- Elements within a set can be accessed using the `elementAt()` or `elementAtOrNull()` [functions](#).
- The `addAll()` and `add()` functions are used to add elements to a collection.
- The `first()` and `last()` functions return the first and last elements of a collection.
- The `clear()` function removes all elements from a collection.

You've now covered the second major collection that exists in Kotlin as well as many other programming languages. Feel free to utilize the empty **Review.kt** file and output terminal on the right to hone your understanding of [sets](#) and practice writing Kotlin code.

7.7 Review Instructions

Click **Up Next** when you're ready to move on.

```
fun main() {  
    // Write your code below  
  
}
```

8. MAPS

8.1 Introduction to Maps

The final collection we'll cover in this section is a map. A map consists of a group of key-value pairs where each key corresponds to a single value. A key-value pair is referred to as an *entry* and resembles two connected pieces of data where a key is a unique identifier, and the value represents its corresponding data.

Maps are applicable in a variety of programming scenarios and their syntax resembles that of real data from the web. Take a look at the image on the right. The legend is in the form of a Kotlin map, and it provides information about the type of animals the journeyer should expect to encounter and watch out for in their path.

The name of each place is in the form of a key, and each corresponding animal is the value. Notice how data in a Kotlin map is enclosed within curly braces as opposed to square brackets for lists and [sets](#).

In this lesson, we'll explore other unique characteristics of the map collection as well as how to declare a map and utilize its various built-in operations.

8.1 Introduction to Maps Instructions

When you're ready to move on, click **Next** to learn more about the map collection in Kotlin.



8.2 Creating an Immutable Map

An immutable map contains read-only entries that cannot be altered. Immutable maps are most commonly used when all of the required data already exists in the group, and no further modification is needed.

To declare an immutable map, we need to use the `mapOf` keyword followed by a pair of parentheses:

```
val/var mapName = mapOf(key1 to val1, key2 to val2, key3 to val3)
```

- Within the parentheses must exist unique keys along with their corresponding values.
- The `to` keyword is used to link each key to its value.
- Each entry must be separated by a comma.

Note: The keys within a map must be unique, however, the values can duplicate.

To apply what we've learned so far, let's create a map of instruments:

```
var instruments = mapOf("String" to "Cello", "Brass" to "Saxophone",  
"Woodwinds" to "Flute")
```

Printing the value of instruments will result in the following output:

```
println(instruments) // {String=Cello, Brass=Saxophone, Woodwinds=Flute}
```

Notice how...

- Each key is linked to its value with an = as opposed to the to keyword used in the original declaration.
- The map is wrapped in curly braces as opposed to square brackets or parentheses.

8.2 Creating an Immutable Map Instructions

1.

In the previous exercise, we saw the following legend on a map:

```
{valley=scorpions, creek=snakes, forest=bears}
```

This code represents the output of a Kotlin map. In **Journey.kt**, let's write code to reproduce this output.

First, declare a variable, `codeysMap`, and assign it to an immutable map with the above key-value pairs.

2.

Next, wrap the name of the map in a `println()` statement to confirm that we've generated the expected output.

```
fun main() {  
    // Write your code below ?  
  
}
```

OUTPUT

```
{valley=scorpions, creek=snakes, forest=bears}
```

8.3 Retrieving Keys and Values

As we've seen with other collection types in Kotlin, we are able to retrieve and access the contents of a collection with the help of built-in [functions](#) and other tools. With the map collection, we are able to retrieve a value using the shorthand `[key]` syntax.

Assume the following example that holds famous band names as keys and lead singers as values:

```
var leadSingers = mapOf("The Rolling Stones" to "Mick Jagger", "Blondie" to  
"Debbie Harry", "Queen" to "Freddie Mercury")
```

We can access any value by wrapping its key in square brackets:

```
leadSingers["Blondie"] // Debbie Harry
```

If we're building an application that will display the names of the bands at the top of the page, we can gather all keys at once using the keys property:

```
leadSingers.keys // [The Rolling Stones, Blondie, Queen]
```

Similarly, a set of all map values can be returned using the values property:

```
leadSingers.values // [Mick Jagger, Debbie Harry, Freddie Mercury]
```

Each expression can be wrapped in a print statement to see its output or stored in a variable for later use.

8.3 Retrieving Keys and Values Instructions

1.

In **Transportation.kt**, we've declared a map containing various forms of unique transportation in different countries. Retrieve a transportation mode that exists in Peru and wrap it in a print statement using a String template:

```
"A unique mode of transportation that exists in Peru is mapName[key]."
```

Make sure to replace the placeholders, mapName and key, with the correct names.

2.

On the following line, output all keys from the uniqueTransport map using a single property. Wrap this code in a println() statement.

Next, output all values from the map using a single property. Wrap this code in a `println()` statement as well.

```
fun main() {  
    var uniqueTransport = mapOf("India" to "Tuktuk", "Egypt" to "Felucca", "Peru"  
    to "Totoro Boat", "Hong Kong" to "Junk Boat")  
  
    // Write your code below 🚗  
  
}
```

OUTPUT

A unique mode of transportation that exists in Peru is Totoro Boat.

[India, Egypt, Peru, Hong Kong]

[Tuktuk, Felucca, Totoro Boat, Junk Boat]

8.4 Creating a Mutable Map

As we've seen with lists and [sets](#), a map can be declared mutable or immutable. So far, we've only worked with maps whose entries are used for read-only operations. Let's explore how to declare mutable maps and have more control of the data they hold.

To declare a mutable map, use the term, `mutableMapOf` in a map declaration:

```
var/val mapName = mutableMapOf(key1 to val1, key2 to val2, key3 to val2)
```

The above syntax can be used to declare the following mutable map of students and their age:

```
var students = mutableMapOf("Willow" to 15, "Elijah" to 17, "Florence" to 16, "Muhammed" to 15)
```

Using the [key] syntax, we are able to retrieve and reassign key values. Assume Willow just celebrated her birthday and turned 16. We can update the age value as so:

```
students["Willow"] = 16
```

Notice how we've used the assignment operator to assign a new value to the key, "Willow".

Note: In a mutable map, values can be updated, however, keys can never change. In order to update a key, the pair must be removed entirely and re-added. In the next exercise, we'll see how to add and remove values from a map.

Instructions

1.

In **Shows.kt**, declare a mutable map, `tvShows` that will store key-value pairs of a show's name and its total number of episodes. Use the following information to populate your map:

- "The Big Bang Theory": 278
- "Modern Family": 250
- "Bewitched": 254
- "That '70s Show": 200

2.

The Big Bang Theory's final, 279th episode, aired on May 16th, 2019. Update the value of this show in the map.

On the following line, wrap tvShows in a print statement to see the final map.

```
fun main() {  
    // Write your code below   
  
}
```

OUTPUT

```
{The Big Bang Theory=279, Modern Family=250, Bewitched=254, That '70s  
Show=200}
```

8.5 Adding and Removing Entries

Since a mutable map supports read and write operations, we are able to manipulate its contents by adding or removing key-value pairs.

Assume we have the following mutable map that stores a country's national tree:

```
var nationalTrees = mutableMapOf("Italy" to "Strawberry Tree", "Greece" to  
"Olive", "Romania" to "Oak", "Canada" to "Maple")
```

We can utilize Kotlin's built-in put() function to add a new entry to the mutable map:

```
nationalTrees.put("Albania", "Olive")
```

Within the parentheses of the put() function, we've placed the key followed by a comma and the value. The key-value pair will be added to the end of the map.

Say that for our program, we only need to track national trees in Europe. To remove "Canada" from the map, we can use the `remove()` function:

```
nationalTrees.remove("Canada")
```

Notice how we only need to specify the unique identifier, or key, for the pair rather than the entire entry.

Outputting the final map with a `println()` statement will result in:

```
{Italy=Strawberry Tree, Greece=Olive, Romania=Oak, Albania=Olive}
```

Explore the other various map-specific operations [here](#), as well as other general map supported functionalities in Kotlin's [documentation](#).

8.5 Adding and Removing Entries Instructions

1.

In **Paintings.kt**, we've populated a mutable map of famous paintings in [MoMa](#) and their painters. We'll use our knowledge of adding and removing entries to update this map.

The `if` expression utilizes one of map's built-in functions, [containsValue\(\)](#), to check if a painting by Claude Monet does not exist in the map.

Within its body, use another function to add, "Water Lillies", painted by Claude Monet to the map.

2.

The Mona Lisa permanently resides at The Louvre in Paris, France, and not at the MoMA. Remove the key-value pair associated with this painting.

On the following line, use a print statement to output the final map.

```
fun main() {  
    // Write your code below ?  
  
    var momaPaintings = mutableMapOf("Les Demoiselles d'avignon" to "Pablo  
Picasso", "The Starry Night" to "Vincent Van Gogh", "Mona Lisa" to "Leonardo Da  
Vinci", "The Persistence of Memory" to "Salvador Dali")  
  
    if (!momaPaintings.containsValue("Claude Monet")) {  
        // Write your code for Step 1 here  
  
    }  
  
    // Write your code for Step 2 here  
  
}
```

OUTPUT

```
{Les Demoiselles d'avignon=Pablo Picasso, The Starry Night=Vincent Van Gogh,  
The Persistence of Memory=Salvador Dali, Water Lilies=Claude Monet}
```

8.6 Review

Woohoo! Excellent work. You've completed the final lesson on collection types in Kotlin. Let's review what was covered:

- A map is a collection that stores key-value pairs of data.
- The keys within a map must be unique, however, the values can duplicate.
- An immutable map is declared with the `mapOf` keyword and indicates a map whose entries cannot change throughout a program.
- A mutable map is declared with the `mutableMapOf` keyword and indicates a Map whose entries can be altered.
- Values within a map can be accessed using shorthand `[key]` syntax.
- To return all the values in a map, use the `values` property. To return all the keys in a map, use the `keys` property.
- The `put()` function accepts a key and a value to add to a map.
- The `remove()` function accepts a key and removes the entry associated with that key.

You've now covered the third and final major collection that exists in Kotlin. Feel free to utilize the empty **Review.kt** file and output terminal on the right to hone your understanding of maps and practice writing Kotlin code.

8.6 Review Instructions

Click **Up Next** when you've completed the review and are ready to move on.

```
fun main() {  
    // Write your code below
```

```
}
```

8. QUIZ

1. Fill in the blanks to add and remove entries from the given map.

var fictionMovies = _____ ("Action and Adventure" to "Jumanji", "Drama" to "The Martian", "Fairytale" to "The Beauty and the Beast", "History" to "Hidden Figures")

fictionMovies._____("Mystery" _____ "Sherlock Holmes") // Add the "Mystery" movie, "Sherlock Holmes" to the map

fictionMovies._____(_____) // Remove the non-fiction entry from the map

- :
- clear
- put
- ,
- "History"
- remove
- mutableSetOf
- "Fairytale"
- mapOf
- mutableMapOf

Click or drag and drop to fill in the blank

2. True or False: The map collection supports the properties, **keys** and **values**, that are used to return a map's keys and values in the form of a set.

- a) **True**
- b) False

3. Which of the following functions CANNOT be used on an immutable list?

- a) None of these choices.
- b) The **random()** function
- c) The **remove()** function
- d) The **contains()** function

4. Select the group of data that would best be stored in a map collection over a list or set.

- a) A group of all unique usernames.
- b) A group of all usernames including duplicates.
- c) A group of usernames and their corresponding email addresses.

5. Which property is used to retrieve the number of elements in a collection?

- a) The **length** property
- b) The **numberOfElementsInACollection** property
- c) The **total** property
- d) The **size** property

6. What is the difference between a mutable list and an immutable list?

- a) A mutable list contains ordered elements, meanwhile an immutable list stores unordered elements.
- b) A mutable list possesses read-only functionalities since its contents cannot be altered, however, an immutable list possesses read and write functionalities since its contents can be changed throughout a program.

- c) A mutable list must be shorter in size than an immutable list since its contents can change throughout a program.
- d) A mutable list possesses read and write functionalities, however, an immutable list possesses read-only functionalities.

7. Fill in the blanks with the type of Set needed for this program.

```
var womenInTech = _____("Ada Lovelace", "Grace Hopper", "Radia Perlman",  
"Sister Mary Kenneth Keller")  
womenInTech.add("Katherine Johnson")
```

```
println(womenInTech) // Prints: [Ada Lovelace, Grace Hopper, Radia Perlman,  
Sister Mary Kenneth Keller, Katherine Johnson]
```

- setOf
- mutableSetOf
- immutableSetOf

Click or drag and drop to fill in the blank

8. Fill in the blanks to correctly retrieve, **"Yahoo"**, from the list.

```
var searchEngines = listOf("Google", "Bing", "Yahoo", "Yandex")
```

```
_____[_____]
```

- 3
- searchEngines
- 1
- "Yahoo"
- listOf
- searchengines

- 2

Click or drag and drop to fill in the blank

9. To retrieve a single element from a set, we can use the **elementAt()** function where the position of the element would be specified between the parentheses.

Why are we able to specify the position of an element if a set is an unordered collection?

```
var cuisines = setOf("French", "American", "Italian", "Carribean")  
cuisines.elementAt(2) // Prints: Italian
```

- a) A Set is an ordered collection, thus we can use **elementAt()** to retrieve an element from using its index.
- b) Although the Set is not an ordered collection, we can still use the **elementAt()** function because the Set gets converted into a List and a List has indexed access.
- c) Although the Set is not an ordered collection, it still provides indexed access which we can use to retrieve elements.
- d) Sets can store elements in the order of which each element gets added. We can rely on this order to still retrieve an element from a numerical position.

10. What is a list?

- a) A list is a type of collection that stores key-value pairs.
- b) A list is a type of collection that stores ordered elements.
- c) A list is a type of collection that stores only Strings.
- d) A list is a primitive data type that exists in Kotlin.

11. Fill in the blanks to retrieve the value **"The Big Apple"** from the map.

```
var cityNicknames = mapOf("Denver" to "The Mile High City", "Seattle" to  
"Emerald City", "Philadelphia" to "City of Brotherly love", "New York City" to "The  
Big Apple")
```

cityNicknames _____

- }
- [
- "New York City"
- "Seattle"
- {
- "The Big Apple"
- "Denver"
-]

Click or drag and drop to fill in the blank

8.PROJECT

Animal Sanctuary

Did you know that chimpanzees share 95 to 98 percent of the same DNA with humans? Chimpanzees are some of the most intelligent, skillful, and remarkable animals on the planet and it's paramount that we do everything in our power to protect and cherish their species.

For this reason, we are going to take a virtual trip over to Africa, home to many chimpanzees, and volunteer at an animal sanctuary where we'll be taking special care of chimps. To do right by the chimps and be as successful as possible in our role, we decide to write a Kotlin program that will help us keep track of the chimps we're caring for and our daily responsibilities.

We'll have Bonnie, Jubilee, Frodo, and Foxie: 4 young chimpanzees in our care. Foxie is the youngest of the group so she will require some extra attention.

Our program will ensure that during our shift we've...

- Fed all the chimps
- Played a random game with them
- Conducted a health check on Foxie

If at any point you'd like to reference the solution code, be sure to check out the link in the final step. Good luck!

Preparing for the Day

1.

First and foremost, let's gather our responsibilities for the day and organize them in a list.

Declare a variable, `responsibilities`, which stores a list with the following tasks:

- "feed the chimps"
- "play a random game"
- "conduct a health check on Foxie"

We don't expect to change our responsibilities, so feel free to store this data in an immutable list. Also, make sure to organize your list in the same order as above.

2.

Next, we'll declare 3 variables to keep track of our time and how many responsibilities we've completed.

First, declare a variable, `responsibilitiesComplete`, and assign it the value, 0.

Additionally, declare a variable, `timeSpent`, and also assign it the value of 0.

As we go through our program and check off tasks, we'll add to these values.

Finally, declare a variable, `totalShiftTime`, which represents the hour length of our daily shift. Set this value to 4. This value will not change throughout the program.

3.

Finally, we're going to need to create the following two map collections:

- `foxiesHealthCheck` should be defined as an empty mutable map with `String` keys and `Any?` values. Specifying `Any?` will allow us to store a value of any type. We will add data on Foxie's well being to this map later in the project.
- `chimpsHaveEaten` should be defined as a mutable map populated with the following values:
 - "Bonnie" to false
 - "Jubilee" to false
 - "Frodo" to false
 - "Foxy" to false

Feeding the Chimps 🍏

4.

We've clocked in and we're ready to tackle the first responsibility on our list!

Add a `println()` statement specifying what tasks needs to be accomplished. Within the print statement, write code to output the first responsibility:

`First, responsibilities[index].`

5.

Now it's time to feed the chimps one by one. We'll start with Bonnie. Construct another print statement that relays this information:

Feeding Bonnie....

Following this print statement, write code to update the "Bonnie" key in the map to hold the value true now that she's been fed.

Repeat this process for each chimp in the map.

6.

Before we move on to the next responsibility, we must track the time that it took and how many responsibilities we've completed thus far.

We spent an hour feeding all the chimps. Add 1 to timeSpent.

We've completed a single responsibility; also add 1 to responsibilitiesComplete.

Lastly, create a println() statement that outputs the following text:

All chimps have now been fed! You've completed [responsibilitiesComplete] / [total responsibilities]) responsibilities.

Replace [total responsibilities] with a property appended to responsibilities that returns the number of items in the list.

Playing a Random Game ?

7.

Now, let's move on to the second responsibility on the list.

Add a print statement specifying what task needs to be accomplished now. Within the print statement, write code to output the second responsibility:

Next, responsibilities[index].

8.

Next, the chimps will get to play a random game that's fun and stimulates their thinking.

First, declare a set, `games`, containing the following values:

- "tug-of-war with a blanket"
- "catch and throw"
- "number game"

On the following line, use a function on `games` to return a random game from the set. Store this code in a variable, `randomGame`.

Lastly, add a print statement to output the value of `randomGame` to see which game the chimps will play.

9.

While the chimps finish their game, let's write code to wrap up this task by updating the time spent and the responsibilities we've completed thus far.

We've spent an hour preparing the game and playing with the chimps.
Add 1 to `timeSpent`.

Since we've completed another responsibility, also
add 1 to `responsibilitiesComplete`.

Lastly, create a print statement that outputs the following text:

Each chimp has now played a game of `[randomGame]`! You've completed `[responsibilitiesComplete]` / `[total responsibilities]` responsibilities.

Conducting a Health Check

10.

Now, let's move on to the third and final item on our to-do list.

Add a print statement specifying what task needs to be accomplished now. Within the print statement, write code to output the third responsibility:

Next, responsibilities[index].

11.

To do a health check on Foxie, we need to measure her temperature and assess her mood. As we complete each task, we'll update the foxiesHealthCheck map with information on how she's feeling today. Let's get started.

Grabbing the thermometer and applying it under her arm, we see that Foxie's temperature is 32.7. Let's write code that adds a "temperature" key with a value of 32.7 to the map.

Next, assessing Foxie's mood, we conclude that she's in a "happy" state. Let's write code that adds a "mood" key with a value of "happy" to the map.

Finally, let's add a print statement to relay our findings:

Foxie has a temperature of [temperature] and is feeling [mood].

12.

Now that we've completed the final task, let's update the time spent and the responsibilities we've completed.

We've spent an hour completing the health assessment. Add 1 to timeSpent.

Similarly, add 1 to responsibilitiesComplete.

Lastly, create a print statement that outputs the following text:

You've now completed [responsibilitiesComplete] / [total responsibilities] responsibilities.

Wrapping Up

13.

Finally, we'll add some logic to our program to check if we've completed all of our responsibilities on time.

Set up an if-else expression with an else-if condition that accomplishes the following:

- Check if the `timeSpent` was less than or equal to `totalShiftTime` and if all of the responsibilities were complete.
- If so, output text that congratulates the caregiver on a great job of completing the responsibilities on time.
- Check if the `timeSpent` was greater than or equal to `totalShiftTime` and if all of the responsibilities were complete.
- If so, output text that states that all responsibilities were complete but with overtime.
- In all other cases, output text that says all of the responsibilities were not complete and took extra time.

14.

Excellent work! 🙌 You've completed the project and built a useful program for tracking the responsibilities of a caregiver.

Run your program several times to ensure it's working as expected. You may see that the text is all jumbled together. Use your knowledge of escape sequences to format some of the print statements to your liking.

Finally, test your code with different values for `timeSpent` and see the different outcomes for the random games.

Optional Challenges

15.

If you're up for a challenge, check out these additional tasks:

- A chimp's average body temperature is 32.7 degrees Celsius. Anything higher than that should be notified to the vet. Add this logic to the program.
- Optimize parts of your program as you see fit. For example, how can we improve upon and avoid the repetitive use of responsibilities.size?
- Add an additional responsibility to the list and adjust the project as needed.

```
fun main() {  
    // Write your code below  
  
}
```

OUTPUT

First, feed the chimps

Feeding Bonnie...

Feeding Jubilee...

Feeding Frodo...

Feeding Foxy...

All chimps have now been fed! You've completed 1 / 3 responsibilities.

Next, play a random game

number game

Each animal has now played a game of number game. You've completed 2 / 3 responsibilities.

Next, conduct a health check on Foxie

Foxie has a temperature of 37.2 and is feeling Happy.

You've now completed 3 / 3 responsibilities.

Awesome work! You've completed all of your responsibilities for the day, and you can clock out.

9. LOOPS

9.1 Introduction to Loops

Why write 100 lines of code when you can write 2? When we come across code we want to repeat, we can rely on [loops](#). Loops give us the ability to repeat specified blocks of code as many times as we want.

Let's say we wanted to output elements in a list. The process wouldn't be that bad if the list had 5 elements. We could write five `println()` statements. What if the list had 100 elements? Or 1,000? Or even 100,000?!

Loops answer the call when we need to repeat a task over and over again, even if it is millions of times.

In this lesson, we will look at loops that will repeat code a defined number of times or based on a specific condition. We will look at how to combine loops to obtain a higher number of repetitions as well as how to exit loops early or skip code repetitions.

By the end of this lesson you will understand the best way to repeat tasks in a controlled manner while keeping your code organized and readable.

9.1 Introduction to Loops Instructions

Looking at the animation on the right, we can see that Codey is hiking through the Land of Loops. We use a type of Kotlin loop called `while` to instruct Codey to continue to hike as long as `time < 18`. Once time is equal to 18, Codey will stop hiking and set up camp.

This is just one way to construct a loop, so just like Codey, let's start our journey into the Land of Loops! Click **Next** to get started with loops using Kotlin.



9.2 for Loop

When we know how many times we want a section of code to repeat, we often use a *for loop*.

A for loop starts with the `for` keyword and is followed by a statement that defines a loop variable followed by identifying an *iterator*:

```
for(i in 1..4) {
  println("I am in a loop!")
}
```

- `for` is a keyword used to declare a for loop.

- We define `i` as the loop variable. This variable holds the current iteration value and can be used within the loop body.
- The `in` keyword is between the variable definition and the iterator.
- The range `1..4` is the for loop iterator.

An [iterator](#) is an object that allows us to step through and access every individual element in a collection of values. In most cases, the iterators we use in for [loops](#) are ranges and collections. In this example, the range has 4 elements so the for loop repeats 4 times:

```
I am in a loop!  
I am in a loop!  
I am in a loop!  
I am in a loop!
```

It is important to note that the loop variable only exists within the loop's code block. Trying to access the loop variable outside the for loop will result in an error.

Here is an example of using the loop variable within the loop body:

```
for (i in 1..4) {  
  println("i = $i")  
}
```

While the above example uses literal numbers `1..4` to define the range, we can also use [variables](#) to define the boundaries of our loop's range, giving us more dynamic control over our loop's iteration:

```
var num = 4  
for (i in 1..num) {  
  println("i = $i")  
}
```

Both of the code snippets produce the same final output:

```
i = 1  
i = 2  
i = 3  
i = 4
```

9.2 for Loop Instructions

1.

Create a for loop that outputs "Hello, Codey!" five times.

Make sure to use:

- i as the loop variable.
- The range 1 through 5 as the iterator.
- a println() in the loop body.

2.

Great job on setting up your first Kotlin for loop!

Now instead of just outputting text, it's time to use the loop variable i in the loop body.

Add another println() with a string template directly below the first println() so the loop creates the following output:

```
Hello, Codey!  
i = 1  
Hello, Codey!  
i = 2  
Hello, Codey!  
i = 3  
Hello, Codey!
```

```
i = 4
Hello, Codey!
i = 5
```

```
fun main() {
    // Write your code below

}
```

OUTPUT

```
Hello, Codey!
i = 1
Hello, Codey!
i = 2
Hello, Codey!
i = 3
Hello, Codey!
i = 4
Hello, Codey!
i = 5
```

9.3 Controlling the Iteration

Sometimes we want to count backwards, or count by 2s, or both! Using certain [functions](#) alongside or instead of the normal range operator (..) can enhance the iterative abilities of our for [loops](#). The

functions `downTo`, `until` and `step` give us more control of a range and therefore more control of our loops.

The `downTo` function simply creates a reverse order group of values, where the starting boundary is greater than the ending boundary. To accomplish this, replace the range operator (`..`) with `downTo`:

```
for (i in 4 downTo 1) {  
    println("i = $i")  
}
```

We can see in the output that the first number in `i` is 4 and the last is 1:

```
i = 4  
i = 3  
i = 2  
i = 1
```

The `until` function creates an ascending range, just like the (`..`) operator, but excludes the upper boundary:

```
for (i in 1 until 4) {  
    println("i = $i")  
}
```

The upper boundary, 4, is not included in the output:

```
i = 1  
i = 2  
i = 3
```

Up until now, each of these functions, including the range operator (`..`), have counted up or down by one. The `step` function specifies the amount these functions count by:

```
for (i in 1..8 step 2) {  
    println("i = $i")  
}
```

The loop variable `i` now increases by 2 for every iteration. The last number output is 7, since 2 steps up from that is 9 which is outside the defined range, 1..8:

`i = 1`

`i = 3`

`i = 5`

`i = 7`

9.3 Controlling the Iteration Instructions

1.

Let's look at how we can change the behavior of ranges in for loops by implementing a loop that counts backwards.

Create a for loop that contains:

- `i` as the loop variable.
- an iterator that starts at 10 and ends at 1.
- a `println()` statement in the loop body with the string template `"i= $i"`.

2.

Below the first loop, implement a for loop that counts up but stops just before the upper boundary of the range. Make sure it contains:

- `j` as the loop variable.
- the range 1 **up to but not including** 10 as the iterator.

- a `println()` statement in the loop body with string template `"j = $j"`.

3.

Finally, implement a for loop that iterates over a range in steps greater than 1. Make sure it contains:

- `k` as the loop variable.
- a range 1 through 10 as the iterator that counts up by 2.
- a `println()` statement in the loop body with string template `"k = $k"`.

Run the code and you will see that the new loop does not output the iterator's upper boundary 10. Counting up by 2 from 1 does not include 10.

```
fun main() {
    println("-- 1st for loop output --")
    // Write your code below

    println("\n-- 2nd for loop output --")
    // Write your code below

    println("\n-- 3rd for loop output --")
    // Write your code below

}
```

OUTPUT

```
-- 1st for loop output --
```

i = 10

i = 9

i = 8

i = 7

i = 6

i = 5

i = 4

i = 3

i = 2

i = 1

-- 2nd for loop output --

j = 1

j = 2

j = 3

j = 4

j = 5

j = 6

j = 7

j = 8

j = 9

-- 3rd for loop output --

k = 1

k = 3

k = 5

k = 7

k = 9

9.4 Iterating Through Collections

Instead of using a range, we can use the structure of a collection as an iterator. A for loop will iterate through each of the collection's elements with the loop variable holding the value of the current element. We will focus on lists and [sets](#) in this exercise and maps in the next one.

```
val fruitList = listOf("apples", "oranges", "bananas")
```

```
for (fruit in fruitList) {  
    println("I have $fruit.")  
}
```

In this example, we declare fruit as the loop variable and use fruitList as the iterator. Each iteration of the loop will set fruit to the value of the current element of fruitList. Therefore, the number of elements contained in fruitList determines the number of times println("I have \$fruit.") is executed:

I have apples.

I have oranges.

I have bananas.

When we first learned about lists and sets in the collections lesson, we discussed their commonality as well as their differences. An additional similarity is that the syntax for iterating through a list and a set is the same.

When iterating through a collection, it is often useful to know what element number, or iteration, we are at. To iterate through the indices of a collection you can use its `indices` property:

```
val fruitSet = setOf("apples", "oranges", "bananas")
```

```
for (setIndex in fruitSet.indices) {  
    println("Index = $setIndex")  
}
```

Here we see the indices of `fruitSet` output. Remember that the first index of a list or a set is always 0:

Index = 0

Index = 1

Index = 2

We can also get the index AND the iterator element using the collection's `withIndex()` function. In this case we need to [destructure](#) the loop variable declaration by declaring two loop [variables](#) and enclosing them in parentheses:

```
val fruitList = listOf("apples", "oranges", "bananas")
```

```
for ((listIndex, fruit) in fruitList.withIndex()) {  
    println("$listIndex is the index of the element $fruit")  
}
```

Using `withIndex()` and destructuring, we are able to access both the index and the element of `fruitList`:

0 is the index of the element apples

1 is the index of the element oranges

2 is the index of the element bananas

9.4 Iterating Through Collections Instructions

1.

Now you'll use collections as iterators. To start, implement a for loop using the list `mySchedule`. The loop should contain:

- `task` as the loop variable.
- the list `mySchedule` as the iterator.
- a `println()` statement in the loop body that outputs the loop variable.

2.

Great, now look at the set `myTasks` which is declared with the same elements as the list. We know that `myTasks` will have fewer elements since some of them are repeated and will only be represented once in the set. Let's confirm this by printing out the indices AND the elements of the set.

Create another for loop that contains:

- `taskIndex` and `task` as destructured loop variables. Be sure to separate them with a comma and enclose them in parentheses.
- `myTasks` using the `withIndex()` function as the iterator.
- a `println()` statement in the loop body with the output `taskIndex` and `task` separated by a colon (`:`). Example output: `0: Eat Breakfast`

```
fun main() {
```

```
    val mySchedule = listOf("Eat Breakfast", "Clean Up", "Work", "Eat Lunch", "Clean Up", "Work", "Eat Dinner", "Clean Up")
```

```
val myTasks = setOf("Eat Breakfast", "Clean Up", "Work", "Eat Lunch", "Clean  
Up", "Work", "Eat Dinner", "Clean Up")
```

```
println("-- mySchedule Output --")
```

```
// Write your code below
```

```
println("\n-- myTasks Output --")
```

```
// Write your code below
```

```
}
```

OUTPUT

```
-- mySchedule Output --
```

```
Eat Breakfast
```

```
Clean Up
```

```
Work
```

```
Eat Lunch
```

```
Clean Up
```

```
Work
```

```
Eat Dinner
```

```
Clean Up
```

```
-- myTasks Output --
```

```
0: Eat Breakfast
```

```
1: Clean Up
```


- 2: Work
- 3: Eat Lunch
- 4: Eat Dinner

9.5 Iterating Through Maps

Unlike a list or a set, a map is a collection of entries. When using a map as a for loop iterator, we can iterate through each entry of the map or through just the keys, or just the values.

When iterating through a map, the loop variable will hold one entry per iteration. The entry's key and value can be accessed using the properties, `key` and `value`:

```
val myClothes = mapOf("Shirts" to 7, "Pairs of Pants" to 4, "Jackets" to 2)
```

```
for (itemEntry in myClothes) {  
    println("I have ${itemEntry.value} ${itemEntry.key}")  
}
```

Notice that in order to access the attributes in a String template, we surround the loop variable and the attribute in curly brackets.

We can also access the key and value of each entry by destructuring using two loop [variables](#) in parentheses. The first variable is the entry's key and the second is the value:

```
val myClothes = mapOf("Shirts" to 7, "Pairs of Pants" to 4, "Jackets" to 2)
```

```
for ((itemName, itemCount) in myClothes) {  
    println("I have $itemCount $itemName")  
}
```

Both examples have the same output:

I have 7 Shirts
I have 4 Pairs of Pants
I have 2 Jackets

Lastly, we can iterate through just a map's keys or values. A map has the properties keys and values which can each be used as an iterator:

```
val myClothes = mapOf("Shirts" to 7, "Pairs of Pants" to 4, "Jackets" to 2)
```

```
println("KEYS")
for (itemName in myClothes.keys) {
    println(itemName)
}
```

```
println("\nVALUES")
for (itemCount in myClothes.values) {
    println(itemCount)
}
```

Here is the output:

KEYS
Shirts
Pairs of Pants
Jackets

VALUES
7
4
2

9.5 Iterating Through Maps Instructions

1.

The map `favoriteColors` holds the names of people and their favorite colors. The keys of the map are the people's names and the values of the map are their favorite colors. Start by iterating through the map `favoriteColors` to access each entry and output its key and value.

Implement a for loop that contains:

- `favoriteEntry` as the loop variable
- `favoriteColors` as the iterator
- a `println()` statement in the loop body that outputs the key and value of each entry separated by a colon. Example output, `Jesse: Violet`

Be sure to use curly brackets in the string template when accessing the attributes of the entry in the loop variable.

2.

Nice work. Now you're going to create a loop that iterates through just the map's values.

Create another for loop that contains:

- `color` as the loop variable
- the `values` attribute of `favoriteColors` as the iterator.
- a `println()` statement in the loop body that outputs the value of each entry.

```
fun main() {  
    val favoriteColors = mapOf("Jesse" to "Violet", "Megan" to "Red", "Tamala" to  
    "Blue", "Jordan" to "Pink")
```

```
println("\n-- Key: Value Output --")
```

```
// Write your code below
```

```
println("\n-- Only Values Output --")
```

```
// Write your code below
```

```
}
```

OUTPUT

```
-- Key: Value Output --
```

```
Jesse: Violet
```

```
Megan: Red
```

```
Tamala: Blue
```

```
Jordan: Pink
```

```
-- Only Values Output --
```

```
Violet
```

```
Red
```

```
Blue
```

```
Pink
```

9.6 while Loop

When repeating code we may not have a range or defined collection to dictate the number of [loops](#) we need to execute. In this case, we can rely on a *while loop* which repeats code as long as a specified condition is true. If we revisit the

animation in the first exercise, we can see that Codey uses a while loop to hike until time is no longer less than 17.

A while loop is similar to an if expression because it tests a condition and when the condition is true a body of code is executed. The main difference is the while loop will complete the body of code, check the condition, and repeat the code if the condition is still true. If the condition is false, the while loop is done and the program moves on:

```
var myAge = 16
```

```
while (myAge < 20) {  
  println("I am a teenager.")  
  myAge += 1  
}  
println ("I am not a teenager.")
```

In this example the while loop condition `myAge < 20` initially evaluates to true. The loop body is executed which outputs some text and `myAge` is incremented. The loop repeats until `myAge` equals 20 at which point the loop exits and continues running code outside the loop:

```
I am a teenager. // myAge is 16  
I am a teenager. // myAge is 17  
I am a teenager. // myAge is 18  
I am a teenager. // myAge is 19  
I am not a teenager. // myAge is 20
```

If `myAge` wasn't incremented inside the loop body the loop condition would always be true and repeat forever. This is known as an *infinite loop*. Infinite loops lead to application crashes or other unwanted behavior so it is important that they are avoided.

When a while loop condition never evaluates to true, the entire block of code is skipped:

```
var myAge = 44

while (myAge > 12 && myAge < 20) {
  println("I am a teenager.")
  myAge += 1
}
println ("I am not a teenager.")
```

Since we've set the value of myAge to 44 and the condition will only be true once the value is between 12 and 20, the code block is skipped and the final println() gets executed:

I am not a teenager.

9.6 while Loop Instructions

1.

Create a while loop that:

- repeats as long as the variable counter is less than 5.
- uses println() in the loop body to output the variable counter.
- increments the variable counter by 1 in the loop body.

Note: If the program keeps running and never ends, you have likely created an **infinite loop**. To fix this, refresh the page and take another look at your loop and make sure it has a terminating condition.

2.

A while loop condition can also check the elements of a collection. You'll use another counter variable, index, to access the collection's elements.

Implement a second while loop that:

- has the condition, `schoolGrades[index] != "6th"`.
- uses `println()` in the loop body to output the current element value of `schoolGrades`.
- increments the variable `index` by 1 in the loop body.

When you're done, you'll see that the while loop exits once the element "6th" is the current one.

```
fun main() {  
    var counter = 0  
    var index = 0  
    val schoolGrades = listOf("Kindergarten", "1st", "2nd", "3rd", "4th", "5th", "6th",  
    "7th")  
  
    println("-- counter Output --")  
    // Write your code below  
  
    println("\n-- Elementary School Grades --")  
    // Write your code below  
  
}
```

OUTPUT

-- counter Output --

0

1

2

3

4

-- Elementary School Grades --

Kindergarten

1st

2nd

3rd

4th

5th

9.7 do..while Loop

A do..while loop is just like a while loop except the looping condition is checked at the end of the loop body. This is known as an exit-condition loop and means the code in the body will execute at least once:

```
val myCondition = false
```

```
do {  
    print("I loop once!")  
} while(myCondition)
```

This example shows that even though we have a false condition our loop will still run the loop body once:

I loop once!!!!

One reason we would use a do..while loop is when our loop condition initialization and update commands are the same. Using psuedocode we can compare a while loop with a do..while loop using how someone might decide to go outside:

Is it sunny outside?

```
while ( not sunny ) {
```

```
    Is it sunny outside?
```

```
    Stay inside.
```

```
}
```

Go outside!

The while loop example has a line before the loop to check if it is sunny outside. The loop is entered if it is not sunny and then repeatedly checks if it is sunny outside. When it is sunny, the loop exits and it's time to go outside:

```
do {
```

```
    Is it sunny outside?
```

```
    Stay inside.
```

```
} while ( not sunny )
```

Go outside!

The do..while loop enters the loop and starts by checking if isSunny is true. The loop body repeats if the isSunny is false.

The extra check prior to the loop is unnecessary since it is performed inside the loop anyway. If we have to look outside to get our initial loop condition **and** to update our loop condition, we might as well have that be the loop body and check the condition at the end.

This difference may seem subtle, but if this loop is executed hundreds or thousands of times a second, removing the one line of code using a `do..while` loop may save time.

9.7 `do..while` Loop Instructions

1.

We'll implement a `do..while` loop that performs a Celsius to Fahrenheit temperature conversion.

First, construct a `do..while` loop that checks if `fahr` does not equal 212.0. Inside the loop body:

- paste the following 2 lines of code.

```
fahr = celsiusTemps[index] * fahr_ratio + 32.0
println("${celsiusTemps[index]}C = ${fahr}F")
```

- increment the value of `index`.

Note: If the program keeps running and never ends, you have likely created an **infinite loop**. To fix this, refresh the page and take another look at your loop and make sure it has a terminating condition.

```
fun main() {
    var index = 0

    val celsiusTemps = listOf(0.0, 87.0, 16.0, 33.0, 100.0, 65.0)
    val fahr_ratio = 1.8
    var fahr: Double
```

```
println("-- Celsius to Fahrenheit --")
```

```
// Write your code below
```

```
}
```

OUTPUT

```
-- Celsius to Fahrenheit --
```

```
0.0C = 32.0F
```

```
87.0C = 188.6F
```

```
16.0C = 60.8F
```

```
33.0C = 91.4F
```

```
100.0C = 212.0F
```

9.8 Nested Loops

[Loops](#) inside loops, oh my! A nested loop represents one loop placed inside the body of another loop which results in a higher dimensions of iterations. This strategy is most often used when we'd like to compare the contents of two different collections/iterators.

Below is an example of nested loops. The outer loop iterates through the range 1..2 and the inner loop iterates through the range 'A'..'C':

```
for (i in 1..2) {  
  for (j in 'A'..'C') {  
    println("Outer loop: $i - Inner loop: $j")  
  }  
}
```

It's important to understand that for every iteration in the outer loop, the inner loop will run through all of its own iterations.

In the example above, the inner loop iterates 3 times for each of the outer loop's 2 iterations. Therefore, the `println()` statement is run 6 times:

Outer loop: 1 - Inner loop: A

Outer loop: 1 - Inner loop: B

Outer loop: 1 - Inner loop: C

Outer loop: 2 - Inner loop: A

Outer loop: 2 - Inner loop: B

Outer loop: 2 - Inner loop: C

When beginning with nested loops they may seem difficult to work with, but they make advanced applications easy to implement.

9.8 Nested Loops Instructions

1.

You'll be building a grid of rows and columns much like the ones that exist in Google and Excel sheets using nested loops. First start with a single loop.

Create a for loop that contains:

- `i` as the loop variable.
- a range of 1 through 6 as the iterator.

2.

Now you will implement another loop inside the first one.

Inside the first loop body, create a for loop that contains:

- j as the loop variable.
- a range of 'A' through 'F' as the iterator.
- a print() statement that outputs "\$j\$i ".

The empty space at the end of the String template is used to create a gap between columns which helps us visualize what is happening in the code.

3.

There is now a single line of each loop variable concatenated to each other. The last step will show off the dimensional nature of nested loops.

Outside of the inner loop but still inside of the outer loop:

- add an empty println() statement.

When you run the code you can now see that each inner loop iteration outputs a column, while each outer loop iteration creates a new row.

```
fun main() {  
    // Write your code below  
  
}
```

OUTPUT

```
A1 B1 C1 D1 E1 F1  
A2 B2 C2 D2 E2 F2  
A3 B3 C3 D3 E3 F3  
A4 B4 C4 D4 E4 F4  
A5 B5 C5 D5 E5 F5  
A6 B6 C6 D6 E6 F6
```

9.9 Jump Expressions

The jump expressions, `break` and `continue`, are used to change the standard loop behavior by exiting a loop early or skipping a single repetition.

The `break` expression is used to exit the loop at a particular iteration:

```
val myNumbers = listOf(4, 8, 2, 9, 12, 7, 16, 10, 3)
for (num in myNumbers) {
    if (num > 9) {
        break
    }
    println(num)
}
```

In this example:

- the `for` loop iterates through the list of `Ints`.
- each iteration the current number, `num`, is tested to see if it is greater than 9.
- if false, the number is output.
- if true, the `break` expression exits the loop.

We can see the below output stops before printing the number 12:

Output

```
4
8
2
9
```

The `continue` expression will skip the current execution of the loop body, but the loop will keep going if there are iterations left. If the `continue` expression gets executed, any code left in the current iteration will be skipped:

```
val myNumbers = listOf(4, 8, 2, 9, 12, 7, 16, 10, 3)
for (num in myNumbers) {
    if (num > 9) {
        continue
    }
    println(num)
}
```

This example is the same as the last but we replaced the break with continue. The below output skips all values of num greater than 9, while printing out the rest of numbers:

Output

```
4
8
2
9
7
3
```

The first example outputs the elements in myNumbers **until** you reach one greater than 9. The second example outputs **all** elements in myNumbers **except** those that are greater than 9.

Since jump expressions are usually contained inside if expressions, they give us the ability to add a condition to exit a for loop or using another condition to exit a while loop.

Don't worry if this all doesn't stick right away; keep practicing with each jump expression and you'll quickly get the hang of how and when each one is used.

9.9 Jump Expressions Instructions

- 1.

The code on the right is “looking” for something in different rooms of a house. Thankfully we know that the item being searched for is in the "Living Room".

Your job is to instruct the for loop to stop looking once it reaches "Living Room":

- After the `print("$room: ")` statement add an if expression that tests if the loop variable `room` is equal to "Living Room".
- Inside the if block use a `println()` statement to output the String, "Found It!"
- Inside the if block add the appropriate jump expression that will stop the iteration.

2.

Good work! We also come across situations where we may want to skip a loop iteration. The second for loop in the editor causes a [division by zero](#) error. This is undesirable, and we need to add code to the body of the for loop to avoid this.

- At the start of the loop add an if expression that tests if `number` is 0.
- Inside the if expression add a `continue` expression.

```
fun main() {  
    val rooms = listOf("Kitchen", "Living Room", "Basement", "Bathroom")  
  
    println("-- Room Search --")  
    for (room in rooms) {  
        print("$room: ")  
        // Write your code below  
  
        println("Found Nothing.")  
    }  
}
```



```
println("\n-- Divide By Zero --")
for (number in 12 downTo -12 step 4) {
    // Write your code below

    println("120/number = ${120/number}")
}
}
```

OUTPUT

-- Room Search --

Kitchen: Found Nothing.

Living Room: Found It!

-- Divide By Zero --

120/number = 10

120/number = 15

120/number = 30

120/number = -30

120/number = -15

120/number = -10

9.10 Labeled Jump Expressions

“How would we use jump expressions inside nested loops?” Good question! The answer is *labeled jump expressions*. By adding a label to the outer loop of nested [loops](#), we can execute a jump expression from the inner loop and have it act on the labeled outer loop.

First we need to give a name to our label. In the following example we choose the name `rps`. The outer loop is marked using our label `rps` followed by the `@` symbol:

```
val game = listOf("Rock", "Paper", "Scissor")
```

```
rps@ for (p1 in game) {  
  for (p2 in game) {  
    if (p1 == "Paper") {  
      break@rps  
    }  
    println("$p1 vs. $p2")  
  }  
}
```

The inner loop has an `if` expression that contains a `break` expression with `@rps` appended. When the condition, `p1 == "Paper"` is true, `break@rps` executes and exits the outer loop:

```
Rock vs. Rock  
Rock vs. Paper  
Rock vs. Scissor
```

Now let’s look at what happens in our example when we change the `break` expression to a `continue`:

```
val game = listOf("Rock", "Paper", "Scissor")
```

```
rps@ for (p1 in game) {  
  for (p2 in game) {  
    if (p1 == "Paper") {  
      continue@rps  
    }  
  }  
}
```

```
}  
    println("$p1 vs. $p2")  
}  
}
```

What we see now is the outer loop skips an iteration when `p1 == "Paper"` but resumes with the final iteration when `p1` is equal to `"Scissor"`:

```
Rock vs. Rock  
Rock vs. Paper  
Rock vs. Scissor  
Scissor vs. Rock  
Scissor vs. Paper  
Scissor vs. Scissor
```

Labeled jump expressions give us more control over nested loops to help us achieve the desired behavior from our code. Using them can be tricky, but once understood they can be very powerful.

9.10 Labeled Jump Expressions Instructions

1.

Recall the 6x6 grid with `"A1"` to `"F6"` cells from the previous exercise. We'll take a closer look at how we can manipulate the structure and output of this grid using labeled jump expressions.

First, add a label grid to the outer loop.

2.

Add an if expression inside the inner loop, before the `print("$j$i ")` statement. Have the condition test if the inner loop variable is equal to 'C'.

3.

Inside the if expression add a `continue` expression that jumps to the grid label.

Once you run the code you should see a single line of output. The outer loop numbers go from 1 through 6, but the inner loop letters only go up through 'B'.

It is important to note that the `println()` statement at the end of the outer loop is also skipped. The result is only one row is printed. This shows that a labeled `continue` will skip any code proceeding it in the inner loop and at the end of the outer loop.

```
fun main() {  
    // Write your code below  
    for (i in 1..6) {  
        for (j in 'A'..'F') {  
            // Write your code below  
  
            print("$j$i ")  
        }  
        println()  
    }  
}
```

OUTPUT

A1 B1 A2 B2 A3 B3 A4 B4 A5 B5 A6 B6

9. QUIZ

1. Which program achieves the following output?

1 2 3 4 5 6

1 2 3 4 5 6

- a)

```
for(i in 'a'..'b'){  
    for(j in 1..6){  
        print("$j ")  
    }  
    println()  
}
```
- b)

```
for(i in 'a'..'b'){  
    for(j in 1..6){  
        print("$j ")  
    }  
}
```
- c)

```
for(i in 1..6){  
    for(j in 1..6){  
        print("$j ")  
    }  
    println()  
}
```
- d)

```
for(i in 1..6){  
    for(j in 'a'..'b'){  
        print("$j ")  
    }  
}
```

```
    }  
    println()  
}
```

2. Fill in the blanks to identify the components of a for loop.

```
for ( _____ in _____ ) {  
    _____  
}
```

- Loop Body
- Iterator
- Attribute
- Loop Variable
- Condition

Click or drag and drop to fill in the blank

ANS:

```
for (Loop Variable in Iterator) {  
    Loop Body  
}
```

3. True or False: A continue expression skips the remaining code in a loop body and starts with the next loop iteration.

- a) True
- b) False

4. Fill in the blanks to complete the program.

Implement a labeled jump expression with the label myJump. The code should output the values of i and j when j is less than or equal to 4.

```
_____ for (i in 1..3) {  
    for (j in 1..10) {  
        if (_____)
```

```

    }
    println("i = $i, j = $j")
  }
}

```

- continue@myJump
- myJump@
- j == 4
- j > 4
- break@myJump
- label@

Click or drag and drop to fill in the blank

ANS:

```

myJump@ for (i in 1..3) {
  for (j in 1..10) {
    if (j > 4) {
      break@myJump
    }
    println("i = $i, j = $j")
  }
}

```

5. What number sequence will the following program output?

```

var counter = 5

```

```

while (counter > 0) {
  print("$counter ")
  counter--
}

```

a) 5 4 3 2 1 0

b) 4 3 2 1

c) 5 4 3 2 1

d) 1 2 3 4 5

6. Which of the following numbers will NOT be printed from this code?

```
val numbers = listOf(2,4,6)
```

```
for (num in numbers) {  
    print("${num*2} ")  
}
```

a) 8

b) 12

c) 6

d) 4

7. When iterating through a map, what does the loop variable hold in each iteration?

a) The map's current entry key.

b) A list containing the map's current entry key and value.

c) The map's current entry value.

d) The map's current entry.

8. What does a break expression accomplish?

a) A break expression only works with a label.

b) A break expression exits the loop it is contained in.

c) A break expression is only used to avoid infinite loops.

d) A break expression skips the current loop iteration and starts the next iteration.

9. How many iterations will the following program complete?

```
for (i in 1 until 10 step 3) {  
  print("$i ")  
}
```

- a) 3
- b) 5
- c) 2
- d) 4

10. Select the correct output for the following program?

```
var num = 20
```

```
do {  
  print("$num ")  
  num++  
} while (num < 21)
```

- a) 20
- b) 20 21
- c) 21
- d) There is no output.

11. True or False: An infinite loop occurs when a while or do..while loop condition is never false.

- a) False
- b) True

6.PROJECT

Shape Maker

How many output statements do you need to draw two-dimensional shapes? Using loops, the answer to this question is **2**. In this project, you will create a square and a triangle of any size using nested loops with one `print()` statement and one `println()` statement.

We'll create a basic square pattern using loop concepts we covered in the previous lesson. We'll even take it a step further by getting our program to output a checkerboard pattern.

Then, we'll explore how nesting a while loop inside of a for loop can be used to create a triangle pattern in the output terminal. This will give us a different perspective of nested loops and how to control variables when using while loops. Good luck with the project and have fun!

Creating a Square Pattern

1.

In **Shapes.kt**, there are two groups of variables: one for making a square and one for making a triangle.

The first group of variables will be used to draw a square using nested for loops:

- `sqSide` defines the length of the square's side.
- `sqChar1` and `sqChar2` define the output characters used to create the square and checker board.

`sqChar1` and `sqChar2` each represent one column and are each 3 characters long to account for the space between rows.

Review the variables and move to the next task when ready.

2.

Start your code under the first group of variables by creating a for loop with an iterator that starts at 1 and goes through the value of sqSide. This will act as the outer loop of the code.

Put a blank println() in the loop body. This will create the rows of the square.

3.

Now, let's add an inner loop to output each column in a row.

Inside the outer for loop, create an inner for loop before the println() statement.

Use the same iterator as the outer loop.

Inside the inner loop body use a print() statement to output the value of sqChar1.

Creating a Checker Board Pattern

4.

To create a checker board, we need to use the loop variables in an if expression to decide which character to output.

In the inner loop, replace the print() statement with an if expression.

The if expression should output sqChar1 when both loop variables are even OR both loop variables are odd.

Otherwise, output sqChar2.

Creating a Triangle Pattern

5.

Now we'll move on to creating a triangle pattern.

Take a look at the next group of variables that will be used to draw a triangle:

- triRows defines the triangle rows.
- triCount is a counter for the inner while loop.
- triRowLen keeps track of each row length.

There are also 2 more String variables to use for output: triChar1 and triChar2.

Review the variables and move to the next task when ready.

6.

Start off the code for the triangle pattern by using a for loop that counts down from triRows to 1. This will act as the outer loop of the code.

Inside the loop body insert a blank println().

7.

Inside the for loop, create a while loop before the `println()` statement. The while loop should repeat when `triCount` is less than `triRowLen`.

Increment `triCount` inside the loop. Make sure to run your code only after you've added the increment, otherwise, you'll run into an infinite loop. If you have, you might need to refresh the web page.

8.

Add a `print()` statement to the while loop body directly below the increment to output `triChar1`.

Run the code and look at the output. Do you see a single line of characters? Can you figure out why your output is the way it is?

9.

The while loop is responsible for outputting `triChar1`. There is a possibility that the condition `triCount < triRowLen` is true only for the first outer loop iteration causing the inner loop to only output a single line.

To prevent this, outside the while loop (but inside the for loop), set `triCount` to its initial value, 0. This will ensure that at the start of each iteration of the for loop, `triCount` is less than `triRowLen` and the while loop body will execute.

10.

Run your code. Do you see a square? If so, why? The goal is to make a triangle!

To make our pattern look like a triangle, we need to adjust the length of each new row. To accomplish this, decrement the value of `triRowLen` at the end of the for loop's body.

Creating a Triangle Outline Pattern

11.

We're going to modify our triangle code to just output the triangle's outline. Feel free to make a copy of the code created in steps 6-10 if you want to keep the full triangle output.

If you copied the code to make a 2nd triangle, be sure to set the value of `triRowLen` to `triRows` just prior to the newly copied code.

Confirm the output of a 2nd triangle and move to the next task to implement the triangle outline.

12.

To create a triangle border, we need to output a visible character at the triangle edges and blank space everywhere else.

In the inner loop, replace the `print()` statement with an if expression.

The if expression should test if the output is not an edge. There are three conditions that must be true in order for a character to not be an edge:

- The while loop counter is not equal to 1.
- The while loop counter is not equal to the current row length.
- The for loop variable is not equal to the total number of rows.

If **all** three conditions are true, then output `triChar2`. Otherwise, output `triChar1`

Optional

13.

Great work on the project! Here are a couple more tasks you can work on if you want to keep going.

- Change the values of `sqChar1`, `sqChar2`, `triChar1` and `triChar2` to see how they affects the look of each shape.
- Use different conditions within the if expressions of the square shape to explore what other patterns you can generate.
- In the triangle outline code, instead of using an else to output `triChar1`, use a continue in the if block to skip the rest of the while loop code.

```
fun main() {  
    // Square Variables  
  
    val sqSide = 7  
  
    val sqChar1 = "X "  
    val sqChar2 = "O "  
  
    // Write your code below
```

```
// Triangle Variables
val triRows = 10
var triCount = 0
var triRowLen = triRows
val triChar1 = "/"
val triChar2 = " "

// Write your code below

}
```

OUTPUT

```
X O X O X O X
O X O X O X O
X O X O X O X
O X O X O X O
X O X O X O X
O X O X O X O
X O X O X O X
// // // // // // //
// // // // // // //
// // // // // // //
// // // // // // //
```

/ / / / / / /

/ / / / /

/ / / /

/ / /

/ /

/

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

/ / / / / / / / / / / / /

10.FUNCTIONS

10.1 Introduction to Functions

Now that we understand important programming concepts like control flow, collections, and [loops](#), we can learn how to use [functions](#) to make our programs more efficient. A *function* is a reusable block of code that we can call throughout our program to complete a certain task.

You may recognize the term “functions” since we’ve already implemented some in this course. The `println()` function which we’ve used to output values as well as the `main()` function in which all of our code has been written so far are both Kotlin standard library defined functions. In this lesson, we’ll venture outside of the `main()` function and write our own user-designed functions.

User-designed functions are functions we, the developers, create in our programs in order to accomplish a specific task. There are many ways to create and use functions making them an extremely versatile tool.

In this lesson, we’ll focus our efforts on how to write and invoke user-designed functions.

10.1 Introduction to Functions Instructions

Take a look at the image to the right. After spending a lot of time camping, the task of starting a campfire each night has become cumbersome for Codey. Luckily, Function Forest is known to be the habitat of a very smart bird that can remember a set of tasks and repeat them anytime they are called.

Codey teaches one of these birds all the steps needed to make a campfire: finding rocks to contain the fire, collecting kindling, and starting a flame. Now whenever Codey wants to start a campfire, they simply call upon the bird.



10.2 Creating and Calling a Function

In order to declare a function, we'll need to understand the anatomy of a basic function.

A function header contains valuable information about a function including its name, arguments, and its return type.

```
fun funcName(argName: Type): returnType {  
    Function Body  
}
```

- The fun keyword is used to declare a function.
- The function name is used to call the function throughout the program.
- Inside the parentheses are arguments- pieces of data fed to the function.
- The return type declares the type of data the function will return; it is also optional to include.
- Following the function header is the function's body where instructions are contained.

We'll cover arguments and return types later in this lesson. For now, we'll focus our attention on creating a basic function that outputs a value whenever we call it.

Let's create a function called greeting() that prints "Hello friend!":

```
fun greeting() {  
    println("Hello friend!")  
}
```

A function will only execute when it is called or invoked. While the function greeting() will exist outside the main() function, we will call greeting() within main() by writing the function's name followed by parentheses:

```
fun greeting() {  
    println("Hello friend!")  
}
```

```
fun main() {
```

```
// Invoke the function
greeting()
}
// Prints: Hello friend!
```

Anytime we call `greeting()`, "Hello, friend!" will be outputted to the terminal.

10.2 Creating and Calling a Function Instructions

1.

Create a function named `smares()` that outputs the following statements using `println()`:

- "Roast a marshmallow."
- "Place marshmallow on a graham cracker."
- "Place chocolate on marshmallow."
- "Put a new graham cracker on chocolate."
- "Enjoy!"

2.

Invoke `smares()` in the `main()` function.

3.

Optional:

Call `smares()` again. Note how time is saved by calling a function instead of writing the instructions again.

```
// Write your code below
```

```
fun main() {  
    // Write more code below  
  
}
```

OUTPUT

Roast a marshmallow.

Place marshmallow on a graham cracker.

Place chocolate on marshmallow.

Put a new graham cracker on chocolate.

Enjoy!

10.3 Arguments

Arguments are pieces of data we can feed to our [functions](#) in order to produce dynamic results. We can include as many arguments as the function needs.

Think of the `println()` function. When we use it, we give it a single argument: a `String` value. This value is then passed and outputted to the terminal.

To pass arguments into a function, we need to add parameters to our function header. Parameters are the names given to data being passed into a function. For example:

```
fun calculateForce(mass: Int, acceleration: Int) {  
    var force = mass * acceleration  
    println("The force is $force Newtons.")  
}
```

In the parentheses of the function header, we added two parameters: mass and acceleration.

- Each parameter is separated by a comma (,).
- The parameter starts with its name, followed by a colon (:), and then the parameter type.

The parameters can then be referenced within the function body like a variable. The value of each parameter is determined by the arguments used when invoking the function:

```
calculateForce(5, 12)
```

When `calculateForce()` is called, the values 5 and 12 are passed as arguments into the function. The value of the parameter `mass` will be 5 and the value of `acceleration` will be 12. Invoking the function with these argument values will give us the following output:

The force of this object is 60 Newtons.

An important concept to understand with functions is scope. Scope dictates the environment in which a variable can be accessed within a program. If we were to try and reference the variable `force` anywhere in our program outside of the `calculateForce()` function, we would receive an error because the variable only exists within that function.

10.3 Arguments Instructions

1.

Create a function called `getSpeed()` that accepts two `Int` arguments: distance and time.

Inside the function, create a variable called `speed` that is equal to the value of distance / time.

Use `println()` and a String template to output the following statement:

[speed] meters per second

2.

Invoke the `getSpeed()` function inside `main()`.

Set the argument value of distance to 10 and the value of time to 3.

// Write your code below

```
fun main() {
```

```
    // Write more code below
```

```
}
```

OUTPUT

3 meters per second

10.4 Named and Default Arguments

We can go further with arguments by naming them or setting default parameter values.

To improve the readability of our code, we can name our arguments when invoking the function. For example, with our `calculateForce()` function, we can add the names of arguments to our function call:

```
fun calculateForce(mass: Int, acceleration: Int) {  
    var force = mass * acceleration  
    println("The force is $force Newtons.")  
}
```

```
fun main() {  
    // Invoke the function with named arguments.  
    calculateForce(acceleration = 12, mass = 5)  
    // Prints: The force is 60 Newtons.  
}
```

Note that by naming our arguments when we invoke a function, we do not need to place the arguments in the same order as they appear in the function header.

Default arguments provide values to function arguments if none are given when the function is called. To implement this, we add an assignment operator and a value after the argument in the function header, like so:

```
fun greetCustomer(name: String = "Customer") {  
    println("Hello, $name.")  
}
```

In the `greetCustomer()` function, we set the default value of the argument name to "Customer".

If we invoke this function with a String value, that value will become the value of name; however, if no argument is given, name will have a value of "Customer":

```
greetCustomer("Cynara") // Prints: Hello, Cynara.  
greetCustomer() // Prints: Hello, Customer.
```

10.4 Named and Default Arguments Instructions

1.

An online shop has a special coupon code to get 15% off a customer's final purchase.

Create a function called `getPrice()` that accepts a Double type argument named `price` as well as a String argument called `couponCode` that has a default value of "None".

Leave the body empty for now.

2.

Inside the function, declare a Double type variable called `finalPrice`.

Then, create an if/else expression.

- If the value of `couponCode` is "save15", set `finalPrice` to `price * .85`.
- In the else expression, set `finalPrice` to `price`.

Outside the conditional, use `println()` and a String template to output the following statement:

The total price is [finalPrice].

3.

Inside the main() function, invoke getPrice().

When invoking getPrice(), use named arguments to give price a value of 48.0 and couponCode a value of "save15".

4.

Optional:

Invoke getPrice() for a second time. This time, only give the program a single argument: 48.0.

Observe what happens when no argument for couponCode is given.

// Write your code below

```
fun main() {
```

```
    // Write more code below
```

```
}
```

OUTPUT

The total price is 40.8.

10.5 Return Statements

The [functions](#) in the previous exercises produced output via `println()` statements contained within its code body; however, we also have the ability to return a value from a function.

A *return statement* returns a value from a function and passes it back to where the function was invoked. This value can then be used throughout our program.

If we want a function to return a value, we must add the return type to the function header as well as a return statement in the body. Take a look at the following example where we create a function that takes in a list of `Ints` and returns an `Int` value:

```
fun listSum(myList: List<Int>): Int {  
    var sum = 0  
    // iterate through each value and add it to sum  
    for (i in myList) {  
        sum += i  
    }  
    // return statement  
    return sum  
}
```

- The return type describes the data type of the value being returned and is stated after the parentheses and a colon `:` in the function header.
- The keyword `return` is used to declare a return statement. In this case, `sum` is the value being returned.
- Any lines of code that exist after the return statement will not be executed.

Previously, we invoked functions by calling them on their own line of code; however, we can also invoke a function and assign it to a variable for later use.

For example, we can set `total` to the return value of `listSum()` after sending it a list of `Int` values (`billsToPay`):

```
var billsToPay = mutableListOf(44, 29, 104, 79)
```

```
// Set total to the return value of listSum().
```

```
var total = listSum(billsToPay)
println("Your bill total is $total dollars.")
```

This will output the following:

Your bill total is 256 dollars.

10.5 Return Statements Instructions

1.

Following a recipe for vanilla cake, you are asked to add .75 ounces of vanilla extract; however, you only have a teaspoon to measure the ingredients.

Create a function called `ozToTsp()` that takes a `Double` type argument named `oz` and returns a `Double` type value.

Inside the function, create a variable `tsp` whose value is `oz` multiplied by 6. Use a `return` statement to return the value of `tsp`.

2.

Inside `main()`, create a variable called `tspNeeded` and set its value to `ozToTsp()` with an argument of `.75`.

Use `println()` and a `String` template to output the following statement:

```
// Write your code below
```

```
fun main() {
```

```
// Write more code below
```

```
}
```

OUTPUT

You will need 4.5 tsp of vanilla extract for this recipe.

10.6 Single Expression Functions

If one of our [functions](#) contains only a single expression, we can write the function out with shorthand syntax. This syntax allows us to create a function using only a single line of code.

For example, assume we create a function that returns a given number (num) to the power of 2. Using the normal function syntax, our code would look like this:

```
fun powerOf2(num: Int): Int {  
    return num * num  
}
```

Using the single expression function syntax, we can rewrite the above code by removing the brackets ({ and }) as well as the return keyword:

```
fun powerOf2(num: Int): Int = num * num
```

Note how the return value (num * num) is prepended with an assignment operator (=).

We can shorten this code even more by removing the return type. Using type inference, the compiler can deduce the type of the return value, making our function look like this:

```
fun powerOf2(num: Int) = num * num
```

10.6 Single Expression Functions Instructions

1.

Create a single expression function called `pyramidVolume()`.

The function will accept three `Int` types arguments: `l`, `w`, and `h` and return the value of $(l * w * h) / 3$.

2.

Inside `main()`, create a variable called `volume` and set its value to `pyramidVolume()` with the following arguments in their respective order: length, width, height.

Use `println()` and a String template to output the following:

```
// Write your code below
```

```
fun main() {  
    var length = 5  
    var width = 8  
    var height = 14
```

```
// Write more code below
```

```
}
```

OUTPUT

The volume of this pyramid is 186.

10.7 Function Literals

To simplify how we define [functions](#), we can use function literals. A *function literal* is an unnamed function that can be treated as a value: we can call them, assign them as [variables](#), pass them as arguments, and return them from a function as we could with any other value.

We'll be going over two types of function literals: anonymous functions and lambda expressions. While anonymous functions and lambda expressions serve the same purpose, they use a slightly different syntax.

To start, let's create an anonymous function stored in a variable called `quotient` that returns the quotient of two numbers:

```
val quotient = fun(num1: Int, num2: Int): Double {  
    return num1 / num2  
}
```

- The anonymous function is contained in a variable called `quotient`.
- The `fun` keyword is placed after the assignment operator and is not followed by a name (hence the anonymous).
- `quotient` has a [function type](#) of `(Int, Int) -> Double`.

The function type describes the argument type and return type of the anonymous function. The arguments are contained within parentheses while the return type

is stated after `->`. In this case, the anonymous function takes in two Integers and returns a single Double.

To call this anonymous function, we'll simply call `quotient` like we would any other variable:

```
println(quotient(10, 5)) // Prints: 2
```

A lambda expression is very similar to an anonymous function; however, its concise syntax makes lambda expressions a more popular option amongst programmers. It's suggested to use a lambda expression over an anonymous function when creating a function with only one expression.

If we were to recreate our `quotient` variable using a lambda expression, our code would look like this:

```
val quotient = { num1: Int, num2: Int -> num1 / num2 }
```

```
println(quotient(10, 5)) // Prints: 2
```

- The lambda expression is contained within brackets: `{` and `}`
- We state the parameter names as well as their [data types](#).
- The return value `num1 / num2` is placed after the `->` symbol.
- Including the return type is optional because the compiler can use type inference to deduce the data type of the return value.

To learn more about anonymous functions and lambda expressions, check out the [Kotlin Language Guide](#).

10.7 Function Literals Instructions

1.

Working at a landscape company, a customer asks for grass to cover the area of their triangle-shaped yard.

Create a variable called `area` and set its value to an anonymous function that accepts two `Int` arguments: `base` and `height` and returns an `Int` value.

The return value should equal $(\text{base} * \text{height}) / 2$.

Use `println()` to output the value of `area(15, 19)`.

2.

The customer comes back and asks for you to build a fence around their garden.

Create a variable called `perimeter` and set its value to a lambda expression that accepts two `Int` values called `side1` and `side2` and returns an `Int` value.

The return value will equal the value of `side1 + side2` times 2.

Use `println()` to output the value of `perimeter(15, 24)`.

```
fun main() {  
    // Write your code below
```

```
}
```

OUTPUT

142

78

10.8 Review

Congratulations on completing this lesson. Let's go over what we learned:

- A function is a reusable block of code that can be invoked throughout a program.
 - In order to execute a function, it must be called somewhere in the program.
 - An argument is a piece of data fed to the function.
 - Arguments have the option to be named and/or given default values.
 - A return statement returns a piece of data from the function to where the function was called.
 - [Functions](#) with only a single expression can be written with a single line of code using shorthand syntax.
 - A function literal is an unnamed function that can be passed as an expression.
-

Recursion

Learn how to create recursive functions in Kotlin.

What is Recursion?

Recursion is the process in which a function calls itself until a condition is met. This concept is similar to a **while** loop in which a body of code will be executed repeatedly until a condition is met.

Recursion is an advanced topic when it comes to functions and programming; it often takes time and practice until a programmer is comfortable with creating and using recursive functions so don't feel discouraged if it doesn't all stick right away.

Breaking Down a Recursive Function

In a recursive function, a base case (usually an **if** expression) is created to check for a condition. If the condition is **true**, then the function will return a value. If the condition is **false**, the function will return a function call to itself with a new set of argument values; this is known as a *recursive call*. The function will continue to call upon itself until the base case is reached.

Let's use a recursive function to get the sum of a group of numbers from **num** until **0**:

```
fun recursiveSum(num: Int): Int {  
    // base case  
    if (num == 0) {  
        return num  
    } else {  
        // recursive call  
        return num + recursiveSum(num - 1)  
    }  
}
```

If we were to call this function with an argument of **4**, we would get a return value of **10**. Let's try and break down how that worked. Until the parameter **num** has a value of **0**, the code in the **else** block, **return num + recursiveSum(num - 1)**, will be executed like so:

- **recursiveSum(4)** returns **4 + recursiveSum(3)**.
- **recursiveSum(3)** returns **3 + recursiveSum(2)**.

- **recursiveSum(2)** returns **2 + recursiveSum(1)**.
- **recursiveSum(1)** returns **1 + recursiveSum(0)**.

Once we call **recursiveSum(0)**, the base case is reached because **num** is **0**. Now, we need to return the value from each call starting from the last call made.

- **recursiveSum(0)** returns **0**.
- **recursiveSum(1)** returns **1 + 0** which equals **1**.
- **recursiveSum(2)** returns **2 + 1** which equals **3**.
- **recursiveSum(3)** returns **3 + 3** which equals **6**.
- **recursiveSum(4)** returns **4 + 6** which equals **10**.

Note: Like a **while** loop, a recursive function can run infinitely if we do not create a base case that eventually stops the recursive calls.

Let's practice creating our own recursive function:

Coding question

Happy New Year! Define a function called **countDown()** that accepts a single `Int` argument called **secondsToMidnight**.

Inside **countDown()**, create an **if/else** expression to represent the base case and recursive call.

- Base case: If **secondsToMidnight** is equivalent to **0**, use **println()** to output **"Happy New Year!"**
- Recursive call: In the **else** block, use **println()** to output the value of **secondsToMidnight**. Then, on the following line, recursively call **countDown()** with an argument of **secondsToMidnight - 1**.

Inside **main()**, invoke **countDown()** with an argument value of **10**.

Tail-Recursive Functions

Recursive functions run the risk of [stack overflow](#). Stack overflow occurs in recursive functions when too much data is being used to store the values created in each recursive call and can cause our program to crash.

Luckily for us, Kotlin has a solution for this: tail-recursive functions. When using a tail-recursive function, the compiler can optimize the code to prevent stack overflow.

Let's recreate the **recursiveSum()** function from earlier. To declare a tail recursive function, prepend **tailrec** to the **fun** keyword in the header:

```
tailrec fun recursiveSum(num: Int, sum: Int = 0): Int {  
    return if (num == 0) sum  
    else recursiveSum(num - 1, sum + num)  
}  
  
fun main() {  
    println(recursiveSum(num = 10)) // Prints: 55  
}
```

We added a second argument, **sum**, to store the final return value. For a recursive function to be considered for tail recursion, the final operation in the function must be a call to itself. In this case, the final instruction is **recursiveSum(num - 1, sum + num)**.

The code **return if (num == 0) sum** translates to the base case being **num == 0** and the return value being **sum**. The code **else recursiveSum(num - 1, sum + num)** is the recursive call that changes the argument **num** to decrease by **1** while adding the value of **num** to the eventual return value **sum**.

Let's practice creating tail-recursive functions:

Coding question

In math, a [factorial function](#) multiplies a positive number from its value down to 1.

For example **4!** represents $4 * 3 * 2 * 1$.

Define a tail-recursive function called **factorial()** that accepts an `Int` argument named **num** and an `Int` argument named **product** that has a default value of **1**. The function will return an `Int` value.

Inside the function, return the value of **product** when the base case, **num == 1**, is reached. Otherwise, recursively call the function with the arguments **num - 1** and **num * product**, respectively.

Inside the **main()** function, create a variable called **factorialValue** and set its value to **factorial(10)**. Then, use **println()** to output **factorialValue**.

Wrap up

Great job reaching the end of this article. Let's go over what we learned:

- Recursion is the process of a function calling itself within the function's code block.
- Every recursive function needs a base case and a recursive call.
- The base case defines what condition must be met in order to exit the recursive function while the recursive call is used to call the function with a new set of arguments.
- A tail-recursive function is a more efficient way of producing a recursive function because it prevents stack overflow.

Recursion is one of the more advanced topics in this course; don't feel discouraged if the concept didn't click right away. Recursion isn't a fundamental programming concept, but once you get the hang of it, it can be a valuable tool for coding projects, interview questions, and more.

10. QUIZ

1. What is a function?

- a) A function is a block of code that is repeated a set number of times.
- b) A function is an unordered collection of unique elements.
- c) A function is a named, reusable block of code that can be called throughout a program.
- d) A function is a true or false value.

2. Which of the following choices is optional to include in a basic function header?

- a) The fun keyword.
- b) The return type.
- c) A function name.
- d) Parentheses after the function name.

3. What will the following program output?

```
fun greetFriend(name: String = "Buddy") {  
    println("Hi there, $name!")  
}
```

```
fun main() {  
    greetFriend("Kamala")  
}
```

- a) Hi there, Kamala!
- b) Hi there, Buddy!
- c) Hi there, \$name!
- d) This code will have no output.

4. What will the following program output?

```
fun numberOfOddNumbers(myList: List<Int>): Int {  
    var numOdd = 0
```

```

for (i in myList) {
    if (i % 2 != 0) {
        numOdd++
    }
}
return numOdd
}

```

```

fun main() {
    var listOfNumbers = mutableListOf(2, 3, 6, 7, 9, 10)
    println(numberOfOddNumbers(myList = listOfNumbers))
}

```

- a) numOdd
- b) [2, 3, 6, 7, 9, 10]
- c) 3
- d) There is no output.

5. Fill in the code to have findDifference() return the variable difference which equals the value of num1 subtracted by num2.

```

fun findDifference(num1: Int, num2: Int): _____ {
    var difference = num1 _____
    _____ difference
}

```

- -
- return
- String
- num4
- *
- num2
- Int

- sendBack

Click or drag and drop to fill in the blank

ANS:

```
fun findDifference(num1: Int, num2: Int): Int {
    var difference = num1 - num2
    return difference
}
```

6. Is the following example an anonymous function or a lambda expression?

```
var negate = { num: Int -> num * -1 }
```

- a) Lambda expression
- b) Anonymous function

7. Recreate the following function using the syntax for a single expression function.

```
fun findRemainder(num1: Int, num2 Int): Int {
    return num1 % num2
}
```

_____ findRemainder(_____: Int, _____: Int) _____

- return
- fun
- num1
- num1 % num2
- func
- num1 * num2
- ==

- num2
- =

Click or drag and

8. What is one major difference between a function literal and a basic function?

- a) Function literals and basic functions are the same.
- b) A function literal does not have a function name.
- c) Function literals never take in an argument.
- d) Function literals never return a value.

9. Based on the following function header, what data type will the following function return?

```
fun isEvenOrOdd(num: Int): Boolean {
  // Code goes here
}
```

- a) Int
- b) Boolean
- c) This function returns no value.
- d) String

10. Which will the following program output?

```
fun getForce(mass: Int = 10, acceleration: Int) {
  var force = mass * acceleration
  println("The force is $force Newtons.")
}
```

```
fun main() {
  getForce(acceleration = 2, mass = 5)
}
```

- a) The force is with you.

- b) The force is 10 Newtons.
- c) The force is 2 Newtons.
- d) The force is 20 Newtons.

7.PROJECT

DIY Functions

Up until this point, whenever we have wanted to gather information about specific collections in our programs, we have relied on Kotlin standard library functions to give us the answers.

For example, we've used the `sum()` function to tell us the sum of the values in a list and the `average()` function to return the mean. We never had to fully understand how the program calculates this information; all we had to do was call a function on a collection and use whatever value the function returned.

In this project, we're going to dive in and create five user-defined functions to collect and gather information about a list of numbers.

As you go through the project, check off the boxes next to each instruction as you complete them.

Note: We'll be using the [bash terminal](#) to collect [user input](#). For instructions on running the program in the bash terminal, see [this](#) exercise or the hint in instruction 17.

Gathering a List of Values

1.

First, we need to gather some `Int` values to store in a list. We'll use this list as our arguments in the functions we'll create later.

Create a function header called `getListOfNumbers()` that takes no parameters and returns a list of `Int` values.

Leave the body blank for now.

2.

Inside `getListOfNumbers()`, declare an empty mutable list called `myList`.

Then, create a loop that iterates 7 times and does the following:

- Ask the user to enter a number
- Append the user's answer as an `Int` to `myList`. The value must be stored as an `Int` to prevent errors in your code.

Outside the loop, return the value of `myList`.

Note: See the **Hint** to find out how to store user input as an `Int`.

3.

Inside the `main()` function, create a variable called `values` and set its value to `getListOfNumbers()`.

Output the value of `values`.

Finding the Maximum Value

4.

We're going to create a function that returns the largest number in a list.

First, declare a function called `findMax()` that takes in a list of `Ints` as an argument and returns an `Int` value.

Leave the body empty for now.

5.

Inside the function, do the following:

- Declare a variable called `largestNumber` and set it to the zeroeth index of the list argument.
- Loop through each element of the list.
- Inside the loop, use a conditional to check if the element's value is larger than `largestNumber`; if it is, set `largestNumber` to that value.
- Return the value of `largestNumber`.

6.

Inside `main()` declare a variable called `largestValue` and set its value to `findMax()` with values as its argument.

Then output the following message:

The largest number is [`largestValue`].

Finding the Minimum Value

7.

Create a function called `findMin()`.

This function will accept a list of `Ints` and return a single `Int`.

Leave the body empty for now.

8.

Inside the function, do the following:

- Declare a variable called `smallestNumber` and set it to the first element of the list argument.
- Loop through each element in the list argument.
- If the element's value is less than `smallestNumber`, set `smallestNumber` to that value.
- Return `smallestNumber`.

9.

Declare a variable called `smallestValue` in `main()` and set its value to `findMin()`.

Use `values` as the argument in the function call.

Then, output the following statement:

The smallest number is [`smallestValue`].

Finding the Average

10.

Let's create a function that returns the average value of a list of numbers.

Create a function called `findAverage()` that accepts a list of `Ints` as an argument and returns an `Int` value.

Leave the body empty for now.

11.

Inside the function, do the following:

- Declare a variable called `sum` and set its value to 0.
- Iterate through each value in the list argument.
- Inside the loop, add the value of the list variable to `sum`.

- Outside the loop, return the value of sum divided by the length of the list argument.

12.

Inside `main()`, declare a variable called `average` and set its value to `findAverage()` using `values` as the list argument.

Then, output the following statement:

The average is `[average]`.

Finding if a List Contains a Value

13.

For our last function, we'll create a function that checks if a specific number exists inside of a list.

Create a function called `checkIfListContains()` that accepts a list of `Ints` as an argument as well as an `Int` value. The function should return a `Boolean` value.

Leave the body empty for now.

14.

Inside the function, do the following:

- Loop through each element of the list argument.
- Inside the loop, check if the value of the loop variable is equal to the `Int` argument.
- If the condition is true, return `true`.
- Outside the loop, return `false`.

15.

Let's get a value to check for from the user.

Inside `main()` use a print statement to ask the user to enter a number to check for.

Store this value as an `Int` variable called `numToFind`.

Note: See the **Hint** to find out how to store user input as an `Int`.

16.

Also in `main()`, declare a variable called `containsValue` and set its value to `checkIfListContains` with the arguments values and `numToFind`.

If the value of `containsValue` is true, output that the value exists within the list.

Else, output that the value does not exist within the list.

Checking Your Code

17.

Nice job creating the functions!

Run the code in the bash terminal to test your program.

See the **Hint** for instructions on running the code in the bash terminal.

18.

Run your code several times with different values to make sure the output works correctly.

See the **Hint** for some test values.

Optional

19.

If you want to keep challenging yourself, check out these additional tasks:

- Create a function that returns the difference between the largest and smallest number in the list.
- Create a function that takes in a list of numbers and returns a list with all the values squared.

```
// Write your code below
```

```
fun main() {  
    // Write more code below  
  
}
```

CONCEPT REVIEW

Functions

A function is a named, reusable block of code that can be called and executed throughout a program.

A function is declared with the `fun` keyword, a function name, parentheses containing (optional) arguments, as well as an (optional) return type.

To call/invoke a function, write the name of the function followed by parentheses.

```
fun greet() {  
    println("Hey there!")  
}
```

```
fun main() {  
    // Function call  
    greet() // Prints: Hey there!  
}
```

Function Arguments

In Kotlin, an argument is a piece of data we can pass into a function when invoking it.

To pass data into a function, the function's header must include parameters that describe the name and data type of the incoming data. If a function is declared with parameters, then data must be passed when the function is invoked. We can include as many parameters as needed.

```
fun birthday(name: String, age: Int) {  
    println("Happy birthday $name! You turn $age today!")  
}
```

```
fun main() {  
    birthday("Oscar", 26) // Prints: Happy birthday Oscar! You turn 25 today!  
    birthday("Amarah", 30) // Prints: Happy birthday Amarah! You turn 30 today!  
}
```

Default Arguments

We can give arguments a default value which provides an argument an automatic value if no value is passed into the function when it's invoked.

```
fun favoriteLanguage(name, language = "Kotlin") {  
    println("Hello, $name. Your favorite programming language is $language")  
}
```

```
fun main() {  
    favoriteLanguage("Manon") // Prints: Hello, Manon. Your favorite programming  
    language is Kotlin  
  
    favoriteLanguage("Lee", "Java") // Prints: Hello, Lee. Your favorite programming  
    language is Java  
}
```

Named Arguments

We can name our arguments when invoking a function to provide additional readability.

To name an argument, write the argument name followed by the assignment operator (=) and the argument value. The argument's name must have the same name as the parameter in the function being called.

By naming our arguments, we can place arguments in any order when the function is being invoked.

```
fun findMyAge(currentYear: Int, birthYear: Int) {  
    var myAge = currentYear - birthYear  
    println("I am $myAge years old.")  
}
```

```
fun main() {  
    findMyAge(currentYear = 2020, birthYear = 1995)  
    // Prints: I am 25 years old.  
    findMyAge(birthYear = 1920, currentYear = 2020)  
    // Prints: I am 100 years old.  
}
```

Return Statement

In Kotlin, in order to return a value from a function, we must add a return statement to our function using the return keyword. This value is then passed to where the function was invoked.

If we plan to return a value from a function, we must declare the return type in the function header.

```
// Return type is declared outside the parentheses
```

```

fun getArea(length: Int, width: Int): Int {
    var area = length * width

    // return statement
    return area
}

fun main() {
    var myArea = getArea(10, 8)
    println("The area is $myArea.") // Prints: The area is 80.
}

```

Single Expression Functions

If a function contains only a single expression, we can use a shorthand syntax to create our function.

Instead of placing curly brackets after the function header to contain the function's code block, we can use an assignment operator = followed by the expression being returned.

```

fun fullName(firstName: String, lastName: String) = "$firstName $lastName"

fun main() {
    println(fullName("Ariana", "Ortega")) // Prints: Ariana Ortega
    println(fullName("Kai", "Gittens")) // Prints: Kai Gittens
}

```

Function Literals

Function literals are unnamed functions that can be treated as expressions: we can assign them to variables, call them, pass them as arguments, and return them from a function as we could with any other value.

Two types of function literals are anonymous functions and lambda expressions.

```
fun main() {  
    // Anonymous Function:  
    var getProduct = fun(num1: Int, num2: Int): Int {  
        return num1 * num2  
    }  
    println(getProduct(8, 3)) // Prints: 24  
  
    // Lambda Expression  
    var getDifference = { num1: Int, num2: Int -> num1 - num2 }  
    println(getDifference(10, 3)) // Prints: 7  
}
```

11. CLASSES

11.1 Introduction to Classes

Kotlin, much like Java, is an object-oriented programming language. The term, “object-oriented” refers to a language that is made up of [objects](#) and can utilize various methodologies such as [encapsulation](#), [polymorphism](#), and [inheritance](#). In this lesson, we’ll make a brief introduction into the world of objects with a tool known as [classes](#).

Classes allow us to create blueprints of everyday objects and then utilize this blueprint to create as many unique instances of it as we’d like. Assume we’re building a program for a car dealership. We’d need to store some of the following basic characteristics for each car:

- Model
- Year
- Color

We can use a class to create a blueprint with all of these properties from which we would then create each individual car. Each unique car would be an instance of this class, also known as an *object*.

With each new concept we’ve learned about in this course, we’ve made our programs more dynamic and customizable, and classes will be no different. With classes, we have the ability to mimic real-world world objects in our code and tailor them to our liking.

In this lesson, we’ll focus on creating classes, generating individual objects from them, and exploring their available functionalities.

11.1 Introduction to Classes Instructions

Take a look at the image to the right.

With all of your help and hard work thus far, Codey's finally en route to DevCity! DevCity utilizes blueprints (classes) to generate each one of its skyscrapers. Notice how although each blueprint is the same, the final result, or object, is different.

Click **Up Next** to move on and help Codey understand and master the concept of classes as they journey through the city.



11.2 Creating a Class

Now let's practice writing our own [classes](#). A Kotlin class is declared at the top of a Kotlin file, outside the `main()` function, with the `class` keyword followed by its name:

```
class Name {  
    // class body  
}
```

The name of a class should always start with an upper case letter and use the camel case form. The class body is contained within curly braces and will hold all of the properties and [functions](#) related to that class. We will take a closer look at class functions in a later exercise.

Note: It is optional for a Kotlin class to contain a body, thus if the class has no body, we can omit its curly braces, resulting in the following declaration:

```
class Name
```

Let's revisit our real-world, Car example. Suppose we create the following class:

```
class Car
```

Our blueprint is complete! But it's not of much use to us empty. Any car possesses several properties including the year it was made, its model name, and color. Let's add these:

```
class Car {  
    val year = 2011  
    val model = "Jeep"  
    val color = "Blue"  
}
```


We've created the properties, year, model, and color and initialized them with values. Now any time we create a car object from this class, it will possess the properties, year, model and color and hold their respective values. In the next exercise, we'll learn how to create [objects](#) from a class. For now, let's practice with the Kotlin class syntax. 🚗

11.2 Creating a Class Instructions

1.

In **Book.kt**, declare an empty class called Book.

2.

Add the following properties and the values they hold:

- pages with the value, 320
- title with the value, "Harry Potter and the Sorcerer's Stone"
- author with the value, "J.K. Rowling"

// Write your code above 📖

```
fun main() {  
  
}
```

11.3 Creating an Instance

Now that we've created our first class, let's create an instance of it. An *instance* of a class resembles an object that is a member of that given class.

Recall the Car class from the previous exercise:

```
class Car {  
    val year = 2011  
    val model = "Jeep"  
    val color = "Blue"  
}
```

To make an instance of this class, we'd utilize the following syntax:

```
val myCar = Car()
```

- We've declared the name of our instance/object to be myCar. This name is arbitrary and is up to the developer's choosing.
- Following the assignment operator, we've invoked the Car class with the name of the class followed by parentheses - the same syntax we use when invoking a function.
- With this line of code, the myCar instance now possesses all the properties and their respective values that were specified within the class body.

Just like we've done with Kotlin's built-in properties such as .length and .size, we can utilize the dot notation and append our class property names on our instance/object and retrieve their values.

```
println(myCar.year) // Prints: 2011  
println(myCar.model) // Prints: Jeep  
println(myCar.color) // Prints: Blue
```

Voila! You've created your first object. Think about what you've learned about [classes](#) and their properties. What is one way that we might need to improve upon with this code? Follow the checkpoints below to find out.

11.3 Creating an Instance Instructions

1.

Within the `main()` function of **Building.kt**, create an instance of the `Building` class and assign it to the variable, `residentialBuilding`.

2.

Create three print statements that use dot notation to output each value of the instance, `residentialBuilding`.

3.

Optional:

If you create another instance of the `Building` class, say `commercialBuilding`, and output its property values, what do you expect to see?

Check the **Hint** for a clue.

```
class Building {  
    val yearBuilt = 2016  
    val height = 400 // Feet  
    val type = "Limestone"  
}
```

```
fun main() {  
    // Write your code below ?  
  
}
```

OUTPUT

2016

400

Limestone

11.4 Primary Constructor

Looking to customize your instances? You've come to the right place!

Kotlin [classes](#) provide us with something called a primary constructor that allows us to declare properties and initialize them when we instantiate an object.

Let's see the shorthand syntax:

```
class Name(val x: String, val y: String)
```

We've now introduced a pair of parentheses in the class header that define properties, x and y. Previously, we would've had something like this:

```
class Name {  
    val x = "some value"  
    val y = "another value"  
}
```

Each property would already have its default value set, but with primary constructors, we give our program the ability to set the values when an instance is created.

Refactoring our Car example from the previous exercise to utilize a primary constructor in the header, we get the following line of code:

```
class Car(val year: Int, val model: String, val color: String)
```

Neat, right? Let's see it in action.

```
val myCar = Car(2011, "Jeep", "Blue")  
val friendsCar = Car(2015, "Mazda", "Red")
```

Now we've successfully created unique instances of our Car class and customized their values:

```
println(myCar.year)    // Prints: 2011  
println(friendsCar.year) // Prints: 2015
```

11.4 Primary Constructor Instructions

1.

In **Person.kt**, add a class, Person that contains a primary constructor with the following properties:

- name with the type, String
- age with the type, Int
- favoriteColor with the type, String

2.

Within the main() function, create an instance of the Person class called me and set the values for each property that resemble your characteristics.

On the following line, create another instance of the Person class called myFriend and set the values for each property that resemble their characteristics.

3.

Use a String template along with a println() statement for each object and output the following text:

[name] is [age] years old and ____ favorite color is [favoriteColor].

Make sure to replace the ____ with the appropriate pronoun (my, her, his, or their) to match the instance. Check the hint for guidance.

// Write your class below

```
fun main() {  
    // Create your instances below  
  
  
  
}
```

OUTPUT

Maggie is 26 years old and my favorite color is Green.

Jose is 25 years old and their favorite color is Red.

11.5 The Init Block

The primary constructor does not contain any code other than the properties and their specified [data types](#). What if we'd like to calculate a person's age using the year they were born? We can do this with an init block.

Init is short for *initializer* and allows us to add more logic to our [classes](#).

Suppose the following class, Mascot:

```
class Mascot(val name: String, val platform: String, val yearCreated: Int)
```

We'd like to use the year the mascot was created to calculate its age. We can do so by adding this logic within an init block:

```
class Mascot(val name: String, val platform: String, val yearCreated: Int) {  
    var age: Int  
  
    init {  
        age = 2020 - yearCreated  
        println("$name is a $platform mascot and is $age years old. ")  
    }  
}
```

- We've created a variable, age, within the class that we'll use to store the value of the calculation.
- Within the init block, we add our mathematical expression followed by a println() statement to output the final result.

Now when we call the class, we should expect to see a String output:

```
val codey = Mascot("Codey", "Codecademy", 2018)  
// Prints: Codey is a Codecademy mascot and is 2 years old.
```

11.5 The Init Block Instructions

1.

In **Employee.kt**, we've created a class that accepts several properties in its primary constructor.

On the first line within the class, declare a variable, `fullName` and in it store the combination of `firstName` and `lastName` with a single space in between.

You can use String concatenation or String templates.

2.

Below, create an init block and in its body, set up the following logic:

- if the `yearsWorked` is greater than 1, then output the text, "[full name] is eligible for a raise!"
- else output the text, "[full name] is not eligible for a raise just yet."

Make sure to use the variable you created in the previous step and String templates.

3.

Navigate to the `main()` function and create an instance, `projectManager`, of the class `Employee` and pass in the following values:

- `firstName` is "Maria"
- `lastName` is "Gonzalez"
- `yearsWorked` is 2

```
class Employee(val firstName: String, val lastName: String, val yearsWorked: Int) {
```



```
// Write your code below
```

```
}
```

```
fun main() {
```

```
// Create an instance below 👤📁👜
```

```
}
```

OUTPUT

Maria Gonzalez is eligible for a raise!

11.6 Member Functions

Now let's explore how to add even more functionality to our [classes](#) with [functions](#). A function declared within a Kotlin class is known as a member function.

Member functions possess the same qualities and abilities as the user-designed functions you've learned about in the previous lesson. The syntax is the same as well. Some things to keep in mind:

- When an instance of a class is created, the object has access to any and all functions that exist within the class body.
- Unlike the code within the init block which executes automatically, the code within the functions will only execute when the function is called.

For example,

```
class Cat(val name: String, val breed: String) {

    fun speak() {
        println("Meow!")
    }
}
```

We've created a simple class, Cat, with a primary constructor and a single member function. We can now create an instance of the class as so and call the speak() function:

```
var myCat = Cat("Garfield", "Persian")
myCat.speak() // Prints: Meow!
```

11.6 Member Functions Instructions

1.

Inspect the class, Dog, on the right. Inside the init block, we looped through the list property, competitionsParticipated. Now, let's add a member function below the init block that shows off one of the dog tricks done in the competition.

Add a member function called, speak(), that outputs the following text:

[name] says: Woof!

2.

Create an instance of the Dog class within the main() function. Call the instance, maxie. Pass in the following custom values for the properties:

- name: "Maxie"

- breed: "Poodle"
- list of competitions should include "Westminster Kennel Club Dog Show" and "AKC National Championship"

Run your code. Then, call the `speak()` function on `maxie` on the following line.

```
class Dog(val name: String, val breed: String, val competitionsParticipated:
List<String>) {
```

```
    init {
        for (comp in competitionsParticipated) {
            println("$name participated in $comp.")
        }
    }
}
```

```
// Write your function below
```

```
}
```

```
fun main() {
```

```
    // Write your instance below 🐶
```

```
}
```

OUTPUT

Maxie participated in Westminster Kennel Club Dog Show.

Maxie participated in AKC National Championship.

Maxie says: Woof!

11.7 Review

Excellent work! In this lesson, we've learned the following concepts:

- A class is an object-oriented concept which resembles a blueprint for individual [objects](#). A class can contain properties and [functions](#) and is defined using the class keyword followed by a name and optional body.
- An instance is a member of a class created by calling the class name followed by a pair of parentheses, (), and any necessary arguments.
- We can use dot syntax to access the value of each class property.
- A class can have properties with default values or ones that are customizable with the help of primary constructors.
- A primary constructor allows us to set each property value when we instantiate an object.
- The init block gets invoked with every instance creation and is used to add logic to the class.
- A function declared within a Kotlin class is known as a *member function* of that class. In order to invoke a member function, we must call the function on an instance of the class.

11.7 Review Instructions

You did it! 🎉 You've completed the last lesson in the Learn Kotlin course. You should be immensely proud of yourself and your accomplishments. This was no easy feat, but you've made it to DevCity and can proudly say you're on track to becoming a master of the Kotlin language.

Run the code on the right and enter the following two commands for a special message from Codey:

```
kotlinc Congratulations.kt -include-runtime -d Congratulations.jar
```

```
java -jar Congratulations.jar
```

```
class Congratulations(val name: String) {

    init {
        println("Dear $name,")
    }

    fun message() {
        println("Thank you for embarking on this journey with me. I couldn't have gotten through the treacherous waters of Conditional Creek or hiked up the Looping mountains without your help. You now have the fundamental knowledge to go on and continue your Kotlin journey in the world of android application, web development or more. The world is yours. Good luck! ")
    }

    fun signOut() {
        println("With ❤️ , Codey and the Codecademy Team.")
    }
}

fun main() {
```

```
println("Please enter your name...")  
var learnerName = readLine()  
var toLearner = Congratulations(learnerName.toString())  
toLearner.message()  
toLearner.signOut()
```

OUTPUT

```
$ kotlinc Congratulations.kt -include-runtime  
-d Congratulations.jar  
$ java -jar Congratulations.jar  
Please enter your name...  
Vishva  
Dear Vishva,  
Thank you for embarking on this journey with  
me. I couldn't have gotten through the treach  
erous waters of Conditional Creek or hiked up  
the Looping mountains without your help. You  
now have the fundamental knowledge to go on  
and continue your Kotlin journey in the world  
of android application, web development or m  
ore. The world is yours. Good luck!  
With ♥ , Codey and the Codecademy Team.  
$ □
```

CONCEPT REVIEW

Classes

A class is an object-oriented concept which resembles a blueprint for individual objects. A class can contain properties and functions and is defined using the class keyword followed by a name and optional body.

If the class does not have a body, its curly braces can be omitted.

// A class with properties that contain default values

```
class Student {  
    var name = "Lucia"  
    var semester = "Fall"  
    var gpa = 3.95  
}
```

// Shorthand syntax with no class body

```
class Student
```

Class Instances

A new instance of a class is created by calling the class name followed by a pair of parentheses () and any necessary arguments.

When creating an instance of a class, we must declare a variable in which we intend to store our instance and assign it equal to the class call. Once the instance has been created, we can use dot syntax to access and retrieve the value of each property.

// Class

```
class Student {  
    var name = "Lucia"
```

```

var semester = "Fall"

var gpa = 3.95
}

fun main() {
    var student = Student() // Instance
    println(student.name)    // Prints: Lucia
    println(student.semester) // Prints: Fall
    println(student.gpa)     // Prints: 3.95
}

```

Primary Constructor

A primary constructor defines each property value within a class header and allows us to then set unique values when the object is instantiated.

To create a primary constructor using the shorthand syntax, we can follow the name of the class with a pair of parentheses () inside of which each property is defined and assigned a data type.

```

class Student(val name: String, val gpa: Double, val semester: String, val
estimatedGraduationYear: Int)

```

```

fun main() {
    var student = Student("Lucia", 3.95, "Fall", 2022)
    println(student.name)    // Prints: Lucia
    println(student.gpa)     // Prints: 3.95
    println(student.semester) // Prints: Fall
    println(student.estimatedGraduationYear) // Prints: 2022
}

```


Init Blocks

The init block gets invoked with every instance that's created and is used to add logic to the class. The init keyword precedes any member functions and is followed by a pair of curly braces.

```
class Student(val name: String, val gpa: Double, val semester: String, val
estimatedGraduationYear: Int) {

    init {
        println("$name has ${estimatedGraduationYear - 2020} years left in college.")
    }
}
```

```
fun main() {
    var student = Student("Lucia", 3.95, "Fall", 2022) // Prints: Lucia has 2 years left
in college.
}
```

Member Functions

A function declared within a class is known as a member function of that class. In order to invoke a member function, we must call the function on an instance of the class.

```
class Student(val name: String, val gpa: Double, val semester: String, val
estimatedGraduationYear: Int) {

    init {
        println("$name has ${estimatedGraduationYear - 2020} years left in college.")
    }
}
```

```
// Member Function
fun calculateLetterGrade(): String {
    return when {
        gpa >= 3.0 -> "A"
        gpa >= 2.7 -> "B"
        gpa >= 1.7 -> "C"
        gpa >= 1.0 -> "D"
        else -> "E"
    }
}
}
```

// When an instance is created and the function is called, the when expression will execute and return a letter grade

```
fun main() {
    var student = Student("Lucia", 3.95, "Fall", 2022) // Prints: Lucia has 2 years left
    in college.

    println("${student.name}'s letter grade is ${student.calculateLetterGrade()}") //
    Prints: Lucia's letter grade is A.
}
```

11. QUIZ

1. What is the difference between the execution of code within an **init** block and within a member function?

- a) The **init** block needs to be called meanwhile all member functions execute automatically when an instance is created.
- b) There is no difference.
- c) There must always be a member function within a class meanwhile an **init** block is optional.
- d) **The init block gets executed automatically when an instance is created, meanwhile, a member function needs to be specifically called upon.**

2. Fill in the code to access property values for **rose** and **orchid**.

```
class Flower(val symmetrical: Boolean, val petals: Int, val color: String)
```

```
val rose = Flower(true, 5, "Red")
```

```
val orchid = Flower(true, 3, "Purple")
```

```
println(rose._____) // Prints: 5
```

```
println(_____._____) // Prints: Purple
```

```
println(rose _____ symmetrical) // Prints: true
```

- color
- orchid
- Int
- flower
- ,

- rose
- petals
- 5
- .

Click or drag and drop to fill in the blank

ANS:

```
println(rose.petals)    // Prints: 5
println(orchid.color)   // Prints: Purple
println(rose.symmetrical) // Prints: true
```

3. What is a class?

- A class is a type of Kotlin collection that stores unique elements.
- A class repeats a body of code based on a given number or condition.
- A class is a blueprint that contains properties and functions from which we can instantiate objects.**
- A class is a reusable block of code that performs a certain task.

4. Fill in the blanks of the class, **Doctor**, and its instance in order to produce the output:

Rakesh Gupta specializes in Cardiology and has 40 years of experience.

```
_____ Doctor(val name: _____, val specialty: String, val yearGraduated: _____) {
    var yearsExperience: Int

    _____ {
        yearsExperience = 2020 - yearGraduated
        println("$name specializes in $specialty and has $yearsExperience years of
experience.")
    }
}
```

```
}
```

```
val doctor = _____("Rakesh Gupta", _____, _____)
```

- Doctor
- 1980
- Patient
- 10
- Boolean
- fun
- Double
- Int
- class
- String
- "Cardiology"
- init

Click or drag and drop to fill in the blank

ANS:

```
class Doctor(val name: String, val specialty: String, val yearGraduated: Int) {  
    var yearsExperience: Int  
  
    init {  
        yearsExperience = 2020 - yearGraduated  
        println("$name specializes in $specialty and has $yearsExperience years of  
experience.")  
    }  
}
```

```
val doctor = Doctor("Rakesh Gupta", "Cardiology", 1980)
```

5. True or False: A primary constructor allows us to set and customize each property value when we create an object.

- a) False
- b) True**

6. Which choice properly creates an instance of the following class, **Droid**?

```
class Droid {  
    val artificialIntelligence = true  
    val name = "R2-D2"  
    val height = 1.08 // meters  
}
```

- a) `val myDroid = Droid`
- b) `val myDroid = Droid.class`
- c) `val myDroid = Droid()`**
- d) `val myDroid = new Class()`

8.PROJECT

Codey's Calculator

Codey's made it to DevCity: a place full of Android applications, web development, and other endless implementations of the Kotlin language!

In this project, we'll utilize our knowledge of Kotlin classes to help Codey build a basic calculator application that many of us have built-into our smartphones.

Our calculator will be able to:

- Find the sum +
- Find the difference -
- Find the product ×
- Find the quotient ÷
- Find the power ^

A variation of this project exists in our [Learn Java](#) course so if you're learning Kotlin with a Java background, we highly recommend comparing the syntaxes and code implementations once you've completed this project to better understand the similarities and differences between the two languages.

Let's get to calculating! 🧮

Create a Class

1.

The first step in creating Codey's individual calculator is to generate a blueprint.

In **Calculator.kt**, declare the class, Calculator, that accepts a name property within its primary constructor.

Note: Running your code will result in the following error:

could not find or load main class CalculatorKt

This error exists due to the absence of a main() function. If you'd like to get rid of this error now, go ahead and also create an empty main() function below the class. Otherwise, move on to the next step.

2.

Within the class body, add an init block that will output the owner of the calculator using the name property.

Add the Calculator's Functions

3.

Now, let's give our calculator the ability to make calculations.

First, declare a member function called, add(), which should calculate and return the sum of two Integer values.

4.

Next, implement the subtract() function which should calculate and return the difference between two Integer values.

5.

Another arithmetic function most commonly used on a calculator is multiplication.

Implement the multiply() function which will calculate and return the product between two Integer values.

6.

The opposite calculation of multiplication is division. Add a member function, divide(), which should calculate and return the quotient between two Integer values.

The member function should return an error message if the user tries to divide by 0.

7.

Lastly, create the `power()` function which should calculate and return the first parameter raised to the power of the second.

Think of how you can implement this within the function body. Give it a shot in the code editor. Move on to the next step for guided instruction.

8.

Within the body of the `power()` function, declare a new variable, `result`, and assign it the value of 1.

To achieve the power calculation, use a looping technique to iterate over each value, from 1 to `param1`. Within the body of the loop, reassign the value of `result` to be `result times param2`.

Return `result` from the function.

Get to Calculating

9.

Now that we've implemented several arithmetic functionalities, let's spring our calculator to action. If you haven't already done so, below the class, declare a `main()` function.

In it, create a variable name of your choosing that will store an instance of the `Calculator` class. Pass in the name, "Codey" in the class call, and run your code. You should see the output statement from the `init` block.

10.

On the following line, call the `add()` function on your instance and pass in two Integer values consecutively. Wrap your code in a `println()` statement to see the output.

11.

Use the same syntax to call each member function on your instance on a separate line and test their calculations. Make sure to wrap each line of code in a `println()` statement to see the output.

Optional Challenges

12.

Excellent work! If you're up for a challenge, check out these additional tasks:

- To mimic a real-world calculator, we can format our output to also include the original calculation. `12 -> 5 + 7 = 12`.
- Having the code as is without any additional output, we can refactor some of the member functions to utilize a single expression syntax and shorten the lines of code that they span.
- We can add another member function to the class that you'd like your calculator to perform.

OUTPUT

Codey's Calculator ?

12

34

Error! Dividing by zero is not allowed.

2